

**ĐẠI HỌC TRÀ VINH
TRƯỜNG KỸ THUẬT VÀ CÔNG NGHỆ**



ĐỒ ÁN TỐT NGHIỆP

**XÂY DỰNG HỆ THỐNG HỖ TRỢ TÌM KIẾM VÀ
GỢI Ý PHÒNG TRỌ SINH VIÊN TRÊN ĐỊA BÀN
TỈNH VĨNH LONG THEO HƯỚNG KẾT HỢP**

Giảng viên hướng dẫn: **THS. VÕ THÀNH C**

Sinh viên thực hiện: **LÊ KHÁNH ĐĂNG**

Mã số sinh viên: **110122047**

Lớp: **DA22TTA**

Khoá: **2022**

Vĩnh Long, tháng ... năm 2026

**ĐẠI HỌC TRÀ VINH
TRƯỜNG KỸ THUẬT VÀ CÔNG NGHỆ**



ĐỒ ÁN TỐT NGHIỆP

**XÂY DỰNG HỆ THỐNG HỖ TRỢ TÌM KIẾM VÀ
GỢI Ý PHÒNG TRỌ SINH VIÊN TRÊN ĐỊA BÀN
TỈNH VĨNH LONG THEO HƯỚNG KẾT HỢP**

Giảng viên hướng dẫn: **THS. VÕ THÀNH C**

Sinh viên thực hiện: **LÊ KHÁNH ĐĂNG**

Mã số sinh viên: **110122047**

Lớp: **DA22TTA**

Khoá: **2022**

Vĩnh Long, tháng ... năm 2026

MỤC LỤC

CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI.....	1
1.1. Lý do chọn đề tài	1
1.2. Mục tiêu nghiên cứu.....	1
1.2.1. Mục tiêu tổng quát	1
1.2.2. Mục tiêu cụ thể.....	2
1.3. Đối tượng nghiên cứu.....	2
1.4. Phạm vi nghiên cứu.....	3
1.4.1. Phạm vi không gian	3
1.4.2. Phạm vi chức năng.....	4
1.5. Phương pháp nghiên cứu.....	4
1.5.1. Phương pháp khảo sát và thu thập dữ liệu	4
1.5.2. Phương pháp phân tích và thiết kế hệ thống.....	5
1.5.3. Phương pháp xây dựng hệ thống	5
1.5.4. Phương pháp kiểm thử và đánh giá	6
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT	7
2.1. Tổng quan hệ thống tìm kiếm phòng trọ	7
2.1.1. Khái niệm hệ thống tìm kiếm phòng trọ	7
2.1.2. Đặc điểm hệ thống tìm kiếm phòng trọ	8
2.1.3. Thực trạng các hệ thống tìm kiếm phòng trọ hiện nay	9
2.2. Tổng quan hệ thống gợi ý	9
2.2.1. Khái niệm hệ thống gợi ý.....	9
2.2.2. Vai trò của hệ thống gợi ý.....	10
2.2.3. Các phương pháp gợi ý phổ biến	10
2.3. Hệ thống gợi ý kết hợp (Hybrid Recommendation System)	12
2.4. Công nghệ ReactJS	13
2.4.1 Giới thiệu ReactJS	13
2.4.2. Ưu điểm	14
2.4.3. Hạn chế	14
2.5. Công nghệ NodeJS và ExpressJS.....	15
2.5.1. Giới thiệu NodeJS và ExpressJS.....	15
2.5.2. Ưu điểm	15

2.5.3. Hạn chế	16
2.6. Cơ sở dữ liệu MongoDB	16
2.6.1. Giới thiệu MongoDB	16
2.6.2. Ưu điểm	16
2.6.3. Hạn chế	17
2.7. Công nghệ Socket.io	17
2.7.1. Giới thiệu Socket.io	17
2.7.2. Ưu điểm	18
2.7.3. Hạn chế	18
2.8. Leaflet và OpenStreetMap	18
2.8.1. Giới thiệu Leaflet và OpenStreetMap	18
2.8.2. Ưu điểm	19
2.8.3. Hạn chế	19
CHƯƠNG 3. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG	20
3.1. Phân tích yêu cầu hệ thống	20
3.1.1. Yêu cầu chức năng	20
3.1.2. Yêu cầu phi chức năng	21
3.2. Biểu đồ Use Case	22
3.2.1. Use Case sinh viên	22
3.2.2. Use Case chủ trọ	23
3.2.3. Use Case quản trị viên	24
3.3. Thiết kế kiến trúc hệ thống	25
3.3.1. Kiến trúc tổng thể	25
3.3.2. Kiến trúc Frontend	27
3.3.3. Kiến trúc Backend	28
3.3.4. Kiến trúc hệ thống gợi ý	30
3.3.4.1. Kiến trúc tổng thể hệ thống gợi ý	30
3.3.4.2. Kiến trúc gợi ý phòng tương tự	36
3.3.4.3. Kiến trúc gợi ý theo tiêu chí tìm kiếm	38
3.3.4.4. Kiến trúc gợi ý cá nhân hóa	41
3.3.4.5. Kiến trúc gợi ý cộng đồng	44
3.4. Thiết kế cơ sở dữ liệu	47

3.4.1. Mô hình dữ liệu tổng thể.....	47
3.4.2. Collection Users.....	48
3.4.3. Collection Rooms.....	48
3.4.4. Collection Appointments	50
3.4.5. Collection Favorites	50
3.4.6. Collection Comments	51
3.4.7. Collection Interactions.....	51
3.4.8. Collection Reports	52
3.4.9. Collection Conversations	53
3.4.10. Collection Messages	53
3.4.11. Collection Notifications	54
CHƯƠNG 4. KẾT QUẢ NGHIÊN CỨU	55
4.1. Môi trường phát triển	55
4.1.1. Công cụ phát triển.....	55
4.1.2. Môi trường triển khai.....	55
4.2. Kết quả xây dựng hệ thống gợi ý phòng trọ	56
4.3. Giao diện hệ thống	58
4.3.1. Trang chủ	58
4.3.2. Trang tìm kiếm phòng trọ	60
4.3.3. Trang chi tiết phòng trọ	60
4.3.4. Trang lịch hẹn xem phòng	62
4.3.5. Trang nhắn tin	62
4.3.6. Trang quản lý phòng trọ.....	63
4.3.7. Trang quản trị hệ thống.....	63
4.4. Kiểm thử hệ thống.....	64
4.4.1. Kiểm thử chức năng.....	64
4.4.2. Đánh giá kết quả kiểm thử	68
CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	69
5.1. Kết quả đạt được	69
5.2. Hạn chế của hệ thống	69
5.3. Hướng phát triển.....	70
DANH MỤC TÀI LIỆU THAM KHẢO	71

LỜI MỞ ĐẦU

Trong những năm gần đây, nhu cầu tìm kiếm phòng trọ của sinh viên ngày càng gia tăng cùng với sự phát triển của các trường đại học, cao đẳng và các khu vực đô thị. Tuy nhiên, việc tìm kiếm một phòng trọ phù hợp với nhu cầu về giá cả, vị trí, diện tích và tiện ích vẫn còn gặp nhiều khó khăn do thông tin phân tán, thiếu tính cập nhật và chưa được cá nhân hóa.

Sự phát triển của công nghệ thông tin và các hệ thống trực tuyến đã tạo điều kiện thuận lợi cho việc xây dựng các nền tảng hỗ trợ tìm kiếm nhà ở. Đặc biệt, các hệ thống gợi ý thông minh có khả năng phân tích dữ liệu và đưa ra các đề xuất phù hợp với nhu cầu của từng người dùng, góp phần nâng cao hiệu quả tìm kiếm và cải thiện trải nghiệm sử dụng.

Xuất phát từ thực tiễn đó, đề tài “xây dựng hệ thống hỗ trợ tìm kiếm và gợi ý phòng trọ sinh viên trên địa bàn tỉnh vĩnh long theo hướng kết hợp” được thực hiện nhằm xây dựng một hệ thống hỗ trợ sinh viên tìm kiếm phòng trọ một cách thuận tiện, nhanh chóng và hiệu quả. Hệ thống không chỉ cung cấp các chức năng quản lý và tìm kiếm phòng trọ mà còn tích hợp mô hình gợi ý phòng trọ dựa trên thông tin phòng, vị trí địa lý và hành vi người dùng để đưa ra các đề xuất phù hợp.

Đề tài được xây dựng trên nền tảng công nghệ web với Frontend sử dụng ReactJS, Backend sử dụng Node.js và ExpressJS, cơ sở dữ liệu MongoDB cùng các công nghệ hỗ trợ khác như Socket.IO và Leaflet. Kết quả của đề tài góp phần tạo ra một giải pháp hỗ trợ sinh viên trong quá trình tìm kiếm nơi ở, đồng thời là cơ sở để tiếp tục nghiên cứu và phát triển các hệ thống gợi ý thông minh trong tương lai.

LỜI CẢM ƠN

Trước hết, em xin bày tỏ lòng biết ơn sâu sắc đến quý thầy cô trong khoa đã tận tình giảng dạy, truyền đạt kiến thức và tạo điều kiện thuận lợi cho em trong suốt quá trình học tập tại trường cũng như trong quá trình thực hiện đồ án này.

Em xin chân thành cảm ơn thầy Võ Thành C giảng viên hướng dẫn đã luôn dành thời gian, tâm huyết để hướng dẫn, góp ý và giải đáp những thắc mắc của em trong suốt quá trình nghiên cứu và thực hiện đề tài. Những ý kiến quý báu của thầy đã giúp em hiểu rõ hơn về nội dung đề tài, từ đó có thể hoàn thiện đồ án theo đúng mục tiêu và yêu cầu đề ra.

Em cũng xin gửi lời cảm ơn đến gia đình và bạn bè đã luôn động viên, khích lệ và hỗ trợ em trong quá trình học tập cũng như khi thực hiện đồ án. Sự quan tâm và giúp đỡ của mọi người là nguồn động lực to lớn giúp em vượt qua những khó khăn và hoàn thành đề tài đúng tiến độ.

Mặc dù đã rất cố gắng, nhưng do kiến thức và thời gian có hạn, đồ án không tránh khỏi những thiếu sót. Em rất mong nhận được những ý kiến đóng góp quý báu từ quý thầy cô và các bạn để đề tài có thể được hoàn thiện hơn trong tương lai.

Cuối cùng, em xin chân thành cảm ơn và kính chúc quý thầy cô cùng các bạn luôn dồi dào sức khỏe và thành công trong công tác và học tập.

Trân trọng,

Vĩnh Long, ngày tháng năm

Sinh viên thực hiện

(Ký tên và ghi rõ họ tên)

Lê Khánh Đăng

NHẬN XÉT
(Của giảng viên hướng dẫn trong đề án, khoá luận của sinh viên)

[illegible]

Giảng viên hướng dẫn
(ký và ghi rõ họ tên)

Võ Thành C

UBND TỈNH VĨNH LONG
ĐẠI HỌC TRÀ VINH

CỘNG HOÀ XÃ HỘI CHỦ NGHĨA VIỆT NAM
Độc lập – Tự do – Hạnh Phúc

BẢN NHẬN XÉT ĐỒ ÁN, KHÓA LUẬN TỐT NGHIỆP
(Của giảng viên hướng dẫn)

Họ và tên sinh viên: MSSV:

Ngành: Khóa:

Tên đề tài:

.....

.....

Họ và tên Giáo viên hướng dẫn:

Chức danh: Học vị:

NHẬN XÉT

1. Nội dung đề tài:

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

4. Điểm mới đề tài:

.....

.....

.....

.....

.....

5. Giá trị thực trên đề tài:

.....

.....

.....

.....

.....

.....

.....

7. Đề nghị sửa chữa bổ sung:

.....

.....

.....

.....

.....

.....

.....

8. Đánh giá:

.....

.....

.....

.....

Vĩnh Long, ngày tháng năm 20...
Giảng viên hướng dẫn
(Ký & ghi rõ họ tên)

Võ Thành C

NHẬN XÉT
(Của giảng viên chấm trong đề án, khoá luận của sinh viên)

[illegible]

Giảng viên chấm
(ký và ghi rõ họ tên)

UBND TỈNH VĨNH LONG
ĐẠI HỌC TRÀ VINH

CỘNG HOÀ XÃ HỘI CHỦ NGHĨA VIỆT NAM
Độc lập – Tự do – Hạnh phúc

BẢN NHẬN XÉT ĐỒ ÁN, KHÓA LUẬN TỐT NGHIỆP
(Của cán bộ chấm đồ án, khóa luận)

Họ và tên người nhận xét:

Chức danh: Học vị:

Chuyên ngành:

Cơ quan công tác:

Tên sinh viên:

Tên đề tài đồ án, khóa luận tốt nghiệp:

.....
.....

I. Ý KIẾN NHẬN XÉT

1. Nội dung:

.....
.....
.....
.....
.....
.....
.....
.....
.....
.....

2. Điểm mới các kết quả của đồ án, khóa luận:

.....
.....
.....

3. Ứng dụng thực tế:

.....
.....
.....
.....
.....

.....

.....

II. CÁC VẤN ĐỀ CẦN LÀM RÕ
(Các câu hỏi của giáo viên phản biện)

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

III. KẾT LUẬN

(Ghi rõ đồng ý hay không đồng ý cho bảo vệ đồ án khóa luận tốt nghiệp)

....., ngày tháng năm 20...

Người nhận xét

(Ký & ghi rõ họ tên)

DANH MỤC HÌNH

Hình 3.1. Biểu đồ Use Case Sinh viên	23
Hình 3.2. Biểu đồ Use Case chủ trọ	24
Hình 3.3. Biểu đồ Use Case quản trị viên	25
Hình 3.4. Kiến trúc tổng thể hệ thống	26
Hình 3.5. Kiến trúc Frontend.....	28
Hình 3.6. Kiến trúc Backend	29
Hình 3.7 Kiến trúc tổng thể hệ thống gợi ý.....	31
Hình 3.8. Mô hình dữ liệu tổng thể	47
Hình 4.1. Chức năng gợi ý phòng trọ tương tự trên hệ thống	56
Hình 4.2. Chức năng gợi ý phòng trọ cá nhân hóa.....	57
Hình 4.3. Chức năng gợi ý phòng trọ theo tiêu chí tìm kiếm.....	58
Hình 4.4. Giao diện trang chủ	59
Hình 4.5. Giao diện trang tìm kiếm phòng trọ	60
Hình 4.6. Giao diện trang chi tiết phòng trọ.....	61
Hình 4.7. Giao diện trang lịch hẹn xem phòng	62
Hình 4.8. Giao diện trang nhắn tin	62
Hình 4.9. Giao diện trang quản lý phòng trọ.....	63
Hình 4.10. Giao diện trang quản trị hệ thống.....	63
Hình 4.11. Kiểm thử API đăng nhập bằng Postman	65
Hình 4.12. Kiểm thử API tìm kiếm phòng trọ bằng Postman	65
Hình 4.13. Kiểm thử API Lưu phòng yêu thích bằng Postman	66
Hình 4.14. Kiểm thử API Đặt lịch hẹn xem phòng bằng Postman	66
Hình 4.15. Kiểm thử API Gợi ý cá nhân hóa bằng Postman.....	67
Hình 4.16. Kiểm thử API Gợi ý phòng trọ tương tự bằng Postman	67
Hình 4.17. Kiểm thử API Gợi ý dựa trên hoạt động cộng đồng bằng Postman.....	68

DANH MỤC BẢNG

Bảng 3.1. Nguồn dữ liệu sử dụng trong hệ thống gợi ý	31
Bảng 3.2. Vector nhị phân của thuộc tính loại phòng	32
Bảng 3.3. Số chiều của 21 thuộc tính	33
Bảng 3.4. Quy trình xây dựng tập ứng viên	37
Bảng 3.5. Dữ liệu đầu vào của thuật toán gợi ý theo tiêu chí tìm kiếm	39
Bảng 3.6. Trọng số của thuật toán gợi ý theo tiêu chí tìm kiếm	40
Bảng 3.7. Dữ liệu đầu vào của thuật toán gợi ý cá nhân hóa	41
Bảng 3.8. Trọng số các hành vi tương tác	42
Bảng 3.9. Trọng số của thuật toán gợi ý cá nhân hóa	43
Bảng 3.10. Dữ liệu đầu vào của thuật toán gợi ý cộng đồng	45
Bảng 3.11. Trọng số các hành vi cộng đồng	46
Bảng 3.12. Trọng số của thuật toán gợi ý cộng đồng	46
Bảng 3.13. Collection Users	48
Bảng 3.14. Collection Rooms	49
Bảng 3.15. Collection Appointments	50
Bảng 3.16. Collection Favorites	50
Bảng 3.17. Collection Comments	51
Bảng 3.18. Collection Interactions	52
Bảng 3.19. Collection Reports	52
Bảng 3.20. Collection Conversations	53
Bảng 3.21. Collection Messages	53
Bảng 3.22. Collection Notifications	54
Bảng 4.1. Kết quả kiểm thử chức năng	64

CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI

1.1. Lý do chọn đề tài

Trong những năm gần đây, cùng với sự phát triển của các cơ sở giáo dục đại học trên địa bàn tỉnh Vĩnh Long, số lượng sinh viên từ các địa phương khác đến học tập ngày càng tăng. Điều này dẫn đến nhu cầu tìm kiếm phòng trọ của sinh viên ngày càng lớn. Tuy nhiên, quá trình tìm kiếm phòng trọ hiện nay chủ yếu thông qua các kênh như mạng xã hội, hội nhóm trực tuyến, người quen giới thiệu hoặc môi giới. Các phương thức này còn tồn tại nhiều hạn chế như thông tin không được kiểm chứng, khó so sánh giữa các phòng trọ, mất nhiều thời gian tìm kiếm và tiềm ẩn nguy cơ gặp phải các tin đăng không chính xác.

Bên cạnh đó, phần lớn các hệ thống tìm kiếm phòng trọ hiện nay chỉ dừng lại ở việc cung cấp danh sách phòng trọ theo các tiêu chí cơ bản mà chưa hỗ trợ phân tích nhu cầu thực tế của người dùng để đưa ra các gợi ý phù hợp. Trong khi đó, sự phát triển của các hệ thống gợi ý thông minh đã được ứng dụng rộng rãi trong nhiều lĩnh vực như thương mại điện tử, giải trí và mạng xã hội, góp phần nâng cao trải nghiệm người dùng và tối ưu hóa quá trình tìm kiếm thông tin.

Xuất phát từ những thực tế trên, đề tài “Xây dựng hệ thống hỗ trợ tìm kiếm và gợi ý phòng trọ sinh viên tại tỉnh Vĩnh Long theo hướng kết hợp” được thực hiện nhằm xây dựng một hệ thống giúp sinh viên dễ dàng tìm kiếm phòng trọ phù hợp với nhu cầu cá nhân, đồng thời ứng dụng các phương pháp gợi ý thông minh để nâng cao chất lượng kết quả đề xuất. Hệ thống không chỉ hỗ trợ sinh viên trong việc tìm kiếm chỗ ở mà còn tạo môi trường kết nối hiệu quả giữa người thuê và chủ trọ, góp phần số hóa hoạt động tìm kiếm phòng trọ trên địa bàn tỉnh Vĩnh Long.

1.2. Mục tiêu nghiên cứu

1.2.1. Mục tiêu tổng quát

Xây dựng hệ thống website hỗ trợ tìm kiếm và gợi ý phòng trọ dành cho sinh viên tại tỉnh Vĩnh Long, giúp người dùng tìm kiếm phòng trọ nhanh chóng, thuận tiện và hiệu quả thông qua việc kết hợp các phương pháp gợi ý dựa trên nội dung, hành vi người dùng và vị trí địa lý nhằm nâng cao độ chính xác của kết quả đề xuất.

1.2.2. Mục tiêu cụ thể

- Khảo sát thực trạng tìm kiếm phòng trọ của sinh viên tại tỉnh Vĩnh Long và phân tích những khó khăn, hạn chế trong các phương thức tìm kiếm hiện nay.
- Phân tích yêu cầu hệ thống, xác định các nhóm người dùng chính gồm sinh viên, chủ trọ và quản trị viên cùng các chức năng tương ứng.
- Thiết kế và xây dựng hệ thống website hỗ trợ tìm kiếm phòng trọ với giao diện thân thiện, dễ sử dụng và tương thích trên nhiều thiết bị.
- Xây dựng chức năng đăng ký, đăng nhập và quản lý tài khoản người dùng.
- Xây dựng chức năng tìm kiếm và lọc phòng trọ theo nhiều tiêu chí như giá thuê, diện tích, khu vực, loại phòng, tiện ích, khoảng cách và trạng thái phòng.
- Xây dựng chức năng xem thông tin chi tiết phòng trọ, lưu danh sách yêu thích, bình luận và đánh giá phòng trọ.
- Xây dựng chức năng đặt lịch hẹn xem phòng và trao đổi trực tuyến giữa sinh viên và chủ trọ thông qua hệ thống nhắn tin thời gian thực.
- Xây dựng chức năng quản lý phòng trọ dành cho chủ trọ bao gồm đăng tin, cập nhật thông tin, quản lý trạng thái phòng và lịch hẹn.
- Xây dựng chức năng quản trị hệ thống nhằm quản lý người dùng, kiểm duyệt nội dung, xử lý báo cáo vi phạm và theo dõi hoạt động của hệ thống.
- Tích hợp bản đồ số để hiển thị vị trí phòng trọ, hỗ trợ tìm kiếm theo vị trí địa lý, tính khoảng cách và chỉ đường.
- Nghiên cứu và xây dựng hệ thống gợi ý phòng trọ theo hướng kết hợp dựa trên thông tin phòng trọ, hành vi tương tác của người dùng và vị trí địa lý nhằm đề xuất các phòng trọ phù hợp với nhu cầu của từng người dùng.
- Kiểm thử, đánh giá hiệu năng và mức độ đáp ứng yêu cầu thực tế của hệ thống trước khi triển khai sử dụng.

1.3. Đối tượng nghiên cứu

Đối tượng nghiên cứu của đề tài là hệ thống hỗ trợ tìm kiếm và gợi ý phòng trọ dành cho sinh viên trên nền tảng web. Đề tài tập trung nghiên cứu các phương pháp tổ

chức, quản lý và khai thác dữ liệu phòng trọ nhằm hỗ trợ người dùng tìm kiếm thông tin hiệu quả hơn.

Cụ thể, các đối tượng nghiên cứu bao gồm:

- Sinh viên có nhu cầu tìm kiếm phòng trọ phục vụ học tập và sinh hoạt.
- Chủ trọ có nhu cầu đăng tải và quản lý thông tin phòng trọ cho người thuê.
- Quản trị viên thực hiện nhiệm vụ quản lý người dùng, kiểm duyệt nội dung và vận hành hệ thống.
- Dữ liệu phòng trọ bao gồm thông tin về giá thuê, diện tích, loại phòng, tiện ích, vị trí địa lý, hình ảnh và trạng thái phòng.
- Các phương pháp gợi ý phòng trọ dựa trên nội dung, hành vi người dùng và vị trí địa lý.
- Các công nghệ phát triển hệ thống web hiện đại như ReactJS, NodeJS, ExpressJS, MongoDB, Socket.io và Leaflet.

Thông qua việc nghiên cứu các đối tượng trên, đề tài hướng đến việc xây dựng một hệ thống có khả năng hỗ trợ người dùng tìm kiếm và lựa chọn phòng trọ phù hợp với nhu cầu cá nhân một cách nhanh chóng và hiệu quả.

1.4. Phạm vi nghiên cứu

Do giới hạn về thời gian, nguồn lực và phạm vi thực hiện, đề tài tập trung nghiên cứu, thiết kế và xây dựng hệ thống hỗ trợ tìm kiếm và gợi ý phòng trọ dành cho sinh viên trên nền tảng web. Hệ thống tập trung vào các chức năng hỗ trợ quản lý thông tin phòng trọ, tìm kiếm, đặt lịch hẹn, nhấn tin và gợi ý phòng trọ dựa trên mô hình Hybrid Recommendation. Đề tài không đi sâu vào việc xây dựng ứng dụng di động, tích hợp thanh toán trực tuyến hoặc các chức năng nâng cao khác.

1.4.1. Phạm vi không gian

Đề tài được triển khai và thử nghiệm trên địa bàn tỉnh Vĩnh Long. Dữ liệu phòng trọ được xây dựng tập trung tại các khu vực có nhiều sinh viên học tập và sinh sống như khu vực gần các trường đại học, cao đẳng và trung tâm thành phố Vĩnh Long. Hệ thống được thiết kế theo hướng mở, cho phép mở rộng và áp dụng cho các địa phương khác trong tương lai.

1.4.2. Phạm vi chức năng

Trong phạm vi nghiên cứu, hệ thống được xây dựng cho ba nhóm người dùng gồm sinh viên, chủ trọ và quản trị viên.

Đối với sinh viên, hệ thống hỗ trợ đăng ký và đăng nhập tài khoản, quản lý thông tin cá nhân, tìm kiếm và lọc phòng trọ, xem thông tin chi tiết, hiển thị vị trí phòng trọ trên bản đồ, lưu phòng yêu thích, bình luận và đánh giá phòng trọ, đặt lịch hẹn xem phòng, nhắn tin với chủ trọ và nhận các gợi ý phòng trọ phù hợp.

Đối với chủ trọ, hệ thống cho phép quản lý thông tin cá nhân, đăng và cập nhật thông tin phòng trọ, quản lý trạng thái phòng, quản lý lịch hẹn xem phòng, phản hồi bình luận và trao đổi với người thuê thông qua hệ thống nhắn tin.

Đối với quản trị viên, hệ thống hỗ trợ quản lý tài khoản người dùng, kiểm duyệt tin đăng phòng trọ, kiểm duyệt bình luận và xử lý các báo cáo vi phạm nhằm đảm bảo chất lượng nội dung trên hệ thống.

Bên cạnh các chức năng quản lý và tìm kiếm, đề tài tập trung nghiên cứu và xây dựng hệ thống gợi ý phòng trọ theo mô hình Hybrid Recommendation nhằm nâng cao khả năng cá nhân hóa và hỗ trợ người dùng tìm kiếm các phòng trọ phù hợp với nhu cầu và sở thích.

1.5. Phương pháp nghiên cứu

1.5.1. Phương pháp khảo sát và thu thập dữ liệu

Phương pháp khảo sát và thu thập dữ liệu được sử dụng nhằm tìm hiểu thực trạng nhu cầu tìm kiếm phòng trọ của sinh viên trên địa bàn tỉnh Vĩnh Long cũng như các hình thức tìm kiếm phòng trọ đang được sử dụng hiện nay.

Quá trình khảo sát được thực hiện thông qua việc tìm hiểu thông tin từ các nhóm mạng xã hội, các website đăng tin phòng trọ, các diễn đàn sinh viên và trao đổi với một số sinh viên đang có nhu cầu thuê trọ. Thông qua quá trình khảo sát, tác giả xác định được những khó khăn thường gặp như thông tin phòng trọ thiếu chính xác, khó so sánh giữa các phòng, mất nhiều thời gian tìm kiếm và chưa có hệ thống hỗ trợ gợi ý phù hợp với nhu cầu cá nhân.

Bên cạnh đó, dữ liệu về phòng trọ, tiện ích, vị trí địa lý, giá thuê và các thông tin liên quan cũng được thu thập để phục vụ cho quá trình thiết kế cơ sở dữ liệu và xây dựng hệ thống gợi ý phòng trọ.

1.5.2. Phương pháp phân tích và thiết kế hệ thống

Sau khi hoàn thành quá trình khảo sát, đề tài tiến hành phân tích yêu cầu hệ thống nhằm xác định rõ các chức năng cần xây dựng và các đối tượng sử dụng hệ thống.

Các nhóm người dùng chính được xác định bao gồm sinh viên, chủ trọ và quản trị viên. Trên cơ sở đó, các yêu cầu chức năng và phi chức năng của hệ thống được xây dựng làm nền tảng cho quá trình thiết kế.

Để mô tả hoạt động của hệ thống, đề tài sử dụng các mô hình phân tích và thiết kế phổ biến như Use Case Diagram nhằm xác định các chức năng và tác nhân tham gia hệ thống; thiết kế kiến trúc tổng thể của hệ thống; thiết kế cơ sở dữ liệu thông qua mô hình thực thể - liên kết (ERD) và các thực thể dữ liệu phục vụ quá trình quản lý thông tin phòng trọ, người dùng, lịch hẹn, bình luận, tin nhắn và các chức năng liên quan.

Kết quả của giai đoạn này là bộ tài liệu phân tích và thiết kế hoàn chỉnh, làm cơ sở cho việc hiện thực hóa hệ thống trong các giai đoạn tiếp theo.

1.5.3. Phương pháp xây dựng hệ thống

Đề tài áp dụng phương pháp phát triển phần mềm theo hướng thực nghiệm, kết hợp giữa nghiên cứu lý thuyết và triển khai ứng dụng thực tế.

Hệ thống được xây dựng theo mô hình kiến trúc Client - Server, trong đó:

- Frontend được phát triển bằng ReactJS nhằm xây dựng giao diện người dùng hiện đại, trực quan và tương thích trên nhiều thiết bị.
- Backend được phát triển bằng NodeJS và ExpressJS để xử lý các chức năng nghiệp vụ, xác thực người dùng và cung cấp các dịch vụ API.
- MongoDB được sử dụng làm hệ quản trị cơ sở dữ liệu để lưu trữ và quản lý dữ liệu của hệ thống.
- Socket.io được sử dụng để xây dựng chức năng nhắn tin thời gian thực giữa sinh viên và chủ trọ.

- Leaflet kết hợp OpenStreetMap được sử dụng để hiển thị vị trí phòng trọ trên bản đồ, hỗ trợ tìm kiếm theo vị trí và tính khoảng cách.

- Hệ thống gợi ý phòng trọ được xây dựng theo hướng kết hợp (Hybrid Recommendation System) dựa trên thông tin phòng trọ, hành vi tương tác của người dùng và vị trí địa lý nhằm nâng cao chất lượng kết quả đề xuất.

Trong quá trình xây dựng, các chức năng được phát triển và hoàn thiện theo từng giai đoạn nhằm đảm bảo tính ổn định và khả năng mở rộng của hệ thống.

1.5.4. Phương pháp kiểm thử và đánh giá

Sau khi hoàn thành quá trình xây dựng, hệ thống được tiến hành kiểm thử nhằm phát hiện và khắc phục các lỗi phát sinh trước khi đưa vào sử dụng.

Các hoạt động kiểm thử bao gồm:

- Kiểm thử chức năng nhằm đánh giá mức độ đáp ứng của các chức năng theo yêu cầu đã đặt ra.

- Kiểm thử giao diện nhằm đảm bảo hệ thống hiển thị chính xác trên nhiều kích thước màn hình và thiết bị khác nhau.

- Kiểm thử hiệu năng nhằm đánh giá tốc độ xử lý và khả năng đáp ứng của hệ thống khi có nhiều người dùng truy cập.

- Kiểm thử hệ thống gợi ý nhằm đánh giá mức độ phù hợp của các kết quả đề xuất đối với nhu cầu của người dùng.

- Kết quả kiểm thử được sử dụng để đánh giá mức độ hoàn thiện của hệ thống, khả năng đáp ứng các yêu cầu thực tế và làm cơ sở đề xuất các hướng phát triển trong tương lai.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

2.1. Tổng quan hệ thống tìm kiếm phòng trọ

Trong bối cảnh nhu cầu thuê phòng trọ của sinh viên ngày càng gia tăng, việc ứng dụng công nghệ thông tin vào hoạt động tìm kiếm và quản lý phòng trọ trở thành một giải pháp cần thiết. Các hệ thống tìm kiếm phòng trọ trực tuyến được xây dựng nhằm hỗ trợ người thuê dễ dàng tiếp cận thông tin về phòng trọ, đồng thời giúp chủ trọ quảng bá thông tin cho thuê một cách nhanh chóng và hiệu quả.

Hiện nay, nhiều nền tảng tìm kiếm phòng trọ đã được triển khai trên website và ứng dụng di động, cho phép người dùng tìm kiếm theo các tiêu chí như vị trí, giá thuê, diện tích và tiện ích. Tuy nhiên, phần lớn các hệ thống vẫn còn một số hạn chế nhất định trong việc cá nhân hóa trải nghiệm người dùng và hỗ trợ tìm kiếm thông minh.

Do đó, việc nghiên cứu và xây dựng hệ thống tìm kiếm phòng trọ kết hợp với các phương pháp gợi ý thông minh sẽ góp phần nâng cao hiệu quả tìm kiếm, giúp người dùng tiết kiệm thời gian và lựa chọn được phòng trọ phù hợp với nhu cầu cá nhân.

2.1.1. Khái niệm hệ thống tìm kiếm phòng trọ

Hệ thống tìm kiếm phòng trọ là một hệ thống thông tin được xây dựng nhằm hỗ trợ người dùng tìm kiếm, tra cứu và tiếp cận các thông tin liên quan đến phòng trọ hoặc nhà cho thuê thông qua môi trường trực tuyến.

Hệ thống đóng vai trò là cầu nối giữa người có nhu cầu thuê phòng và người cho thuê phòng. Thông qua hệ thống, chủ trọ có thể đăng tải thông tin về phòng trọ như giá thuê, diện tích, địa chỉ, hình ảnh, tiện ích và trạng thái phòng. Người thuê có thể tìm kiếm, xem chi tiết và liên hệ với chủ trọ để trao đổi hoặc đặt lịch xem phòng.

Ngoài chức năng tìm kiếm cơ bản, các hệ thống hiện đại còn tích hợp thêm nhiều tính năng như bản đồ số, đánh giá phòng trọ, lưu danh sách yêu thích, nhấn tin trực tuyến và hệ thống gợi ý thông minh nhằm nâng cao trải nghiệm người dùng.

Về mặt tổng thể, hệ thống tìm kiếm phòng trọ giúp số hóa quá trình tìm kiếm nhà ở, giảm sự phụ thuộc vào các phương thức truyền thống và nâng cao khả năng tiếp cận thông tin cho cả người thuê và chủ trọ.

2.1.2. Đặc điểm hệ thống tìm kiếm phòng trọ

Hệ thống tìm kiếm phòng trọ là nền tảng hỗ trợ người dùng tra cứu, quản lý và tiếp cận thông tin phòng trọ thông qua môi trường trực tuyến. So với phương thức tìm kiếm truyền thống, hệ thống mang lại nhiều ưu điểm về khả năng truy cập, quản lý dữ liệu và hỗ trợ người dùng trong quá trình tìm kiếm. Một số đặc điểm nổi bật của hệ thống gồm:

Khả năng cung cấp thông tin nhanh chóng: Hệ thống cho phép người dùng truy cập và tìm kiếm thông tin phòng trọ mọi lúc, mọi nơi thông qua Internet. Các thông tin về phòng trọ được cập nhật thường xuyên, giúp người dùng dễ dàng tiếp cận các tin đăng mới và giảm thời gian tìm kiếm.

Hỗ trợ tìm kiếm theo nhiều tiêu chí: Hệ thống hỗ trợ tìm kiếm và lọc phòng trọ theo nhiều tiêu chí như giá thuê, diện tích, loại phòng, khu vực, khoảng cách và các tiện ích đi kèm. Việc kết hợp nhiều tiêu chí giúp người dùng nhanh chóng thu hẹp phạm vi tìm kiếm và lựa chọn được phòng trọ phù hợp với nhu cầu.

Khả năng hiển thị trực quan: Thông tin phòng trọ được trình bày dưới dạng hình ảnh kết hợp với bản đồ số, giúp người dùng dễ dàng quan sát, đánh giá điều kiện phòng trọ và khu vực xung quanh trước khi quyết định liên hệ hoặc đến xem trực tiếp.

Tăng tương tác giữa người thuê và chủ trọ: Hệ thống tích hợp các chức năng như nhắn tin, bình luận và đặt lịch hẹn xem phòng, tạo điều kiện để người thuê và chủ trọ trao đổi thông tin trực tiếp, góp phần nâng cao hiệu quả kết nối giữa các bên.

Khả năng lưu trữ và quản lý dữ liệu tập trung: Toàn bộ dữ liệu về người dùng, phòng trọ, lịch hẹn, bình luận và các hoạt động khác được lưu trữ tập trung trong cơ sở dữ liệu, giúp việc quản lý, cập nhật và khai thác thông tin được thực hiện thuận tiện và hiệu quả.

Khả năng tích hợp hệ thống gợi ý: Các hệ thống tìm kiếm phòng trọ hiện đại có thể tích hợp hệ thống gợi ý nhằm phân tích dữ liệu phòng trọ, thông tin người dùng và lịch sử tương tác để đề xuất các phòng trọ phù hợp. Điều này góp phần nâng cao khả năng cá nhân hóa, hỗ trợ người dùng tìm kiếm phòng trọ nhanh chóng và cải thiện trải nghiệm sử dụng.

2.1.3. Thực trạng các hệ thống tìm kiếm phòng trọ hiện nay

Hiện nay, việc tìm kiếm phòng trọ của sinh viên và người lao động chủ yếu được thực hiện qua các nền tảng trực tuyến như mạng xã hội, các hội nhóm cộng đồng, website đăng tin bất động sản và các ứng dụng tìm kiếm phòng trọ. Sự phát triển của các nền tảng này đã góp phần giúp người dùng dễ dàng tiếp cận thông tin phòng trọ, mở rộng phạm vi tìm kiếm và rút ngắn thời gian kết nối giữa người thuê và chủ trọ.

Nhiều hệ thống hiện nay đã cung cấp các chức năng hỗ trợ tìm kiếm theo nhiều tiêu chí như giá thuê, diện tích, khu vực, loại phòng và các tiện ích đi kèm. Bên cạnh đó, một số nền tảng còn tích hợp bản đồ số, hình ảnh thực tế, chức năng nhắn tin và đặt lịch hẹn, giúp người dùng thuận tiện hơn trong quá trình tìm kiếm và trao đổi với chủ trọ.

Tuy nhiên, qua khảo sát thực tế cho thấy các hệ thống hiện có vẫn còn tồn tại một số hạn chế. Thông tin phòng trọ chưa được kiểm chứng đầy đủ, nhiều tin đăng bị trùng lặp hoặc đã hết hiệu lực nhưng chưa được cập nhật kịp thời. Phần lớn các hệ thống chỉ hỗ trợ tìm kiếm dựa trên bộ lọc do người dùng lựa chọn mà chưa khai thác hiệu quả dữ liệu hành vi và sở thích để cá nhân hóa kết quả tìm kiếm. Khả năng gợi ý phòng trọ còn đơn giản, chưa phản ánh đầy đủ nhu cầu thực tế của từng người dùng.

Đối với sinh viên tại tỉnh Vĩnh Long, việc tìm kiếm phòng trọ vẫn chủ yếu thông qua các nhóm Facebook, Zalo hoặc thông tin giới thiệu từ người quen. Hình thức này giúp tiếp cận được nhiều nguồn thông tin nhưng thường mất nhiều thời gian để tìm kiếm, so sánh và xác minh tính chính xác của các tin trước khi đưa ra quyết định thuê phòng.

Từ thực trạng trên có thể thấy việc xây dựng một hệ thống hỗ trợ tìm kiếm và gợi ý phòng trọ kết hợp giữa dữ liệu phòng trọ, vị trí địa lý và hành vi người dùng là cần thiết. Hệ thống không chỉ giúp nâng cao hiệu quả tìm kiếm mà còn góp phần cá nhân hóa kết quả gợi ý, hỗ trợ người dùng lựa chọn được phòng trọ phù hợp một cách nhanh chóng và chính xác hơn.

2.2. Tổng quan hệ thống gợi ý

2.2.1. Khái niệm hệ thống gợi ý

Hệ thống gợi ý là một hệ thống có khả năng phân tích dữ liệu để đưa ra các đề xuất phù hợp với nhu cầu, sở thích hoặc hành vi của người dùng. Mục tiêu của hệ thống gợi ý

ý là hỗ trợ người dùng lựa chọn các đối tượng phù hợp trong một tập dữ liệu lớn, từ đó giảm thời gian tìm kiếm và nâng cao trải nghiệm sử dụng.

Trong quá trình hoạt động, hệ thống gợi ý thu thập và phân tích nhiều nguồn dữ liệu như thông tin đối tượng, hồ sơ người dùng, lịch sử tương tác và các dữ liệu liên quan khác. Dựa trên những thông tin này, hệ thống đánh giá mức độ phù hợp giữa người dùng và các đối tượng để xây dựng danh sách đề xuất.

Hiện nay, hệ thống gợi ý được ứng dụng rộng rãi trong nhiều lĩnh vực như thương mại điện tử, mạng xã hội, giáo dục, giải trí trực tuyến, du lịch và bất động sản.

2.2.2. Vai trò của hệ thống gợi ý

Sự phát triển của công nghệ thông tin và Internet đã làm cho lượng dữ liệu ngày càng gia tăng, khiến người dùng gặp nhiều khó khăn trong việc tìm kiếm và lựa chọn thông tin phù hợp. Hệ thống gợi ý được xây dựng nhằm giải quyết vấn đề quá tải thông tin bằng cách tự động đề xuất các đối tượng có khả năng đáp ứng tốt nhất nhu cầu của từng người dùng.

Đối với người dùng, hệ thống gợi ý giúp rút ngắn thời gian tìm kiếm, giảm số lượng thông tin cần xem và nâng cao khả năng tiếp cận các đối tượng phù hợp với sở thích cá nhân. Đối với nhà cung cấp dịch vụ, hệ thống góp phần tăng khả năng tiếp cận sản phẩm, nâng cao mức độ tương tác của người dùng và cải thiện chất lượng dịch vụ.

Nhờ khả năng phân tích dữ liệu và cá nhân hóa kết quả, hệ thống gợi ý ngày càng trở thành một thành phần quan trọng trong nhiều hệ thống thông tin hiện đại.

2.2.3. Các phương pháp gợi ý phổ biến

Hiện nay, có nhiều phương pháp được sử dụng để xây dựng hệ thống gợi ý. Mỗi phương pháp được phát triển dựa trên những nguyên lý hoạt động khác nhau và có những ưu điểm cũng như hạn chế riêng. Trong đó, ba phương pháp được sử dụng phổ biến nhất là gợi ý dựa trên nội dung, gợi ý cộng tác và gợi ý dựa trên tri thức. Ngoài ra, nhằm khắc phục những hạn chế của từng phương pháp riêng lẻ, mô hình gợi ý kết hợp đã được nghiên cứu và ứng dụng rộng rãi trong nhiều hệ thống hiện đại.

Gợi ý dựa trên nội dung

Gợi ý dựa trên nội dung (Content-Based Filtering) là phương pháp đề xuất các đối tượng có đặc điểm tương đồng với những đối tượng mà người dùng đã từng quan tâm hoặc tương tác trước đó. Phương pháp này hoạt động dựa trên việc phân tích các thuộc tính của đối tượng để xây dựng hồ sơ sở thích của người dùng, từ đó xác định mức độ tương đồng giữa các đối tượng và đưa ra các đề xuất phù hợp.

Phương pháp này có ưu điểm là khả năng cá nhân hóa cao cho từng người dùng, giúp hệ thống đưa ra các gợi ý phù hợp với nhu cầu và sở thích riêng. Ngoài ra, phương pháp không phụ thuộc vào dữ liệu của những người dùng khác nên có thể hoạt động hiệu quả ngay cả trong giai đoạn đầu khi hệ thống chưa có nhiều dữ liệu.

Tuy nhiên, gợi ý dựa trên nội dung cũng tồn tại một số hạn chế. Phương pháp này gặp khó khăn trong việc đề xuất các đối tượng hoàn toàn mới do chưa có đủ thông tin để so sánh. Hiệu quả của hệ thống phụ thuộc nhiều vào chất lượng và mức độ của dữ liệu thuộc tính. Bên cạnh đó, phương pháp này dễ dẫn đến hiện tượng gợi ý lặp lại các đối tượng có đặc điểm tương tự nhau, làm giảm tính đa dạng trong kết quả đề xuất.

Gợi ý cộng tác

Gợi ý cộng tác (Collaborative Filtering) là phương pháp xây dựng các đề xuất dựa trên sự tương đồng về hành vi hoặc sở thích giữa nhiều người dùng. Thông qua việc phân tích các dữ liệu như đánh giá, lượt xem hoặc lịch sử tương tác, hệ thống sẽ xác định những người dùng có hành vi tương tự nhau và từ đó đề xuất các đối tượng mà những người dùng có cùng sở thích đã quan tâm.

Phương pháp này có ưu điểm là khả năng khám phá và đề xuất các đối tượng mới mà người dùng chưa từng tương tác trước đó, đồng thời không yêu cầu phân tích chi tiết thuộc tính của đối tượng. Bên cạnh đó, chất lượng gợi ý có xu hướng được cải thiện khi hệ thống có lượng dữ liệu tương tác lớn và phong phú.

Tuy nhiên, gợi ý cộng tác cũng tồn tại một số hạn chế. Phương pháp này gặp khó khăn trong trường hợp người dùng mới hoặc đối tượng mới do thiếu dữ liệu lịch sử tương tác (cold start problem). Ngoài ra, hiệu quả của hệ thống giảm khi dữ liệu còn thưa hoặc không đầy đủ và phụ thuộc nhiều vào số lượng cũng như chất lượng dữ liệu của cộng đồng người dùng.

Gợi ý dựa trên tri thức

Gợi ý dựa trên tri thức (Knowledge-Based Recommendation) là phương pháp sử dụng các tập luật, tri thức chuyên môn hoặc các điều kiện được xây dựng sẵn để đưa ra các đề xuất phù hợp với yêu cầu cụ thể của người dùng. Phương pháp này thường được áp dụng trong những lĩnh vực mà dữ liệu tương tác còn hạn chế hoặc các đối tượng có giá trị cao, khiến người dùng ít có xu hướng lặp lại hành vi mua hoặc sử dụng.

Phương pháp này có ưu điểm là không phụ thuộc vào lịch sử tương tác của người dùng, do đó vẫn có thể hoạt động hiệu quả trong các hệ thống thiếu dữ liệu. Ngoài ra, nó phù hợp với các bài toán có dữ liệu ít hoặc thay đổi không thường xuyên, đồng thời có khả năng đưa ra các đề xuất dựa trực tiếp trên yêu cầu cụ thể của người dùng.

Tuy nhiên, gợi ý dựa trên tri thức cũng tồn tại một số hạn chế. Việc xây dựng và cập nhật cơ sở tri thức khá phức tạp và đòi hỏi nhiều công sức. Khả năng mở rộng của hệ thống phụ thuộc chặt chẽ vào hệ thống luật và tri thức đã xây dựng, đồng thời chi phí xây dựng và bảo trì cũng tương đối cao so với các phương pháp khác.

Qua phân tích có thể thấy rằng mỗi phương pháp gợi ý đều có những ưu điểm và hạn chế riêng. Vì vậy, trong các hệ thống gợi ý hiện đại, phương pháp gợi ý kết hợp thường được sử dụng nhằm tận dụng ưu điểm của nhiều phương pháp khác nhau, từ đó nâng cao độ chính xác và khả năng cá nhân hóa của kết quả gợi ý.

2.3. Hệ thống gợi ý kết hợp (Hybrid Recommendation System)

Hệ thống gợi ý kết hợp là mô hình gợi ý được xây dựng bằng cách kết hợp từ hai hoặc nhiều phương pháp gợi ý khác nhau nhằm tận dụng ưu điểm và khắc phục những hạn chế của từng phương pháp riêng lẻ. Thay vì chỉ sử dụng một nguồn dữ liệu hoặc một thuật toán duy nhất, hệ thống gợi ý kết hợp khai thác đồng thời nhiều nguồn dữ liệu như thuộc tính của đối tượng, dữ liệu hành vi người dùng, tri thức chuyên môn hoặc các yếu tố ngữ cảnh để tạo ra các đề xuất có độ chính xác và mức độ cá nhân hóa cao hơn.

Trong thực tế, mỗi phương pháp gợi ý đều có những hạn chế nhất định. Chẳng hạn, phương pháp gợi ý dựa trên nội dung có khả năng cá nhân hóa tốt nhưng khó đề xuất các đối tượng mới, trong khi phương pháp gợi ý cộng tác có thể khai thác được sở thích của cộng đồng người dùng nhưng thường gặp vấn đề dữ liệu thưa và người dùng mới.

Việc kết hợp nhiều phương pháp trong cùng một hệ thống giúp giảm thiểu các hạn chế này, đồng thời nâng cao chất lượng của kết quả gợi ý.

Hiện nay, Hệ thống gợi ý kết hợp được xem là một trong những hướng tiếp cận hiệu quả nhất trong lĩnh vực hệ thống gợi ý và được ứng dụng rộng rãi trong nhiều lĩnh vực như thương mại điện tử, mạng xã hội, dịch vụ phát trực tuyến, giáo dục, du lịch và bất động sản. Nhờ khả năng kết hợp nhiều nguồn dữ liệu và nhiều thuật toán khác nhau, hệ thống có thể đưa ra các đề xuất phù hợp với nhu cầu của từng người dùng.

Có nhiều chiến lược được sử dụng để xây dựng hệ thống gợi ý kết hợp, trong đó phổ biến gồm:

- Weighted Hybrid: Kết hợp kết quả của nhiều phương pháp thông qua trọng số.
- Switching Hybrid: Lựa chọn phương pháp gợi ý phù hợp theo từng ngữ cảnh.
- Cascade Hybrid: Một phương pháp tạo danh sách gợi ý ban đầu, các phương pháp khác tiếp tục sắp xếp và tinh chỉnh kết quả.
- Mixed Hybrid: Hiển thị đồng thời kết quả từ nhiều phương pháp gợi ý.
- Feature Combination: Kết hợp dữ liệu đầu vào từ nhiều phương pháp để xây dựng mô hình gợi ý chung.

Trong các chiến lược trên, phương pháp Weighted Hybrid được sử dụng phổ biến nhờ khả năng điều chỉnh mức độ ảnh hưởng của từng phương pháp thông qua các trọng số. Việc lựa chọn trọng số phù hợp giúp hệ thống linh hoạt thích nghi với từng ngữ cảnh và nâng cao chất lượng của kết quả gợi ý.

Nhìn chung, Hệ thống gợi ý kết hợp không chỉ kế thừa những ưu điểm của các phương pháp gợi ý truyền thống mà còn cải thiện khả năng cá nhân hóa, giảm ảnh hưởng của dữ liệu thừa và nâng cao độ chính xác của các đề xuất. Vì vậy, đây là mô hình được nhiều nghiên cứu lựa chọn làm nền tảng để xây dựng các hệ thống gợi ý hiện đại.

2.4. Công nghệ ReactJS

2.4.1 Giới thiệu ReactJS

ReactJS là thư viện JavaScript mã nguồn mở do Facebook phát triển và công bố vào năm 2013 nhằm hỗ trợ xây dựng giao diện người dùng (User Interface - UI) cho các ứng dụng web. ReactJS được thiết kế dựa trên kiến trúc hướng thành phần (Component-

Based Architecture), trong đó giao diện được chia thành các thành phần độc lập có thể tái sử dụng, giúp giảm sự trùng lặp mã nguồn, nâng cao khả năng bảo trì và mở rộng hệ thống. Ngoài ra, ReactJS sử dụng cơ chế Virtual DOM để tối ưu quá trình cập nhật giao diện, chỉ thực hiện thay đổi đối với các thành phần bị ảnh hưởng thay vì tải lại toàn bộ trang, từ đó cải thiện hiệu năng xử lý và mang lại trải nghiệm người dùng tốt hơn. Nhờ khả năng xây dựng các ứng dụng đơn trang (Single Page Application - SPA), cùng hệ sinh thái phong phú và cộng đồng phát triển lớn, ReactJS hiện là một trong những thư viện phát triển giao diện phổ biến trong các ứng dụng web hiện đại [1], [2].

2.4.2. Ưu điểm

ReactJS mang lại nhiều lợi ích trong quá trình phát triển ứng dụng web hiện đại. Với kiến trúc hướng thành phần (Component-Based), ReactJS cho phép chia giao diện thành các thành phần độc lập có khả năng tái sử dụng, giúp giảm sự trùng lặp mã nguồn và thuận tiện trong quá trình bảo trì, mở rộng hệ thống. Bên cạnh đó, cơ chế Virtual DOM giúp giảm số lần thao tác trực tiếp với DOM, từ đó cải thiện hiệu năng hiển thị và tăng tốc độ xử lý giao diện. ReactJS cũng hỗ trợ xây dựng các ứng dụng đơn trang (Single Page Application - SPA), giúp việc chuyển đổi giữa các trang diễn ra nhanh chóng, mang lại trải nghiệm mượt mà cho người dùng. Ngoài ra, thư viện này sở hữu hệ sinh thái phong phú, dễ dàng tích hợp với nhiều công cụ như React Router, Redux Toolkit và Axios, đồng thời được cộng đồng phát triển rộng lớn hỗ trợ với nhiều tài liệu và thư viện, tạo điều kiện thuận lợi cho việc phát triển và bảo trì ứng dụng [1], [2].

2.4.3. Hạn chế

Bên cạnh những ưu điểm, ReactJS vẫn tồn tại một số hạn chế. ReactJS chỉ tập trung vào việc xây dựng giao diện người dùng nên cần kết hợp với các thư viện hoặc framework khác để phát triển một ứng dụng hoàn chỉnh. Việc thiết lập môi trường phát triển ban đầu có thể tương đối phức tạp đối với người mới do phải cấu hình và tích hợp nhiều công cụ hỗ trợ. Ngoài ra, hệ sinh thái ReactJS phát triển nhanh với các phiên bản và thư viện thường xuyên được cập nhật, đòi hỏi lập trình viên phải liên tục học tập và cập nhật kiến thức. Khi quy mô ứng dụng ngày càng lớn, việc quản lý trạng thái (State) cũng có thể trở nên phức tạp nếu không áp dụng các giải pháp quản lý trạng thái phù hợp [1], [2].

2.5. Công nghệ NodeJS và ExpressJS

2.5.1. Giới thiệu NodeJS và ExpressJS

NodeJS là môi trường thực thi JavaScript mã nguồn mở được phát triển dựa trên V8 JavaScript Engine của Google Chrome, cho phép thực thi mã JavaScript ở phía máy chủ. Được giới thiệu lần đầu vào năm 2009 bởi Ryan Dahl, NodeJS sử dụng mô hình lập trình bất đồng bộ (Asynchronous) kết hợp với kiến trúc hướng sự kiện (Event-Driven), giúp xử lý đồng thời nhiều kết nối với hiệu năng cao và sử dụng tài nguyên hiệu quả. Nhờ đó, NodeJS phù hợp để phát triển các ứng dụng web, hệ thống thời gian thực và các dịch vụ API có lượng truy cập lớn [3].

ExpressJS là framework mã nguồn mở được xây dựng trên nền tảng NodeJS nhằm hỗ trợ phát triển các ứng dụng web và dịch vụ RESTful API. ExpressJS cung cấp nhiều tính năng như định tuyến (Routing), quản lý Middleware và xử lý yêu cầu HTTP, giúp đơn giản hóa quá trình xây dựng ứng dụng phía máy chủ. Với kiến trúc tối giản (Minimalist Framework), ExpressJS cho phép lập trình viên dễ dàng mở rộng và tích hợp với nhiều thư viện khác, trở thành một trong những framework phổ biến nhất của hệ sinh thái NodeJS [4].

Sự kết hợp giữa NodeJS và ExpressJS giúp rút ngắn thời gian phát triển, nâng cao hiệu năng xử lý và đáp ứng tốt yêu cầu xây dựng các hệ thống web hiện đại, đặc biệt là các ứng dụng sử dụng kiến trúc RESTful API.

2.5.2. Ưu điểm

NodeJS và ExpressJS mang lại nhiều lợi ích trong quá trình phát triển ứng dụng web hiện đại. Việc sử dụng JavaScript cho cả phía giao diện (Frontend) và phía máy chủ (Backend) giúp thống nhất ngôn ngữ lập trình, giảm thời gian học tập và tăng hiệu quả phát triển hệ thống. Bên cạnh đó, NodeJS áp dụng mô hình xử lý bất đồng bộ kết hợp với kiến trúc hướng sự kiện, cho phép xử lý đồng thời nhiều yêu cầu với hiệu năng cao và sử dụng tài nguyên hiệu quả. ExpressJS cung cấp hệ thống định tuyến (Routing) và Middleware linh hoạt, hỗ trợ xây dựng các dịch vụ RESTful API một cách đơn giản và dễ mở rộng. Ngoài ra, hệ sinh thái npm với số lượng lớn thư viện cùng khả năng tích hợp với nhiều hệ quản trị cơ sở dữ liệu như MongoDB, MySQL và PostgreSQL giúp NodeJS và ExpressJS đáp ứng tốt nhu cầu phát triển các ứng dụng web hiện đại [3], [4].

2.5.3. Hạn chế

Bên cạnh những ưu điểm, NodeJS và ExpressJS vẫn tồn tại một số hạn chế. Do NodeJS hoạt động chủ yếu trên mô hình luồng đơn, công nghệ này chưa thực sự phù hợp với các tác vụ yêu cầu tính toán nặng và sử dụng nhiều tài nguyên CPU. Ngoài ra, việc xử lý bất đồng bộ có thể làm mã nguồn trở nên phức tạp nếu không được tổ chức hợp lý. Đối với ExpressJS, framework này chỉ cung cấp các chức năng cốt lõi nên trong các dự án quy mô lớn thường cần tích hợp thêm nhiều thư viện để đáp ứng đầy đủ yêu cầu của hệ thống. Đồng thời, ExpressJS không áp đặt cấu trúc dự án cụ thể, do đó nếu không xây dựng quy ước phát triển ngay từ đầu thì việc tổ chức mã nguồn có thể thiếu tính thống nhất và gây khó khăn trong quá trình bảo trì, mở rộng ứng dụng [3], [4].

2.6. Cơ sở dữ liệu MongoDB

2.6.1. Giới thiệu MongoDB

MongoDB là hệ quản trị cơ sở dữ liệu thuộc nhóm NoSQL (Not Only SQL), được phát triển bởi MongoDB Inc. và chính thức ra mắt vào năm 2009. Khác với các hệ quản trị cơ sở dữ liệu quan hệ truyền thống như MySQL hay SQL Server, MongoDB lưu trữ dữ liệu theo mô hình tài liệu (Document-Oriented Database), trong đó dữ liệu được biểu diễn dưới định dạng BSON (Binary JSON). Mô hình này cho phép dữ liệu có cấu trúc linh hoạt, dễ dàng mở rộng và phù hợp với các ứng dụng web hiện đại có khối lượng dữ liệu lớn [5].

Trong MongoDB, dữ liệu được tổ chức dưới dạng Collection và Document thay vì bảng, hàng và cột như cơ sở dữ liệu quan hệ. Mỗi Collection chứa nhiều Document và mỗi Document bao gồm các cặp khóa - giá trị (Key - Value). Cách tổ chức này giúp việc bổ sung hoặc thay đổi cấu trúc dữ liệu trở nên đơn giản mà không cần thay đổi toàn bộ cơ sở dữ liệu. Bên cạnh đó, MongoDB hỗ trợ khả năng mở rộng theo chiều ngang (Horizontal Scaling), tối ưu hiệu năng truy vấn và tích hợp tốt với các công nghệ JavaScript như NodeJS và ReactJS, nhờ đó được sử dụng rộng rãi trong các hệ thống thương mại điện tử, mạng xã hội và các ứng dụng web hiện đại [5], [6].

2.6.2. Ưu điểm

MongoDB mang lại nhiều lợi ích trong quá trình phát triển ứng dụng web hiện đại. Với mô hình lưu trữ dữ liệu theo dạng Document, MongoDB cho phép xây dựng cấu

trúc dữ liệu linh hoạt mà không yêu cầu lược đồ (Schema) cố định như các cơ sở dữ liệu quan hệ. Điều này giúp việc bổ sung hoặc thay đổi dữ liệu trở nên dễ dàng hơn khi hệ thống phát triển. Ngoài ra, MongoDB có hiệu năng cao trong các thao tác đọc, ghi và truy vấn dữ liệu, đồng thời hỗ trợ mở rộng theo chiều ngang để đáp ứng các hệ thống có quy mô lớn. Cơ sở dữ liệu này cũng cung cấp nhiều cơ chế tối ưu như Index và Aggregation Framework, đồng thời tích hợp hiệu quả với NodeJS thông qua thư viện Mongoose, giúp đơn giản hóa quá trình phát triển ứng dụng [5], [6].

2.6.3. Hạn chế

Bên cạnh những ưu điểm, MongoDB vẫn tồn tại một số hạn chế. Do không phải là hệ quản trị cơ sở dữ liệu quan hệ, MongoDB không hỗ trợ đầy đủ các ràng buộc như khóa ngoại (Foreign Key) và các phép nối dữ liệu phức tạp như trong SQL. Vì vậy, việc thiết kế mô hình dữ liệu cần được cân nhắc để hạn chế tình trạng dư thừa dữ liệu và tối ưu hiệu năng truy vấn. Ngoài ra, việc tạo quá nhiều chỉ mục (Index) có thể làm tăng dung lượng lưu trữ và ảnh hưởng đến hiệu năng của các thao tác ghi dữ liệu như thêm, cập nhật hoặc xóa Document. Do đó, cần lựa chọn chiến lược thiết kế dữ liệu và xây dựng Index phù hợp để đạt hiệu quả tối ưu trong quá trình vận hành hệ thống [5], [6].

2.7. Công nghệ Socket.io

2.7.1. Giới thiệu Socket.io

Socket.io là thư viện JavaScript mã nguồn mở được xây dựng trên nền tảng WebSocket, hỗ trợ thiết lập kết nối hai chiều (Bidirectional Communication) giữa máy khách (Client) và máy chủ (Server). Thư viện được thiết kế nhằm hỗ trợ xây dựng các ứng dụng truyền dữ liệu theo thời gian thực (Real-Time), cho phép Client và Server trao đổi dữ liệu liên tục thông qua cơ chế sự kiện (Event-Based Communication). Khác với giao thức HTTP truyền thống, Socket.io cho phép Server chủ động gửi dữ liệu đến Client ngay khi có sự thay đổi mà không cần chờ yêu cầu từ phía Client, giúp giảm độ trễ và nâng cao trải nghiệm người dùng [7].

Socket.io ưu tiên sử dụng giao thức WebSocket để truyền dữ liệu và tự động chuyển sang các cơ chế truyền tải dự phòng khi WebSocket không khả dụng, đảm bảo tính ổn định của kết nối trên nhiều trình duyệt và môi trường khác nhau. Nhờ khả năng tích hợp tốt với NodeJS và ExpressJS, Socket.io được sử dụng rộng rãi trong các ứng

dụng như trò chuyện trực tuyến, hệ thống thông báo, theo dõi thời gian thực và các nền tảng thương mại điện tử [7].

2.7.2. Ưu điểm

Socket.io mang lại nhiều lợi ích trong việc phát triển các ứng dụng thời gian thực. Thư viện hỗ trợ giao tiếp hai chiều giữa Client và Server với độ trễ thấp, cho phép dữ liệu được đồng bộ gần như tức thời. Socket.io hoạt động theo cơ chế sự kiện (Event-Driven), giúp việc trao đổi dữ liệu trở nên linh hoạt và dễ triển khai. Ngoài ra, thư viện còn hỗ trợ tự động quản lý và khôi phục kết nối khi xảy ra gián đoạn, đồng thời cung cấp các tính năng như Room, Namespace và Broadcasting để phục vụ các hệ thống có nhiều người dùng truy cập đồng thời. Bên cạnh đó, Socket.io dễ dàng tích hợp với NodeJS và ExpressJS, góp phần đơn giản hóa quá trình phát triển các ứng dụng web hiện đại [7].

2.7.3. Hạn chế

Bên cạnh những ưu điểm, Socket.io vẫn tồn tại một số hạn chế. Việc duy trì nhiều kết nối đồng thời có thể làm tăng mức tiêu thụ tài nguyên của máy chủ, đặc biệt đối với các hệ thống có số lượng người dùng lớn. Khi triển khai trên các hệ thống phân tán hoặc nhiều máy chủ, Socket.io thường cần kết hợp với các giải pháp đồng bộ dữ liệu như Redis Adapter để đảm bảo tính nhất quán giữa các kết nối. Ngoài ra, do cung cấp nhiều tính năng bổ sung như cơ chế dự phòng kết nối và quản lý sự kiện, hiệu năng của Socket.io trong một số trường hợp có thể thấp hơn so với việc sử dụng WebSocket [7].

2.8. Leaflet và OpenStreetMap

2.8.1. Giới thiệu Leaflet và OpenStreetMap

Leaflet là thư viện JavaScript mã nguồn mở được sử dụng để xây dựng các ứng dụng bản đồ tương tác trên nền tảng web. Với dung lượng nhỏ, hiệu năng cao và khả năng tương thích với nhiều trình duyệt, Leaflet cung cấp các chức năng như hiển thị bản đồ, đánh dấu vị trí (Marker), hiển thị cửa sổ thông tin (Popup), vẽ các đối tượng địa lý và quản lý nhiều lớp bản đồ (Layer). Đồng thời, Leaflet có thể tích hợp với nhiều nguồn dữ liệu bản đồ khác nhau, trong đó OpenStreetMap là nguồn dữ liệu được sử dụng phổ biến nhất [8].

OpenStreetMap (OSM) là dự án bản đồ mã nguồn mở được xây dựng và duy trì bởi cộng đồng trên toàn thế giới. Dự án cung cấp dữ liệu bản đồ miễn phí, bao gồm đường giao thông, địa điểm, khu dân cư và nhiều đối tượng địa lý khác. Dữ liệu của OpenStreetMap được cập nhật thường xuyên và có thể tích hợp dễ dàng với Leaflet để xây dựng các ứng dụng bản đồ trực tuyến. Nhờ tính mở, khả năng mở rộng và không yêu cầu chi phí bản quyền, Leaflet kết hợp với OpenStreetMap đã trở thành giải pháp phổ biến trong các hệ thống WebGIS và các ứng dụng dựa trên vị trí địa lý [8], [9].

2.8.2. Ưu điểm

Leaflet và OpenStreetMap mang lại nhiều lợi ích trong quá trình phát triển các ứng dụng bản đồ trực tuyến. Leaflet có dung lượng nhỏ, hiệu năng cao và dễ dàng tích hợp vào các ứng dụng web, đồng thời hỗ trợ nhiều chức năng như hiển thị bản đồ, đánh dấu vị trí, quản lý lớp bản đồ và tương tác với người dùng. OpenStreetMap cung cấp nguồn dữ liệu bản đồ miễn phí, được cập nhật thường xuyên bởi cộng đồng và có phạm vi bao phủ trên toàn thế giới. Sự kết hợp giữa hai công nghệ này giúp giảm chi phí triển khai, dễ dàng mở rộng và đáp ứng tốt các ứng dụng yêu cầu hiển thị vị trí, tìm kiếm địa điểm hoặc hỗ trợ các chức năng dựa trên vị trí địa lý [8], [9].

2.8.3. Hạn chế

Bên cạnh những ưu điểm, Leaflet và OpenStreetMap vẫn tồn tại một số hạn chế. Leaflet chưa tích hợp sẵn nhiều chức năng GIS nâng cao và khi xử lý lượng lớn dữ liệu không gian thường cần kết hợp với các thư viện mở rộng để đảm bảo hiệu năng. Đối với OpenStreetMap, chất lượng và mức độ đầy đủ của dữ liệu có thể khác nhau giữa các khu vực do phụ thuộc vào sự đóng góp của cộng đồng. Ngoài ra, khi triển khai các ứng dụng có lượng truy cập lớn hoặc yêu cầu xử lý bản đồ chuyên sâu, hệ thống có thể cần bổ sung các giải pháp tối ưu hóa và dịch vụ bản đồ chuyên dụng để đáp ứng yêu cầu về hiệu năng và khả năng mở rộng [8], [9].

CHƯƠNG 3. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

3.1. Phân tích yêu cầu hệ thống

3.1.1. Yêu cầu chức năng

Hệ thống hỗ trợ tìm kiếm và gợi ý phòng trọ được xây dựng nhằm hỗ trợ sinh viên tìm kiếm phòng trọ phù hợp, đồng thời tạo môi trường kết nối hiệu quả giữa sinh viên và chủ trọ. Để đáp ứng mục tiêu đó, hệ thống cần cung cấp các nhóm chức năng chính như sau:

- Quản lý tài khoản người dùng: Hệ thống cho phép người dùng đăng ký tài khoản, đăng nhập, đăng xuất và quản lý thông tin cá nhân. Người dùng có thể cập nhật hồ sơ cá nhân, thay đổi mật khẩu và quản lý thông tin liên hệ. Hệ thống hỗ trợ phân quyền giữa sinh viên, chủ trọ và quản trị viên.

- Quản lý thông tin phòng trọ: Chủ trọ có thể đăng tin phòng trọ mới, cập nhật thông tin hoặc xóa các phòng trọ đã đăng. Thông tin phòng trọ bao gồm tiêu đề, mô tả, giá thuê, diện tích, địa chỉ, hình ảnh, tiện ích và vị trí địa lý.

- Tìm kiếm và lọc phòng trọ: Người dùng có thể tìm kiếm phòng trọ theo từ khóa hoặc áp dụng các bộ lọc như giá thuê, diện tích, loại phòng, sức chứa và vị trí nhằm nhanh chóng tìm được phòng phù hợp với nhu cầu.

- Hiện thị vị trí phòng trọ trên bản đồ: Hệ thống tích hợp bản đồ số để hiển thị vị trí các phòng trọ. Người dùng có thể xem vị trí phòng trọ, vị trí hiện tại của mình và hỗ trợ tìm kiếm phòng trọ theo khu vực địa lý.

- Gợi ý phòng trọ phù hợp với người dùng: Hệ thống sử dụng mô hình Hybrid Recommendation để đề xuất các phòng trọ phù hợp dựa trên thông tin phòng trọ, lịch sử tương tác của người dùng và vị trí địa lý. Kết quả gợi ý giúp người dùng tiết kiệm thời gian tìm kiếm và nâng cao trải nghiệm sử dụng.

- Quản lý phòng trọ yêu thích: Người dùng có thể lưu các phòng trọ quan tâm vào danh sách yêu thích để tiện cho việc xem lại và so sánh trong quá trình lựa chọn.

- Quản lý lịch hẹn xem phòng: Sinh viên có thể gửi yêu cầu đặt lịch xem phòng trực tiếp với chủ trọ. Chủ trọ có thể xác nhận, từ chối hoặc thay đổi lịch hẹn. Hệ thống lưu trữ và quản lý toàn bộ lịch sử đặt lịch.

- Hệ thống nhắn tin trực tuyến: Hệ thống cung cấp chức năng nhắn tin thời gian thực giữa sinh viên và chủ trọ, giúp hai bên trao đổi thông tin liên quan đến phòng trọ và lịch hẹn một cách thuận tiện.

- Quản lý bình luận và đánh giá: Người dùng có thể bình luận, đánh giá và chia sẻ ý kiến về các phòng trọ. Chủ trọ có thể phản hồi các bình luận nhằm tăng tính tương tác và minh bạch thông tin.

- Hệ thống thông báo: Hệ thống gửi thông báo cho người dùng khi có tin nhắn mới, lịch hẹn mới, thay đổi trạng thái lịch hẹn hoặc các hoạt động quan trọng khác liên quan đến tài khoản.

- Quản trị hệ thống: Quản trị viên có quyền quản lý người dùng, kiểm duyệt nội dung phòng trọ, xử lý báo cáo vi phạm, quản lý bình luận và theo dõi hoạt động của toàn bộ hệ thống nhằm đảm bảo tính ổn định và an toàn khi vận hành.

3.1.2. Yêu cầu phi chức năng

Hiệu năng:

Hệ thống cần đáp ứng nhanh các yêu cầu của người dùng trong quá trình sử dụng. Các chức năng như tìm kiếm, lọc dữ liệu, hiển thị thông tin phòng trọ và gợi ý phòng trọ phải có thời gian phản hồi ngắn trong điều kiện hoạt động bình thường. Đồng thời, hệ thống phải hỗ trợ nhiều người dùng truy cập và sử dụng đồng thời mà vẫn đảm bảo hiệu năng.

Tính bảo mật:

Hệ thống phải đảm bảo an toàn cho dữ liệu và thông tin của người dùng. Mật khẩu được mã hóa trước khi lưu trữ trong cơ sở dữ liệu. Cơ chế xác thực và phân quyền phải được áp dụng nhằm đảm bảo mỗi người dùng chỉ được truy cập và sử dụng các chức năng tương ứng với vai trò của mình.

Tính chính xác:

Thông tin được hiển thị trên hệ thống phải chính xác, nhất quán và phản ánh đúng dữ liệu lưu trữ trong cơ sở dữ liệu. Các chức năng tìm kiếm, lọc và gợi ý phải trả về kết quả phù hợp với yêu cầu của người dùng. Quá trình lưu trữ và cập nhật dữ liệu phải đảm bảo tính toàn vẹn và hạn chế sai lệch dữ liệu.

Khả năng mở rộng:

Hệ thống cần được thiết kế theo hướng mở, cho phép bổ sung các chức năng mới và mở rộng quy mô dữ liệu khi cần thiết. Kiến trúc hệ thống hỗ trợ việc nâng cấp, tích hợp các công nghệ mới và đáp ứng sự gia tăng về số lượng người dùng trong tương lai.

Tính khả dụng:

Giao diện hệ thống cần được thiết kế trực quan, thân thiện và dễ sử dụng đối với người dùng. Hệ thống phải hoạt động ổn định trên các trình duyệt web phổ biến và tương thích với nhiều loại thiết bị như máy tính, máy tính bảng và điện thoại thông minh.

Khả năng bảo trì:

Mã nguồn được tổ chức theo cấu trúc rõ ràng, thống nhất và dễ hiểu nhằm thuận tiện cho việc kiểm tra, sửa lỗi, nâng cấp và phát triển thêm các chức năng mới trong tương lai.

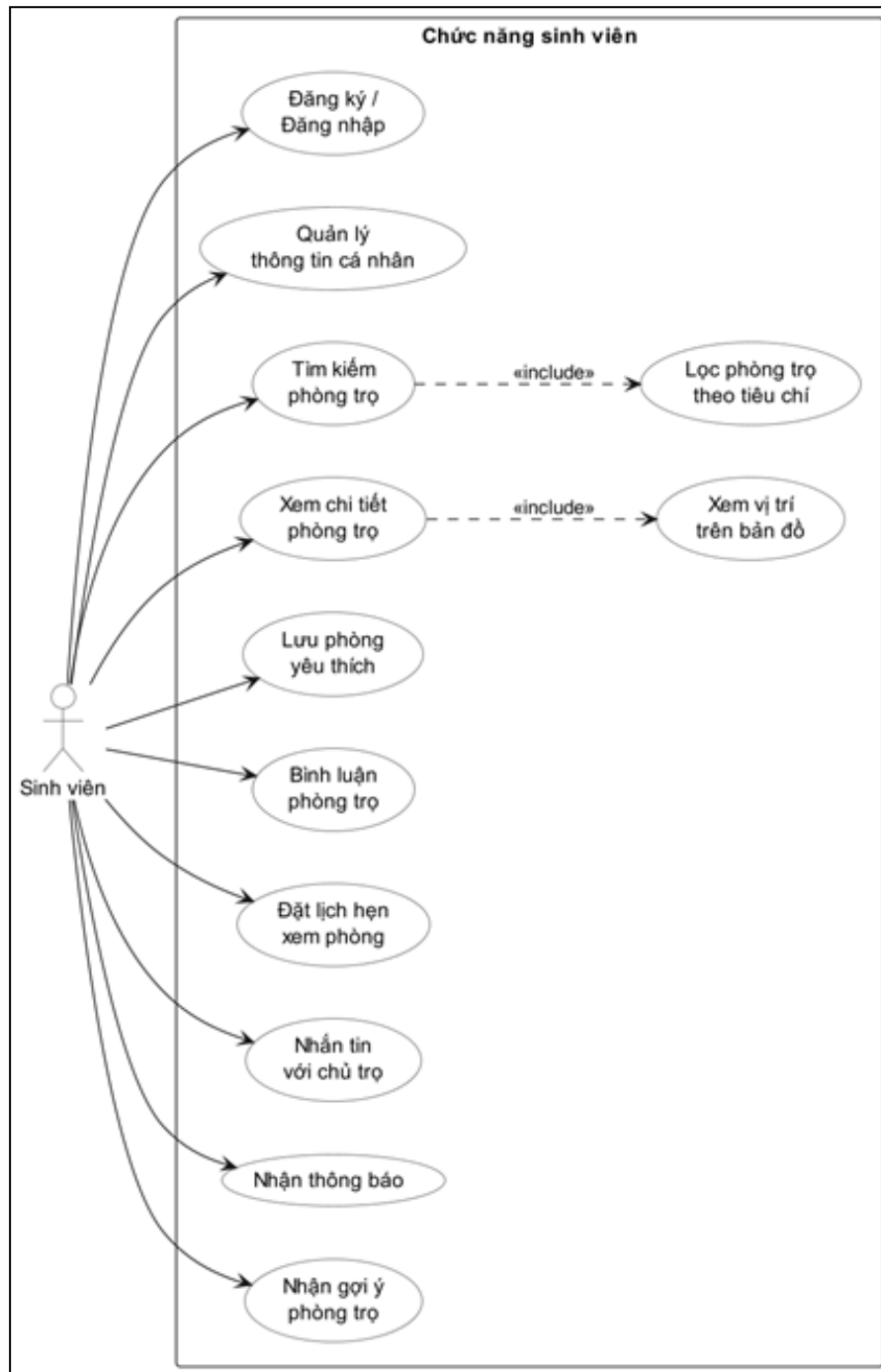
Tính tin cậy:

Hệ thống phải hoạt động ổn định trong thời gian dài, hạn chế tối đa các lỗi phát sinh trong quá trình vận hành. Đồng thời, hệ thống cần đảm bảo khả năng lưu trữ và xử lý dữ liệu chính xác, góp phần duy trì tính liên tục và độ tin cậy

3.2. Biểu đồ Use Case

3.2.1. Use Case sinh viên

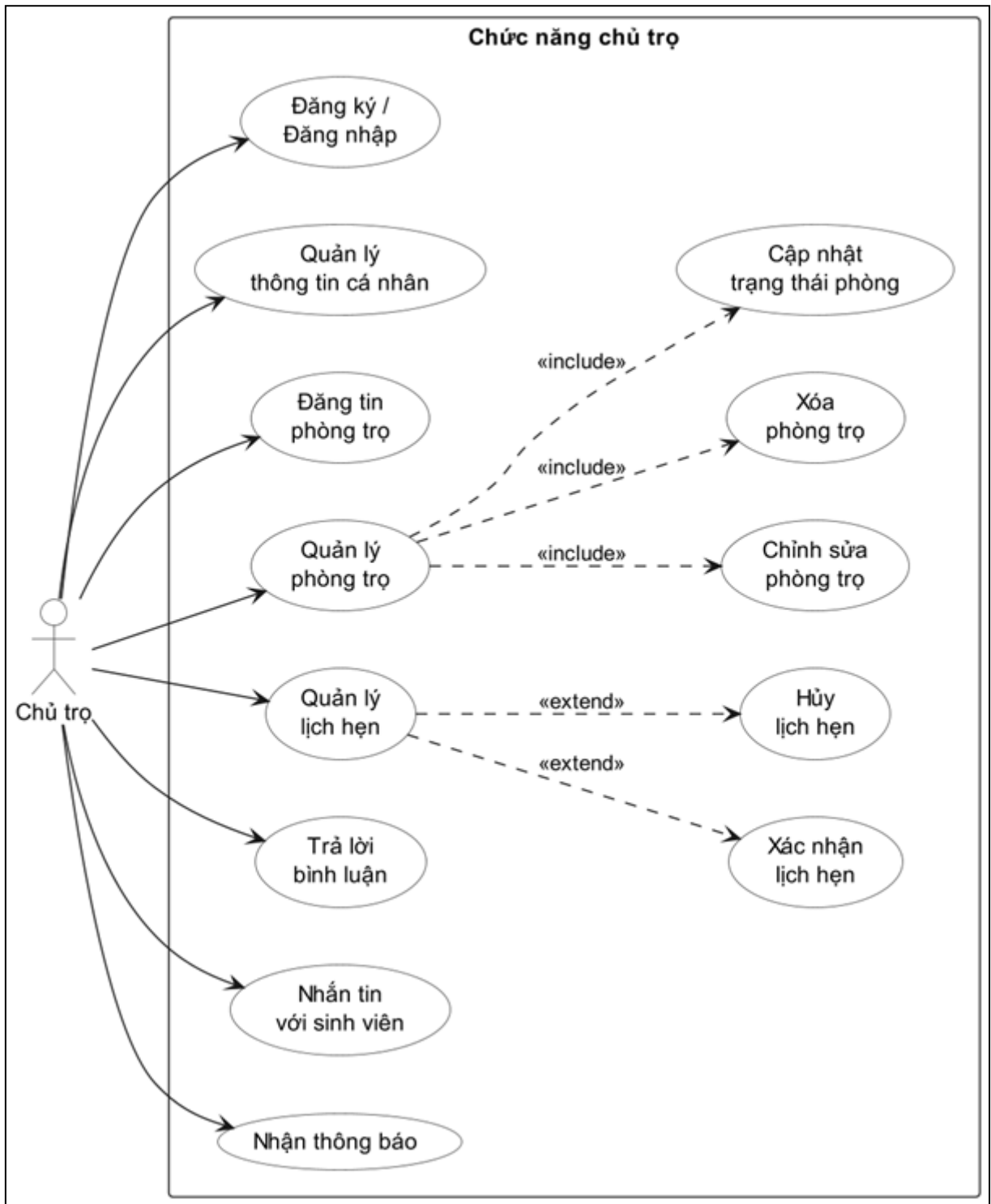
Biểu đồ Use Case sinh viên mô tả các chức năng mà sinh viên có thể thực hiện trên hệ thống như đăng ký tài khoản, đăng nhập, tìm kiếm phòng trọ, xem thông tin chi tiết phòng trọ, lưu phòng trọ yêu thích, đặt lịch xem phòng, nhắn tin với chủ trọ và nhận gợi ý phòng trọ.



Hình 3.1. Biểu đồ Use Case Sinh viên

3.2.2. Use Case chủ trọ

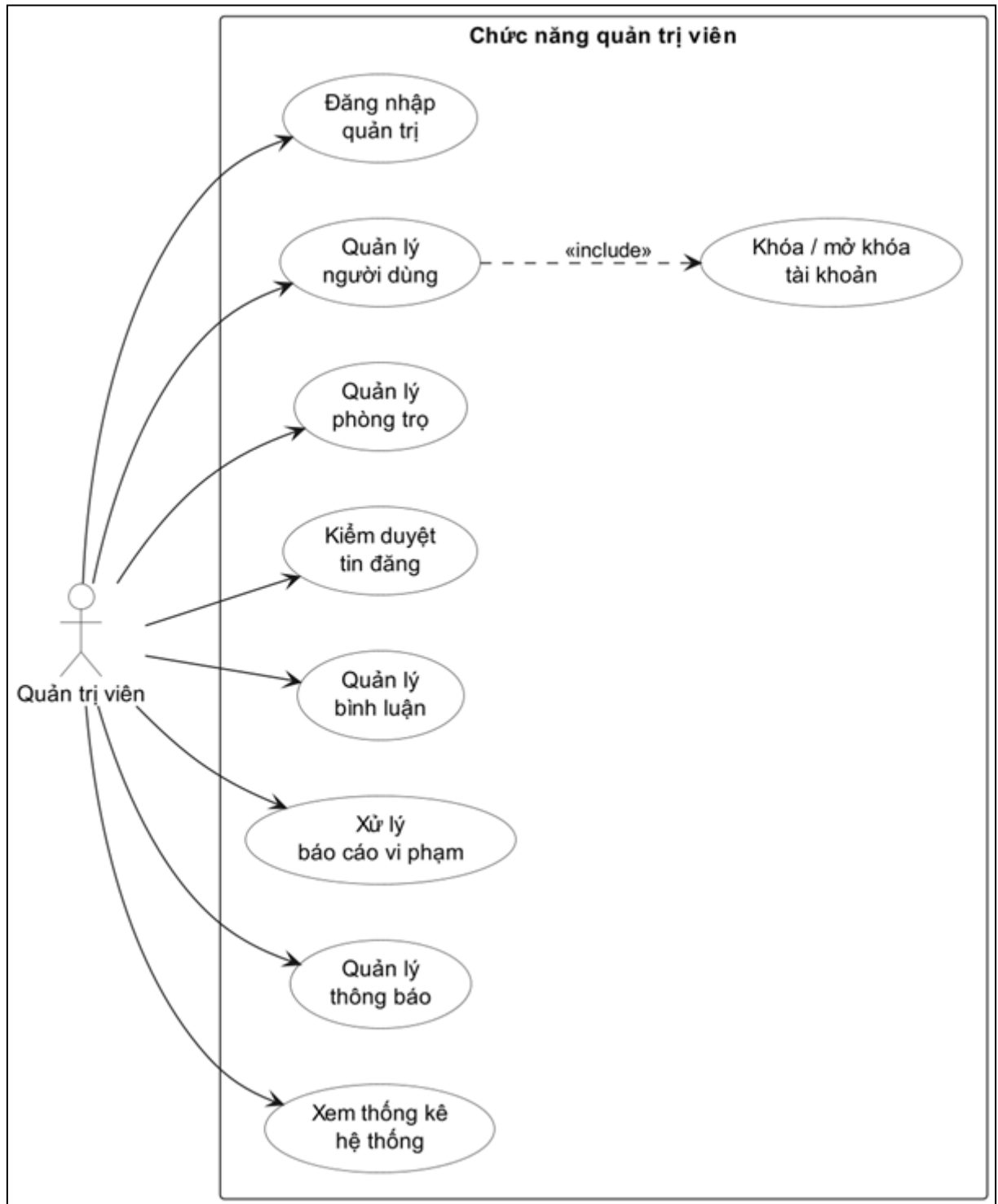
Biểu đồ Use Case chủ trọ mô tả các chức năng liên quan đến việc đăng tải, quản lý và cập nhật thông tin phòng trọ. Ngoài ra, chủ trọ còn có thể quản lý lịch hẹn, trả lời bình luận và trao đổi với sinh viên thông qua hệ thống.



Hình 3.2. Biểu đồ Use Case chủ trọ

3.2.3. Use Case quản trị viên

Biểu đồ Use Case quản trị viên mô tả các chức năng quản lý hệ thống như quản lý người dùng, kiểm duyệt phòng trọ, xử lý vi phạm, quản lý bình luận và theo dõi hoạt động của hệ thống.



Hình 3.3. Biểu đồ Use Case quản trị viên

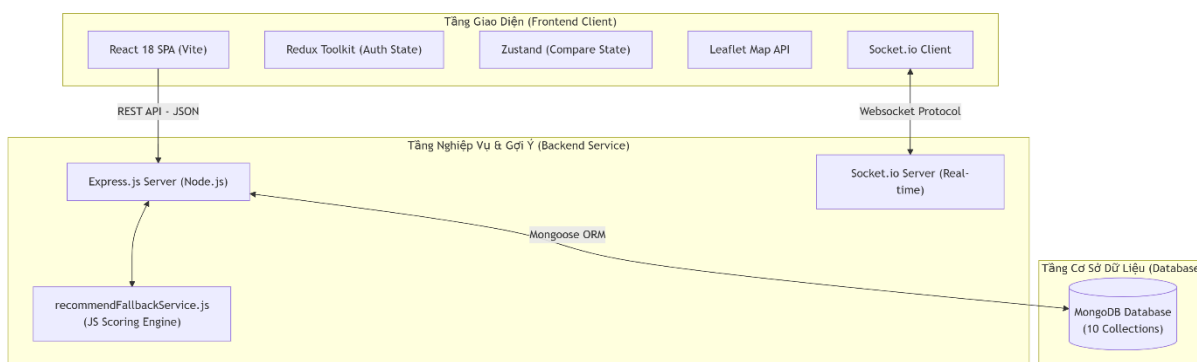
3.3. Thiết kế kiến trúc hệ thống

3.3.1. Kiến trúc tổng thể

Hệ thống hỗ trợ tìm kiếm và gợi ý phòng trọ sinh viên được xây dựng theo mô hình Client - Server, sử dụng ReactJS cho giao diện người dùng, NodeJS và ExpressJS

cho xử lý nghiệp vụ phía máy chủ, MongoDB làm hệ quản trị cơ sở dữ liệu và Socket.io hỗ trợ giao tiếp thời gian thực. Ngoài ra, hệ thống tích hợp mô hình Hybrid Recommendation nhằm cung cấp các gợi ý phòng trọ phù hợp với nhu cầu của từng người dùng.

Kiến trúc tổng thể của hệ thống được xây dựng theo mô hình ba lớp, bao gồm Frontend, Backend và Cơ sở dữ liệu. Trong đó, Frontend chịu trách nhiệm hiển thị giao diện và tiếp nhận các thao tác từ người dùng; Backend đảm nhiệm việc xử lý nghiệp vụ, xác thực người dùng, quản lý dữ liệu và cung cấp các dịch vụ RESTful API; còn MongoDB được sử dụng để lưu trữ toàn bộ dữ liệu của hệ thống. Ngoài ra, mô-đun gợi ý phòng trọ được tích hợp trực tiếp trong Backend nhằm thực hiện các thuật toán tính toán và đề xuất những phòng trọ phù hợp với nhu cầu của người dùng. Quá trình xử lý bắt đầu khi người dùng thực hiện các thao tác trên giao diện web, Frontend sẽ gửi yêu cầu đến Backend thông qua RESTful API. Sau khi tiếp nhận yêu cầu, Backend tiến hành xử lý nghiệp vụ, truy xuất hoặc cập nhật dữ liệu trong MongoDB. Đối với các chức năng gợi ý, Backend sẽ kích hoạt mô-đun Recommendation Engine để phân tích dữ liệu và tạo danh sách phòng trọ phù hợp trước khi trả kết quả về Frontend để hiển thị cho người dùng. Bên cạnh đó, các chức năng nhắn tin và thông báo được triển khai bằng Socket.io, cho phép trao đổi dữ liệu theo thời gian thực giữa máy khách và máy chủ, góp phần nâng cao tính tương tác và trải nghiệm của người dùng.



Hình 3.4. Kiến trúc tổng thể hệ thống

Kiến trúc này giúp hệ thống dễ dàng mở rộng, bảo trì và đáp ứng tốt số lượng lớn người dùng truy cập đồng thời.

3.3.2. Kiến trúc Frontend

Frontend của hệ thống được phát triển bằng ReactJS theo kiến trúc Component-Based Architecture, trong đó mỗi chức năng được xây dựng thành các component độc lập nhằm tăng khả năng tái sử dụng mã nguồn, thuận tiện cho việc mở rộng, bảo trì và nâng cấp hệ thống. Kiến trúc này giúp các thành phần giao diện được tổ chức rõ ràng, giảm sự phụ thuộc lẫn nhau và nâng cao hiệu quả phát triển.

Frontend bao gồm các thành phần chính như hệ thống định tuyến được xây dựng bằng React Router để quản lý điều hướng giữa các trang, hệ thống quản lý trạng thái (State Management) để chia sẻ và đồng bộ dữ liệu giữa các component, hệ thống kết nối API để giao tiếp với Backend và xử lý dữ liệu, cùng với chức năng bản đồ số và định vị địa lý phục vụ việc tìm kiếm và hiển thị vị trí phòng trọ.

Về mặt chức năng, Frontend được chia thành nhiều nhóm giao diện dành cho người dùng, bao gồm các trang chính như Trang chủ, Trang tìm kiếm phòng trọ, Trang chi tiết phòng trọ, Trang lịch hẹn và Trang nhấn tin. Mỗi trang đảm nhiệm một chức năng riêng nhưng được liên kết chặt chẽ thông qua hệ thống định tuyến và cơ chế quản lý trạng thái, góp phần mang lại trải nghiệm sử dụng thống nhất và mượt mà cho người dùng.

Quản lý trạng thái

Hệ thống sử dụng:

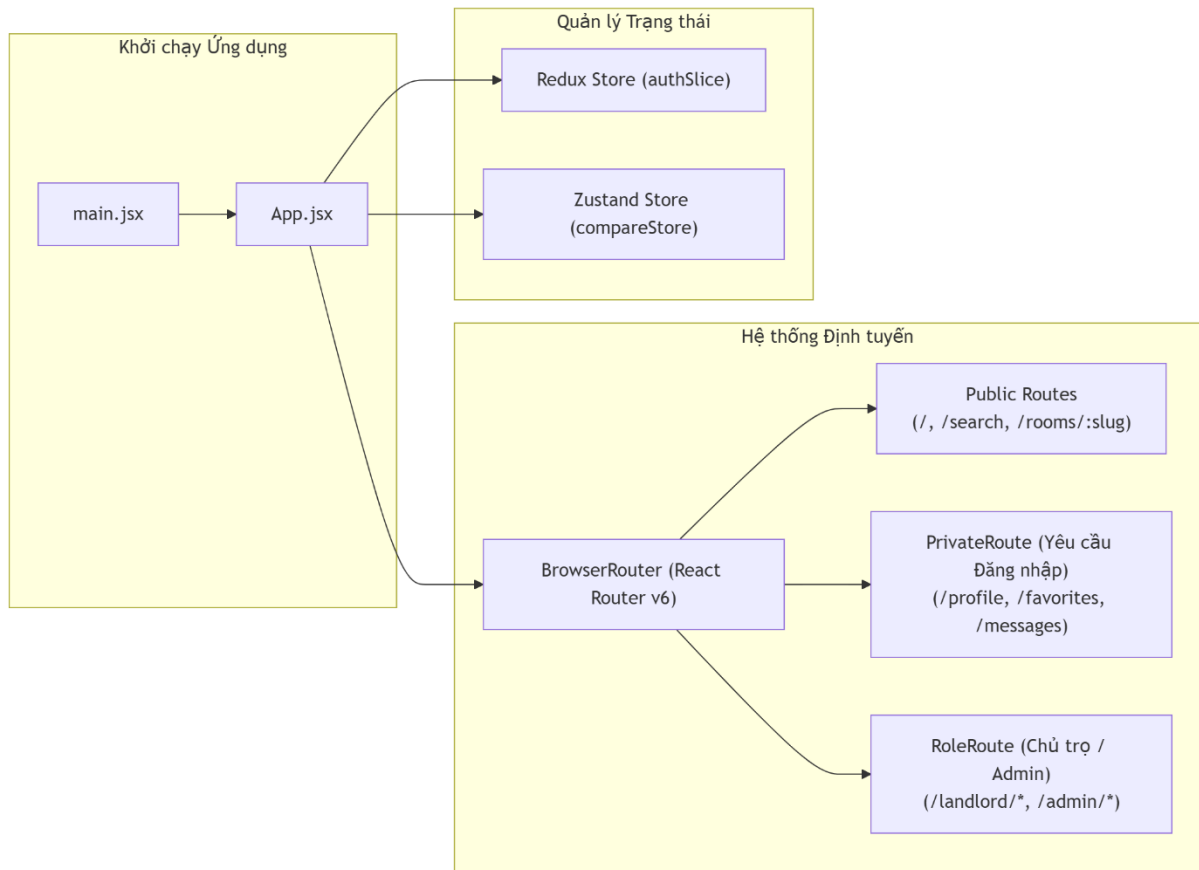
- Redux Toolkit được sử dụng để quản lý trạng thái toàn cục của hệ thống như thông tin người dùng, trạng thái đăng nhập và phân quyền truy cập.
- Zustand để quản lý các trạng thái đơn giản và có tần suất cập nhật cao.

Định tuyến

React Router được sử dụng để điều hướng giữa các trang.

Hệ thống hỗ trợ:

- Public Route cho người dùng chưa đăng nhập.
- Private Route cho người dùng đã xác thực.
- Role Route cho Chủ trọ và Quản trị viên.



Hình 3.5. Kiến trúc Frontend

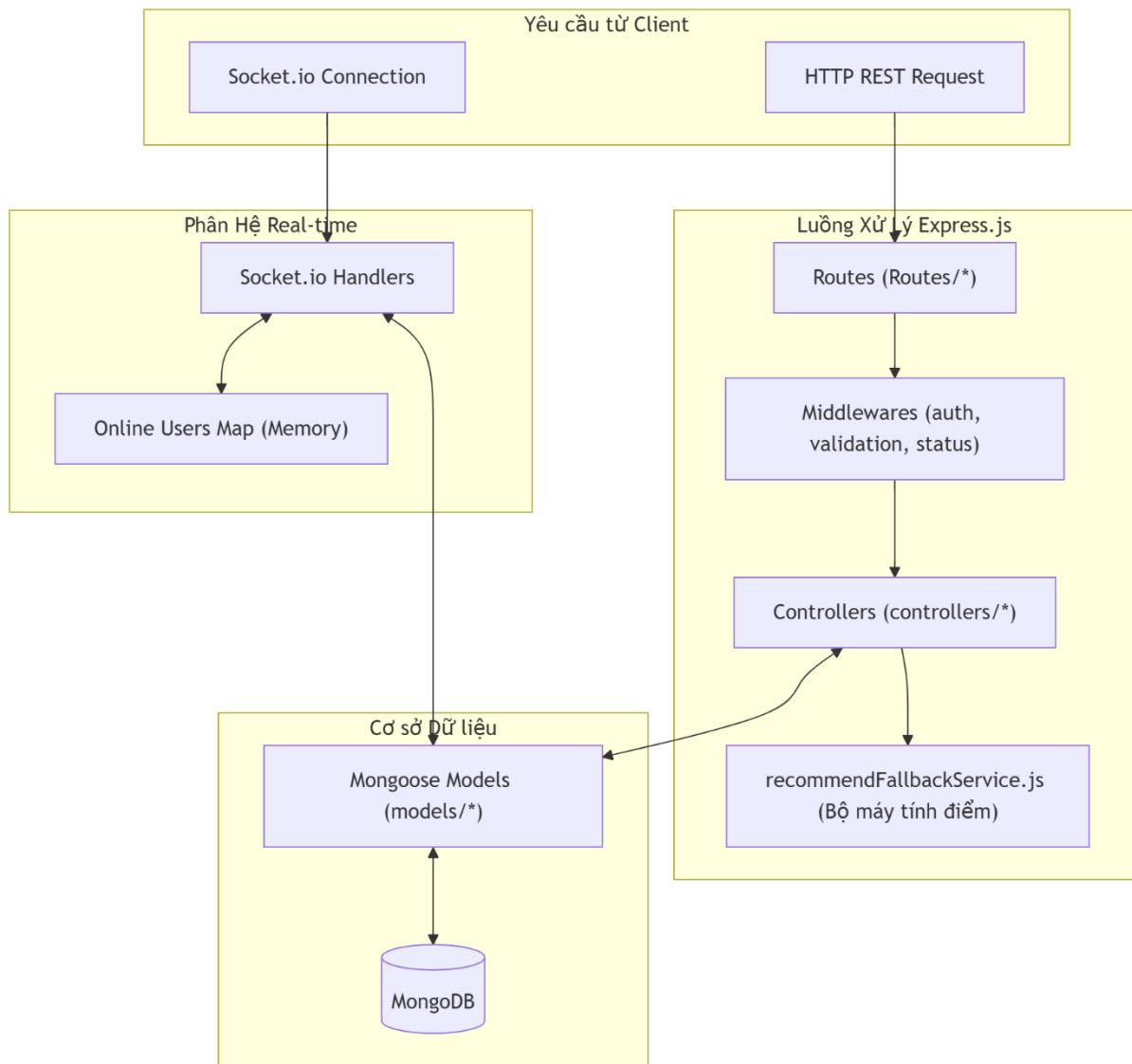
Bản đồ số

Frontend tích hợp Leaflet và OpenStreetMap nhằm:

- Hiển thị vị trí phòng trọ trên bản đồ.
- Hiển thị vị trí người dùng.
- Tính khoảng cách giữa người dùng và phòng trọ.
- Hỗ trợ tìm kiếm theo vị trí địa lý.

3.3.3. Kiến trúc Backend

Backend được xây dựng bằng NodeJS và ExpressJS theo mô hình Layered Architecture nhằm phân tách rõ ràng các chức năng của hệ thống.



Hình 3.6. Kiến trúc Backend

Backend của hệ thống được xây dựng theo kiến trúc phân lớp nhằm tách biệt các thành phần xử lý, giúp mã nguồn dễ quản lý, bảo trì và mở rộng. Mỗi lớp đảm nhận một nhiệm vụ riêng và phối hợp với nhau để xử lý các yêu cầu từ phía người dùng.

Lớp Routing có nhiệm vụ tiếp nhận các yêu cầu (HTTP Request) từ Client thông qua các API Endpoint, sau đó định tuyến và chuyển tiếp yêu cầu đến Controller tương ứng để xử lý.

Lớp Middleware thực hiện các chức năng trung gian trước khi yêu cầu được chuyển đến Controller, bao gồm xác thực người dùng bằng JWT, phân quyền truy cập theo vai trò, kiểm tra tính hợp lệ của dữ liệu đầu vào và xử lý các lỗi phát sinh trong quá trình thực thi.

Lớp Controller chịu trách nhiệm xử lý các nghiệp vụ chính của hệ thống như quản lý người dùng, quản lý phòng trọ, lịch hẹn, bình luận, danh sách yêu thích, báo cáo, tin nhắn và các chức năng gợi ý phòng trọ. Sau khi xử lý nghiệp vụ, Controller sẽ tương tác với lớp Model để truy xuất hoặc cập nhật dữ liệu và trả kết quả về cho Client.

Lớp Model sử dụng thư viện Mongoose để định nghĩa cấu trúc các Collection trong MongoDB, đồng thời cung cấp các phương thức thao tác với cơ sở dữ liệu như thêm, sửa, xóa và truy vấn dữ liệu.

Bên cạnh đó, hệ thống tích hợp Socket.io ở lớp Real-time nhằm hỗ trợ các chức năng thời gian thực như nhắn tin trực tuyến, gửi thông báo tức thời và theo dõi trạng thái trực tuyến của người dùng. Việc tách riêng các chức năng thời gian thực giúp xử lý hiệu quả các kết nối đồng thời mà không ảnh hưởng đến các API truyền thống.

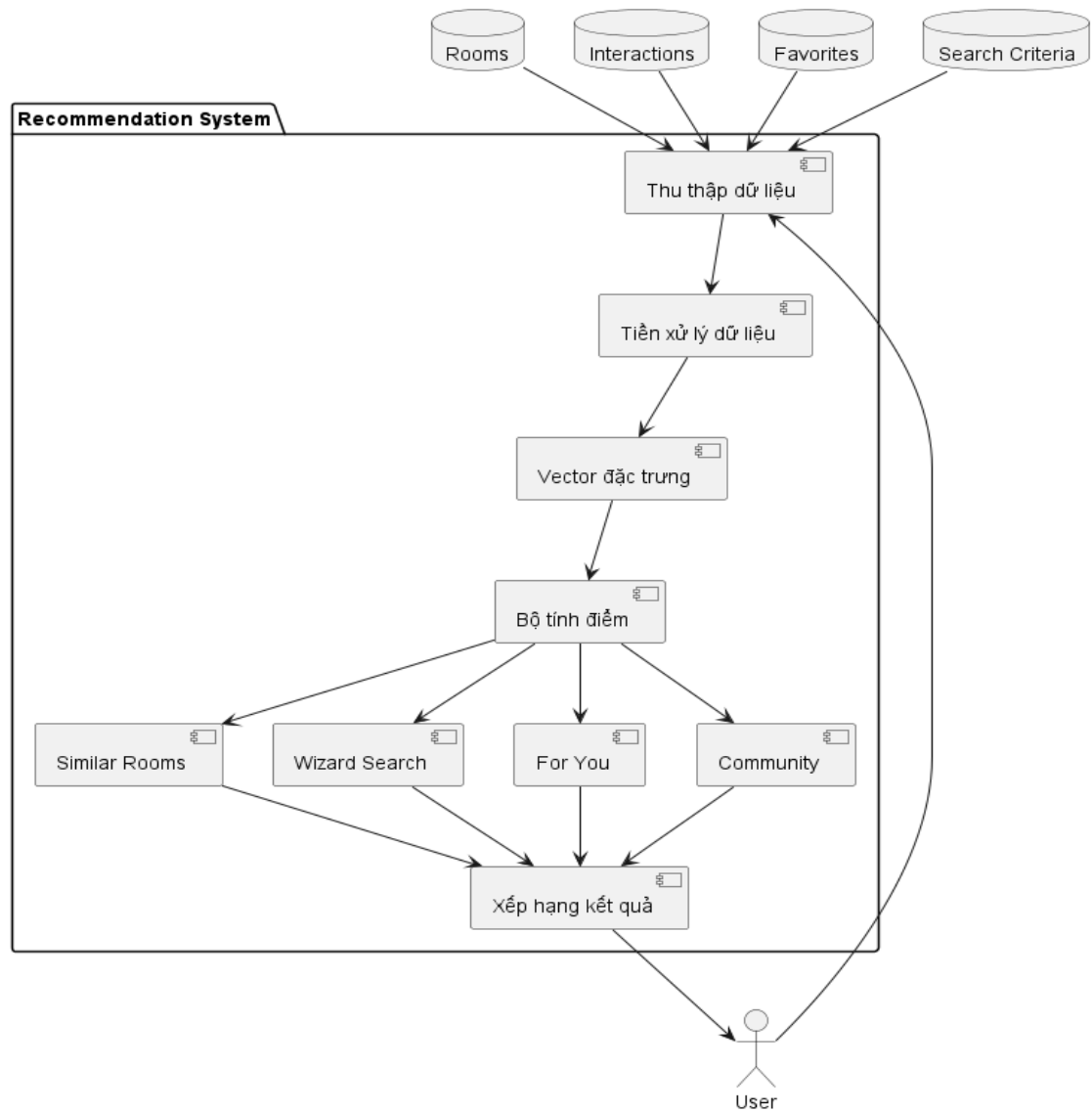
Với kiến trúc phân lớp này, Backend có tính mô-đun cao, giúp việc phát triển, bảo trì và bổ sung các chức năng mới trở nên thuận tiện hơn, đồng thời hạn chế ảnh hưởng đến các thành phần đang hoạt động của hệ thống.

3.3.4. Kiến trúc hệ thống gợi ý

3.3.4.1. Kiến trúc tổng thể hệ thống gợi ý

Hệ thống gợi ý được xây dựng nhằm hỗ trợ người dùng tìm kiếm và lựa chọn phòng trọ phù hợp dựa trên việc phân tích đặc điểm của phòng trọ, hành vi tương tác của người dùng, vị trí địa lý và mức độ phổ biến của phòng trên hệ thống. Thay vì áp dụng một phương pháp gợi ý duy nhất, hệ thống sử dụng mô hình gợi ý lai (Hybrid Recommendation), kết hợp nhiều kỹ thuật khác nhau để nâng cao độ chính xác và khả năng thích ứng với từng ngữ cảnh sử dụng.

Tùy thuộc vào thông tin đầu vào và trạng thái của người dùng, hệ thống sẽ lựa chọn một trong bốn thuật toán gợi ý, bao gồm: gợi ý phòng tương tự (Similar Rooms), gợi ý theo tiêu chí tìm kiếm (Wizard Search), gợi ý cá nhân hóa (For You) và gợi ý cộng đồng (Community Recommendation). Mỗi thuật toán khai thác các nguồn dữ liệu và bộ trọng số khác nhau, tuy nhiên đều tuân theo một quy trình xử lý thống nhất gồm: thu thập dữ liệu, tiền xử lý dữ liệu, xây dựng vector đặc trưng, tính toán các chỉ số đánh giá, tổng hợp điểm theo trọng số và xếp hạng kết quả để tạo danh sách phòng trọ gợi ý.



Hình 3.7 Kiến trúc tổng thể hệ thống gợi ý

Dữ liệu sử dụng trong hệ thống gợi ý được tổng hợp từ nhiều nguồn khác nhau nhằm phản ánh đặc điểm của phòng trọ cũng như nhu cầu và hành vi của người dùng.

Bảng 3.1. Nguồn dữ liệu sử dụng trong hệ thống gợi ý

Nguồn dữ liệu	Thuộc tính sử dụng	Mục đích
Rooms	Giá, diện tích, sức chứa, loại phòng, tiện ích, tọa độ GPS, lượt xem	Xây dựng vector đặc trưng và tính toán độ tương đồng
Favorites	Danh sách phòng yêu thích	Tính điểm chất lượng cộng đồng

Nguồn dữ liệu	Thuộc tính sử dụng	Mục đích
Interactions	View, Save, Chat, Booking, thời gian tương tác	Xây dựng hồ sơ sở thích người dùng và tính điểm cộng đồng
Tiêu chí tìm kiếm	Giá, diện tích, sức chứa, tiện ích, bán kính, GPS	Xây dựng vector tiêu chí tìm kiếm
Vị trí người dùng	Latitude, Longitude	Tính khoảng cách Haversine

Tiền xử lý dữ liệu

Do các thuộc tính của phòng trọ có đơn vị đo khác nhau như giá thuê (VNĐ), diện tích (m²) và sức chứa (người), hệ thống tiến hành chuẩn hóa dữ liệu bằng phương pháp Min-Max Normalization để đưa toàn bộ dữ liệu về cùng miền giá trị từ 0 đến 1.

Công thức Min-Max Normalization

$$f(x) = \frac{x - x_{\min}}{(x_{\max} - x_{\min}) + \varepsilon}$$

Trong đó:

x : giá trị cần chuẩn hóa.

$x_{\min}$: giá trị nhỏ nhất của thuộc tính.

$x_{\max}$: giá trị lớn nhất của thuộc tính.

$\varepsilon = 10^{-9}$ tránh phép chia cho 0

Đối với các thuộc tính phân loại như loại phòng trọ, hệ thống sử dụng kỹ thuật One-Hot Encoding để chuyển đổi thành các vector nhị phân.

Bảng 3.2. Vector nhị phân của thuộc tính loại phòng

Loại phòng	Vector One-Hot
Phòng trọ	(1,0,0,0)
Chung cư mini	(0,1,0,0)

Loại phòng	Vector One-Hot
Nhà nguyên căn	(0,0,1,0)
Ký túc xá	(0,0,0,1)

Các tiện ích của phòng trọ được biểu diễn bằng vector nhị phân gồm 14 chiều, trong đó giá trị 1 thể hiện phòng có tiện ích tương ứng và 0 thể hiện không có.

Xây dựng vector đặc trưng

Sau khi chuẩn hóa dữ liệu, mỗi phòng trọ được biểu diễn dưới dạng vector đặc trưng gồm 21 thuộc tính.

Bảng 3.3. Số chiều của 21 thuộc tính

Nhóm thuộc tính	Số chiều	Nhóm thuộc tính
Giá thuê	1	Giá thuê
Diện tích	1	Diện tích
Sức chứa	1	Sức chứa
Loại phòng (One-Hot)	4	Loại phòng (One-Hot)
Tiện ích	14	Tiện ích

Vector đặc trưng được xác định theo công thức:

$$v = [f_{price}, f_{area}, f_{capacity}, t_{type}, a_{amenities}] \circ w_{feature}$$

Trong đó:

f_{price} , f_{area} , $f_{capacity}$ là các thuộc tính số đã chuẩn hóa.

t_{type} là vector One-Hot của loại phòng.

$a_{amenities}$ là vector biểu diễn các tiện ích.

$w_{feature}$ là vector trọng số thuộc tính.

Vector trọng số được thiết lập như sau:

Nhóm thuộc tính	Trọng số
Giá thuê	2.0
Diện tích	2.0
Sức chứa	2.0
Loại phòng	1.5
Tiện ích	0.7

Việc gán trọng số giúp tăng mức ảnh hưởng của các thuộc tính quan trọng như giá thuê, diện tích và sức chứa trong quá trình tính toán độ tương đồng.

Các chỉ số đánh giá

Sau khi xây dựng vector đặc trưng, hệ thống tiến hành tính toán các chỉ số phục vụ cho quá trình xếp hạng.

- Độ tương đồng nội dung (Cosine Similarity)

Độ tương đồng giữa hai phòng trọ được xác định bằng công thức:

$$\text{Cosine}(A, B) = \frac{A \cdot B}{\|A\| \times \|B\|}$$

Trong đó:

A là vector đặc trưng của phòng thứ nhất.

B là vector đặc trưng của phòng thứ hai.

Giá trị Cosine Similarity nằm trong khoảng từ 0 đến 1; giá trị càng lớn thể hiện hai phòng càng giống nhau.

- Khoảng cách địa lý (Haversine Distance)

Khoảng cách giữa hai vị trí được tính bằng công thức Haversine nhằm xét đến độ cong của Trái Đất.

$$d = 2R \arcsin \left(\sqrt{\sin^2 \frac{\Delta\varphi}{2} + \cos \varphi_1 \cos \varphi_2 \sin^2 \frac{\Delta\lambda}{2}} \right)$$

Trong đó:

$R = 6371$ km là bán kính trung bình của Trái Đất.

φ là vĩ độ.

λ là kinh độ.

Khoảng cách sau đó được chuyển đổi thành Location Score trong khoảng từ 0 đến 1 để phục vụ quá trình xếp hạng.

- Điểm vị trí (Location Score)

Sau khi tính được khoảng cách địa lý bằng công thức Haversine, hệ thống chuyển đổi khoảng cách thực tế thành điểm vị trí (Location Score) trong khoảng từ 0 đến 1 nhằm đánh giá mức độ gần giữa hai vị trí. Các phòng nằm trong vùng ưu tiên sẽ nhận điểm tối đa, trong khi điểm sẽ giảm tuyến tính khi khoảng cách tăng và bằng 0 nếu vượt quá bán kính tìm kiếm.

Location Score được xác định theo công thức:

$$\text{locationScore} = \max \left(0, 1 - \frac{\text{dist} - \text{innerRadius}}{\text{radius} - \text{innerRadius}} \right)$$

Trong đó:

dist: khoảng cách thực tế giữa hai vị trí được tính theo công thức Haversine.

radius: bán kính tìm kiếm của thuật toán.

innerRadius: bán kính vùng ưu tiên, được xác định bằng:

$$\text{innerRadius} = 0.4 \times \text{radius}$$

Điểm Location Score càng lớn thì khoảng cách giữa hai vị trí càng gần, qua đó giúp ưu tiên các phòng trọ nằm gần vị trí tham chiếu trong quá trình xếp hạng.

- Điểm chất lượng cộng đồng (Quality Score)

Mức độ phổ biến của phòng trọ được xác định từ lượt xem và lượt yêu thích.

$$\text{Behavior} = 0.4 \times \frac{\text{View}}{\text{MaxView}} + 0.6 \times \frac{\text{Favorite}}{\text{MaxFavorite}}$$

Sau đó chuẩn hóa:

$$\text{QualityScore} = \frac{\text{Behavior}}{\text{Behavior}_{\max}}$$

Trong đó:

- View là số lượt xem của phòng trọ.
- Favorite là số lượt yêu thích của phòng trọ.
- MaxView là số lượt xem lớn nhất trong tập ứng viên.
- MaxFavorite là số lượt yêu thích lớn nhất trong tập ứng viên.
- Behavior_{max} là giá trị Behavior lớn nhất trong tập ứng viên.

Điểm Quality Score phản ánh sự quan tâm của cộng đồng đối với từng phòng trọ.

Sau khi hoàn thành quá trình thu thập dữ liệu, tiền xử lý, xây dựng vector đặc trưng và tính toán các chỉ số đánh giá, hệ thống lựa chọn thuật toán gợi ý phù hợp với từng ngữ cảnh sử dụng. Mặc dù bốn thuật toán gợi ý sử dụng các nguồn dữ liệu và bộ trọng số khác nhau, tất cả đều tuân theo quy trình xử lý thống nhất gồm xây dựng đặc trưng, tính toán điểm đánh giá, tổng hợp điểm theo trọng số và xếp hạng kết quả. Kiến trúc chi tiết của từng thuật toán được trình bày trong các mục tiếp theo.

3.3.4.2. Kiến trúc gợi ý phòng tương tự

Chức năng gợi ý phòng tương tự được xây dựng nhằm đề xuất cho người dùng các phòng trọ có đặc điểm gần giống với phòng đang được xem. Thuật toán này hỗ trợ người dùng dễ dàng so sánh nhiều phòng có cùng đặc tính về giá thuê, diện tích, loại phòng, tiện ích và vị trí địa lý, từ đó nâng cao khả năng lựa chọn phòng phù hợp.

Khi người dùng truy cập vào trang chi tiết của một phòng trọ, hệ thống tiếp nhận mã định danh (ID) của phòng và truy vấn toàn bộ thông tin của phòng mục tiêu từ cơ sở dữ liệu. Đồng thời, hệ thống xây dựng tập phòng ứng viên để phục vụ quá trình tính toán gợi ý.

Để đảm bảo luôn có đủ số lượng phòng phục vụ xếp hạng, tập ứng viên được xây dựng theo ba cấp như sau:

Bảng 3.4. Quy trình xây dựng tập ứng viên

Cấp	Điều kiện	Số lượng tối đa
Tier 1	Cùng loại phòng, khoảng cách ≤ 10 km	120
Tier 2	Khác loại phòng, khoảng cách ≤ 15 km	80
Tier 3	Các phòng gần nhất trên toàn hệ thống (khi chưa đủ số lượng)	Bổ sung đến đủ

Sau khi xác định tập ứng viên, hệ thống tiến hành xây dựng vector đặc trưng cho từng phòng. Mỗi phòng được biểu diễn bằng vector 21 chiều, bao gồm các thuộc tính giá thuê, diện tích, sức chứa, loại phòng và tiện ích. Các thuộc tính số được chuẩn hóa bằng phương pháp Min-Max Normalization, các thuộc tính phân loại được mã hóa bằng One-Hot Encoding, sau đó nhân với vector trọng số đặc trưng để tăng mức ảnh hưởng của các thuộc tính quan trọng.

Tiếp theo, hệ thống sử dụng Cosine Similarity để xác định mức độ tương đồng giữa phòng mục tiêu và từng phòng ứng viên:

$$\text{ContentScore} = \frac{v_{\text{target}} \cdot v_{\text{candidate}}}{\|v_{\text{target}}\| \times \|v_{\text{candidate}}\|}$$

Trong đó:

- v_{target} là vector đặc trưng của phòng đang xem.
- $v_{\text{candidate}}$ là vector đặc trưng của phòng ứng viên.

Bên cạnh mức độ tương đồng về nội dung, thuật toán còn xem xét khoảng cách địa lý giữa hai phòng. Khoảng cách được tính bằng công thức Haversine, sau đó được chuyển đổi thành điểm vị trí (Location Score). Đối với thuật toán này, hệ thống sử dụng bán kính tìm kiếm 10 km, trong đó các phòng nằm trong phạm vi 4 km sẽ nhận điểm tối đa bằng 1. Các phòng ở xa hơn sẽ có điểm vị trí giảm tuyến tính theo khoảng cách.

Ngoài yếu tố nội dung và vị trí, thuật toán còn đánh giá mức độ phổ biến của phòng thông qua điểm chất lượng cộng đồng (Quality Score). Giá trị này được tính từ số lượt

xem và số lượt yêu thích của từng phòng, sau đó chuẩn hóa về khoảng giá trị từ 0 đến 1 nhằm đảm bảo khả năng so sánh giữa các ứng viên.

Sau khi hoàn thành ba thành phần điểm, hệ thống sử dụng phương pháp Weighted Scoring để tính điểm xếp hạng cuối cùng theo công thức:

$$\text{FinalScore} = 0.55 \times \text{ContentScore} + 0.25 \times \text{LocationScore} + 0.20 \times \text{QualityScore}$$

Trong đó:

- Content Score (55%) phản ánh mức độ tương đồng giữa hai phòng trọ và là tiêu chí quan trọng nhất.
- Location Score (25%) đánh giá khoảng cách địa lý giữa phòng ứng viên và phòng mục tiêu.
- Quality Score (20%) phản ánh mức độ phổ biến của phòng dựa trên hành vi của cộng đồng người dùng.

Sau khi tính điểm cho toàn bộ tập ứng viên, hệ thống tiến hành sắp xếp các phòng theo giá trị Final Score giảm dần và lựa chọn Top N phòng có điểm cao nhất để trả về cho người dùng. Việc kết hợp đồng thời ba yếu tố gồm đặc điểm phòng trọ, vị trí địa lý và mức độ phổ biến giúp thuật toán không chỉ tìm được các phòng có đặc điểm tương đồng mà còn ưu tiên các phòng có vị trí thuận lợi và được nhiều người quan tâm, góp phần nâng cao chất lượng cũng như tính chính xác của kết quả gợi ý.

3.3.4.3. Kiến trúc gợi ý theo tiêu chí tìm kiếm

Chức năng gợi ý theo tiêu chí tìm kiếm (Wizard Search) được xây dựng nhằm hỗ trợ người dùng tìm kiếm các phòng trọ phù hợp với nhu cầu cụ thể thông qua việc thiết lập các tiêu chí như khoảng giá, diện tích, sức chứa, loại phòng, tiện ích, vị trí và bán kính tìm kiếm. Khác với chức năng gợi ý phòng tương tự, thuật toán này không dựa trên một phòng mục tiêu có sẵn mà sử dụng bộ tiêu chí do người dùng cung cấp để đánh giá và xếp hạng các phòng trọ trong hệ thống.

Khi người dùng thực hiện tìm kiếm, hệ thống tiếp nhận bộ tiêu chí tìm kiếm bao gồm loại phòng, khoảng giá, diện tích tối thiểu, sức chứa, danh sách tiện ích mong muốn, vị trí GPS và bán kính tìm kiếm. Các thông tin này được sử dụng để truy vấn cơ sở dữ liệu nhằm tạo tập phòng ứng viên ban đầu.

Để giảm khối lượng dữ liệu cần xử lý nhưng vẫn đảm bảo khả năng tìm được phòng phù hợp, hệ thống thực hiện lọc sơ bộ theo các điều kiện sau:

- Giá thuê được mở rộng trong khoảng $\pm 15\%$ so với mức giá người dùng lựa chọn.
- Diện tích và sức chứa của phòng phải lớn hơn hoặc bằng giá trị yêu cầu.
- Loại phòng phải phù hợp với lựa chọn của người dùng (nếu có).
- Nếu người dùng cung cấp vị trí GPS, chỉ lấy các phòng nằm trong bán kính.
- Số lượng phòng ứng viên tối đa được sử dụng để tính toán là 300 phòng.

Bảng 3.5. Dữ liệu đầu vào của thuật toán gợi ý theo tiêu chí tìm kiếm

Nguồn dữ liệu	Thuộc tính sử dụng	Mục đích
Rooms	Giá thuê, diện tích, sức chứa, loại phòng, tiện ích, tọa độ GPS	Xây dựng tập ứng viên
Tiêu chí tìm kiếm	Giá, diện tích, sức chứa, loại phòng, tiện ích, bán kính, GPS	Xây dựng vector tiêu chí
Favorites	Lượt yêu thích	Tính Quality Score
Rooms	Lượt xem	Tính Quality Score

Sau khi xây dựng tập ứng viên, hệ thống tiến hành biểu diễn bộ tiêu chí tìm kiếm thành vector đặc trưng có cùng cấu trúc với vector đặc trưng của phòng. Điều này giúp việc so sánh giữa tiêu chí tìm kiếm và từng phòng ứng viên được thực hiện trên cùng một không gian đặc trưng.

Đối với mỗi phòng ứng viên, hệ thống tính điểm tương đồng nội dung bằng độ tương đồng Cosine giữa vector tiêu chí và vector đặc trưng của phòng. Đồng thời, điểm này còn được nhân với hệ số đáp ứng tiện ích nhằm ưu tiên các phòng đáp ứng đầy đủ các tiện ích mà người dùng yêu cầu.

Điểm tương đồng được xác định theo công thức:

$$\text{ContentScore} = \text{Cosine}(v_{\text{criteria}}, v_{\text{candidate}}) \times \text{AmenityMatch}$$

Trong đó:

- v_{criteria} là vector biểu diễn bộ tiêu chí tìm kiếm.

- $V_{\text{candidate}}$ là vector đặc trưng của phòng ứng viên.

- AmenityMatch là tỷ lệ giữa số tiện ích phòng đáp ứng và tổng số tiện ích mà người dùng yêu cầu.

Nếu người dùng cung cấp vị trí GPS, hệ thống tiếp tục tính khoảng cách địa lý giữa vị trí người dùng và từng phòng ứng viên bằng công thức Haversine. Khoảng cách sau đó được chuyển đổi thành Location Score trong khoảng từ 0 đến 1 để đánh giá mức độ thuận tiện về vị trí.

Bên cạnh đó, hệ thống còn tính Quality Score dựa trên số lượt xem và lượt yêu thích của từng phòng nhằm phản ánh mức độ phổ biến của phòng trọ trong cộng đồng người dùng.

Sau khi tính toán các thành phần điểm, hệ thống sử dụng phương pháp Weighted Scoring để xác định điểm xếp hạng cuối cùng. Tùy thuộc vào việc người dùng có cung cấp vị trí GPS hay không, hệ thống sử dụng hai bộ trọng số khác nhau.

Đối với trường hợp người dùng cung cấp vị trí GPS, điểm xếp hạng được xác định theo công thức:

$$\text{FinalScore} = 0.35 \times \text{ContentScore} + 0.40 \times \text{LocationScore} + 0.25 \times \text{QualityScore}$$

Trong trường hợp người dùng không cung cấp vị trí GPS, hệ thống chỉ sử dụng hai thành phần là mức độ phù hợp theo tiêu chí và chất lượng cộng đồng:

$$\text{FinalScore} = 0.65 \times \text{ContentScore} + 0.35 \times \text{QualityScore}$$

Trong đó:

- Content Score: đánh giá mức độ tương đồng giữa hai phòng.
- Location Score: đánh giá khoảng cách địa lý.
- Quality Score: đánh giá mức độ phổ biến của phòng.

Bảng 3.6. Trọng số của thuật toán gợi ý theo tiêu chí tìm kiếm

Thành phần	Có GPS	Không GPS
Content Score	0.35	0.65
Location Score	0.40	0.00

Thành phần	Có GPS	Không GPS
Quality Score	0.25	0.35
Tổng	1.00	1.00

Sau khi tính toán điểm tổng hợp, toàn bộ tập ứng viên được sắp xếp theo giá trị Final Score giảm dần và hệ thống trả về danh sách các phòng trọ có điểm số cao nhất cho người dùng.

3.3.4.4. Kiến trúc gợi ý cá nhân hóa

Chức năng gợi ý cá nhân hóa (For You) được xây dựng nhằm đề xuất các phòng trọ phù hợp với sở thích và hành vi của từng người dùng đã đăng nhập. Khác với hai thuật toán trước, thuật toán này không chỉ dựa trên đặc điểm của phòng trọ hoặc tiêu chí tìm kiếm hiện tại mà còn khai thác lịch sử tương tác của người dùng để xây dựng hồ sơ sở thích (User Profile). Việc kết hợp giữa sở thích cá nhân, vị trí địa lý và mức độ phổ biến của phòng giúp hệ thống tạo ra các kết quả gợi ý mang tính cá nhân hóa cao, phù hợp với nhu cầu thực tế của từng người dùng.

Khi người dùng đăng nhập và truy cập chức năng gợi ý, hệ thống trước tiên truy vấn tối đa 30 bản ghi tương tác gần nhất của người dùng từ bảng Interactions. Các hành vi được sử dụng gồm View, Save, Chat và Booking. Đồng thời, hệ thống lấy tối đa 300 phòng trọ từ cơ sở dữ liệu làm tập ứng viên và loại bỏ hoàn toàn các phòng mà người dùng đã từng tương tác nhằm tránh gợi ý lặp lại.

Bảng 3.7. Dữ liệu đầu vào của thuật toán gợi ý cá nhân hóa

Nguồn dữ liệu	Thuộc tính sử dụng	Mục đích
Rooms	Giá, diện tích, sức chứa, loại phòng, tiện ích, tọa độ GPS, lượt xem	Xây dựng tập ứng viên và vector đặc trưng
Interactions	View, Save, Chat, Booking, thời gian tương tác	Xây dựng hồ sơ sở thích người dùng
Favorites	Danh sách phòng yêu thích	Tính Quality Score
Vị trí người dùng	Latitude, Longitude	Tính khoảng cách địa lý

Sau khi thu thập dữ liệu, hệ thống xây dựng hồ sơ sở thích người dùng (User Profile) từ lịch sử tương tác. Mỗi hành vi tương tác được gán một trọng số khác nhau nhằm phản ánh mức độ quan tâm của người dùng đối với từng phòng trọ. Đồng thời, thuật toán áp dụng cơ chế Time Decay để giảm dần ảnh hưởng của các tương tác cũ theo thời gian.

Trọng số của từng tương tác được tính theo công thức:

$$w_i = \text{ActionWeight}_i \times \text{count}_i \times 0.5^{\frac{\Delta t_i}{7}}$$

Trong đó:

- ActionWeight_i : trọng số của loại hành vi.
- count_i : số lần thực hiện hành vi.
- Δt_i : số ngày kể từ lần tương tác gần nhất.

Bảng 3.8. Trọng số các hành vi tương tác

Hành vi	Trọng số
View	1
Save	2
Chat	3
Booking	4

Từ các trọng số trên, hệ thống xây dựng vector sở thích của người dùng bằng trung bình có trọng số của các vector đặc trưng phòng đã tương tác.

$$v_{\text{user}} = \frac{\sum w_i \cdot v_{\text{room}_i}}{\sum w_i}$$

Trong đó:

- v_{room_i} là Vector đặc trưng của phòng đã tương tác.
- w_i là trọng số của lần tương tác thứ i.
- v_{user} là vector sở thích đại diện cho người dùng.

Sau khi xây dựng hồ sơ sở thích, hệ thống tính mức độ phù hợp giữa người dùng và từng phòng ứng viên bằng độ tương đồng Cosine.

$$\text{PersonalScore} = \text{Cosine}(v_{\text{user}}, v_{\text{candidate}})$$

Tiếp theo, hệ thống tính Location Score từ khoảng cách Haversine giữa vị trí hiện tại của người dùng và vị trí của phòng trọ. Đối với thuật toán này, bán kính mặc định được sử dụng là 5 km, trong đó các phòng nằm trong phạm vi 2 km sẽ nhận điểm vị trí tối đa.

Ngoài ra, hệ thống tiếp tục tính Quality Score từ lượt xem và lượt yêu thích của từng phòng nhằm đánh giá mức độ phổ biến trong cộng đồng.

Sau khi hoàn thành việc tính toán các thành phần điểm, hệ thống sử dụng phương pháp Weighted Scoring để xác định điểm xếp hạng cuối cùng.

Điểm cuối cùng của mỗi phòng được xác định theo công thức:

$$\text{FinalScore} = w_l \times \text{LocationScore} + w_p \times \text{PersonalScore} + w_q \times \text{QualityScore}$$

Trong đó:

- w_l : trọng số vị trí.
- w_p : trọng số sở thích cá nhân.
- w_q : trọng số chất lượng cộng đồng.

Do lượng dữ liệu của mỗi người dùng có thể khác nhau, hệ thống sử dụng các bộ trọng số khác nhau tùy theo trạng thái dữ liệu như Bảng 3.x.

Bảng 3.9. Trọng số của thuật toán gợi ý cá nhân hóa

Trường hợp	Personal Score	Location Score	Quality Score
Có lịch sử tương tác + Có GPS	0.40	0.40	0.20
Có lịch sử tương tác + Không GPS	0.70	0.00	0.30

Trường hợp	Personal Score	Location Score	Quality Score
Không có lịch sử + Có GPS	0.00	0.70	0.30
Không có lịch sử + Không GPS	0.00	0.00	1.00

Việc sử dụng nhiều bộ trọng số giúp thuật toán có khả năng thích ứng với từng tình huống dữ liệu. Khi người dùng có đầy đủ lịch sử tương tác và thông tin vị trí, hệ thống ưu tiên đồng thời sở thích cá nhân và khoảng cách địa lý. Ngược lại, nếu thiếu một hoặc cả hai nguồn dữ liệu, trọng số sẽ được tự động điều chỉnh để vẫn đảm bảo chất lượng gợi ý.

Sau khi tính điểm cho toàn bộ tập ứng viên, hệ thống sắp xếp các phòng theo giá trị Final Score giảm dần và lựa chọn Top N phòng có điểm cao nhất để hiển thị cho người dùng. Kiến trúc này cho phép hệ thống học được sở thích của từng người dùng theo thời gian, đồng thời kết hợp với yếu tố vị trí và chất lượng phòng nhằm tạo ra các kết quả gợi ý mang tính cá nhân hóa cao, góp phần nâng cao trải nghiệm và tăng khả năng người dùng lựa chọn được phòng trọ phù hợp.

3.3.4.5. Kiến trúc gợi ý cộng đồng

Chức năng gợi ý cộng đồng (Community Recommendation) được xây dựng nhằm đề xuất các phòng trọ nổi bật và được nhiều người quan tâm trên hệ thống, đặc biệt dành cho người dùng chưa đăng nhập hoặc chưa có đủ dữ liệu để thực hiện gợi ý cá nhân hóa. Thuật toán này khai thác mức độ phổ biến của phòng trọ thông qua các chỉ số tương tác của cộng đồng kết hợp với vị trí địa lý hiện tại của người dùng (nếu có). Nhờ đó, hệ thống vẫn có thể cung cấp danh sách phòng trọ chất lượng ngay cả khi chưa có thông tin về sở thích cá nhân.

Khi người dùng truy cập hệ thống, thuật toán trước tiên kiểm tra xem người dùng có chia sẻ vị trí GPS hay không. Nếu có thông tin vị trí, hệ thống truy vấn tối đa 150 phòng trọ gần vị trí người dùng nhất để làm tập ứng viên. Ngược lại, nếu không có dữ liệu GPS, hệ thống sẽ lựa chọn 150 phòng có lượt xem cao nhất trên toàn hệ thống làm tập ứng viên.

Bảng 3.10. Dữ liệu đầu vào của thuật toán gợi ý cộng đồng

Nguồn dữ liệu	Thuộc tính sử dụng	Mục đích
Rooms	Tọa độ GPS, lượt xem	Xây dựng tập ứng viên và tính điểm cộng đồng
Favorites	Lượt yêu thích	Tính điểm cộng đồng
Interactions	Chat, Booking	Tính điểm cộng đồng
Vị trí người dùng	Latitude, Longitude	Tính khoảng cách địa lý

Sau khi xây dựng tập ứng viên, hệ thống tính điểm tương tác cộng đồng (Behavior Score) cho từng phòng trọ. Điểm này phản ánh mức độ phổ biến của phòng dựa trên bốn loại hành vi của toàn bộ người dùng trên hệ thống, bao gồm lượt xem, lượt yêu thích, số cuộc trò chuyện và số lượt đặt lịch xem phòng.

Điểm tương tác cộng đồng được xác định theo công thức:

BehaviorScore

$$= 0.1 \times \frac{\text{Views}}{\text{MaxViews}} + 0.2 \times \frac{\text{Favorites}}{\text{MaxFavorites}} + 0.3 \times \frac{\text{Chats}}{\text{MaxChats}} + 0.4 \times \frac{\text{Bookings}}{\text{MaxBookings}}$$

Trong đó:

- Views là tổng số lượt xem của phòng.
- Favorites là tổng số lượt lưu yêu thích.
- Chats là tổng số lượt nhắn tin với chủ trọ.
- Bookings là tổng số lượt đặt lịch xem phòng.
- MaxViews, MaxFavorites, MaxChats và MaxBookings là giá trị lớn nhất của từng chỉ số trong tập ứng viên.

Việc gán trọng số lớn hơn cho Booking và Chat giúp thuật toán ưu tiên những phòng có khả năng chuyển đổi thành giao dịch thực tế thay vì chỉ có nhiều lượt xem.

Bảng 3.11. Trọng số các hành vi cộng đồng

Chỉ số	Trọng số
Lượt xem (Views)	0.10
Lượt yêu thích (Favorites)	0.20
Lượt nhắn tin (Chats)	0.30
Lượt đặt lịch (Bookings)	0.40

Nếu người dùng cho phép truy cập vị trí, hệ thống tiếp tục tính Location Score dựa trên khoảng cách Haversine giữa vị trí người dùng và phòng trọ. Khoảng cách sau đó được chuyển đổi về thang điểm từ 0 đến 1.

Sau khi tính toán các thành phần điểm, hệ thống sử dụng phương pháp Weighted Scoring để xác định điểm xếp hạng cuối cùng.

Điểm xếp hạng cuối cùng của mỗi phòng được xác định theo công thức:

Trường hợp có vị trí GPS

$$\text{FinalScore} = 0.60 \times \text{LocationScore} + 0.40 \times \text{BehaviorScore}$$

Trường hợp không có vị trí GPS

$$\text{FinalScore} = \text{BehaviorScore}$$

Trong đó:

- Location Score phản ánh mức độ gần giữa phòng trọ và vị trí người dùng.
- Behavior Score phản ánh mức độ phổ biến của phòng trên toàn hệ thống.

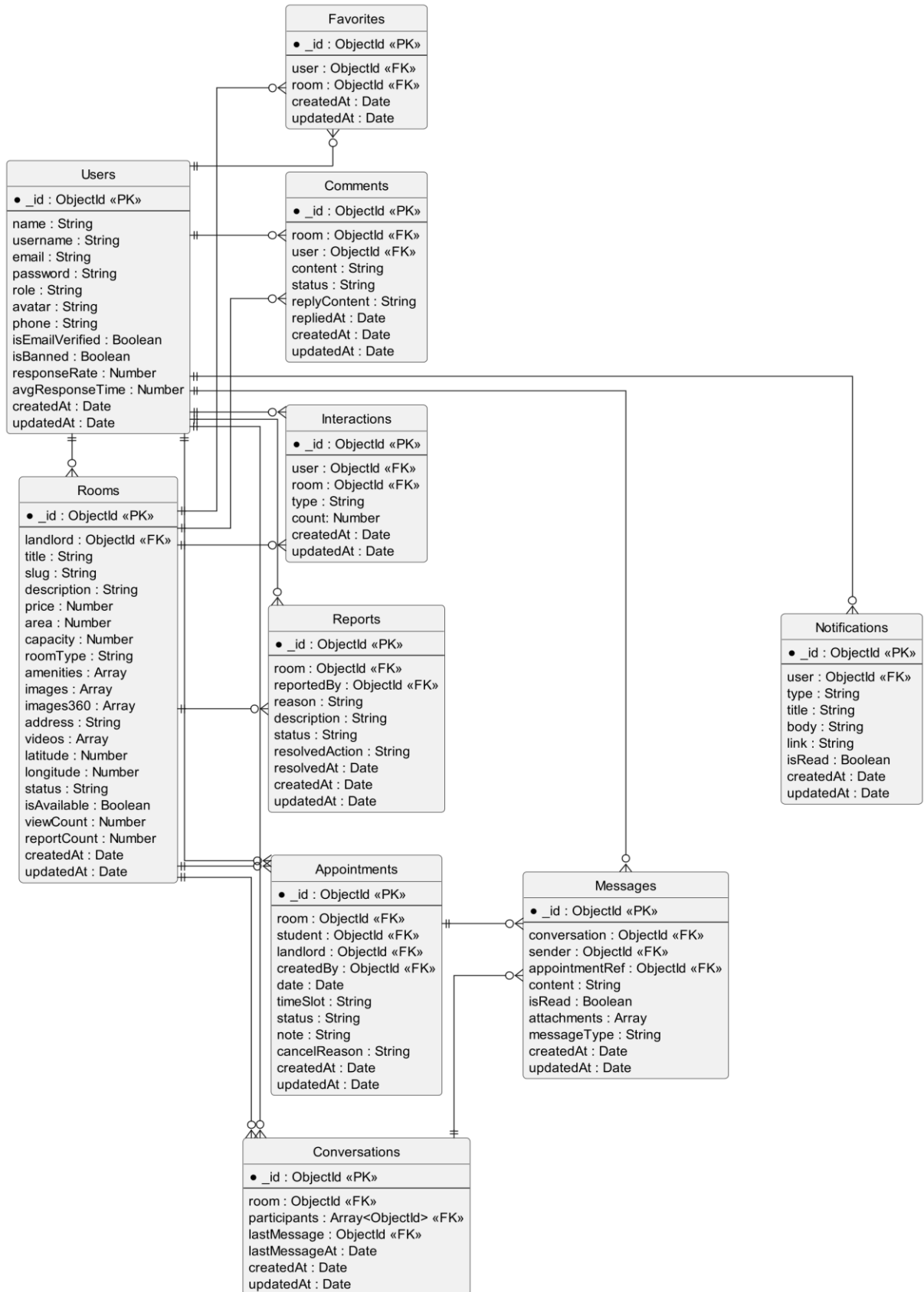
Bảng 3.12. Trọng số của thuật toán gợi ý cộng đồng

Thành phần	Có GPS	Không GPS
Location Score	0.60	0.00
Behavior Score	0.40	1.00

Sau khi tính điểm cho toàn bộ tập ứng viên, hệ thống sắp xếp các phòng theo giá trị Final Score giảm dần và lựa chọn Top N phòng có điểm cao nhất để hiển thị cho người dùng.

3.4. Thiết kế cơ sở dữ liệu

3.4.1. Mô hình dữ liệu tổng thể



Hình 3.8. Mô hình dữ liệu tổng thể

3.4.2. Collection Users

Collection Users lưu trữ thông tin tài khoản của người dùng trong hệ thống, bao gồm sinh viên, chủ trọ và quản trị viên. Collection này phục vụ cho việc xác thực đăng nhập, quản lý hồ sơ cá nhân, theo dõi hoạt động và phân quyền sử dụng hệ thống.

Bảng 3.13. Collection Users

Tên trường	Kiểu dữ liệu	Mô tả
_id	ObjectId	Khóa chính
name	String	Họ và tên
username	String	Tên đăng nhập
email	String	Địa chỉ email
password	String	Mật khẩu đã mã hóa
role	String	Vai trò người dùng
avatar	String	Ảnh đại diện
phone	String	Số điện thoại
isEmailVerified	Boolean	Trạng thái xác thực email
isBanned	Boolean	Trạng thái khóa tài khoản
responseRate	Number	Tỷ lệ phản hồi
avgResponseTime	Number	Thời gian phản hồi trung bình
createdAt	Date	Ngày tạo
updatedAt	Date	Ngày cập nhật

3.4.3. Collection Rooms

Collection Rooms lưu trữ thông tin các phòng trọ được đăng trên hệ thống. Đây là Collection trung tâm phục vụ chức năng tìm kiếm, gợi ý và quản lý phòng trọ.

Bảng 3.14.Collection Rooms

Tên trường	Kiểu dữ liệu	Mô tả
_id	ObjectId	Khóa chính
landlord	ObjectId	Chủ sở hữu phòng trọ
title	String	Tiêu đề phòng trọ
slug	String	Đường dẫn SEO
description	String	Mô tả phòng trọ
price	Number	Giá thuê
area	Number	Diện tích
capacity	Number	Sức chứa
roomType	String	Loại phòng
amenities	Array	Tiện ích
images	Array	Danh sách hình ảnh
images360	Array	Hình ảnh 360 độ
address	String	Địa chỉ
videos	Array	Video giới thiệu
latitude	Number	Vĩ độ
longitude	Number	Kinh độ
status	String	Trạng thái
isAvailable	Boolean	Tình trạng còn trống
viewCount	Number	Lượt xem
reportCount	Number	Số lần báo cáo
createdAt	Date	Ngày tạo
updatedAt	Date	Ngày cập nhật

3.4.4. Collection Appointments

Collection Appointments lưu trữ thông tin lịch hẹn xem phòng giữa sinh viên và chủ trọ.

Bảng 3.15.Collection Appointments

Tên trường	Kiểu dữ liệu	Mô tả
_id	ObjectId	Khóa chính
room	ObjectId	Phòng trọ được đặt lịch
student	ObjectId	Sinh viên
landlord	ObjectId	Chủ trọ
createdBy	ObjectId	Người tạo lịch hẹn
date	Date	Ngày hẹn
timeSlot	String	Khung giờ hẹn
status	String	Trạng thái
note	String	Ghi chú
cancelReason	String	Lý do hủy
createdAt	Date	Ngày tạo
updatedAt	Date	Ngày cập nhật

3.4.5. Collection Favorites

Collection Favorites lưu trữ danh sách các phòng trọ được người dùng đánh dấu yêu thích nhằm hỗ trợ việc xem lại và quản lý các phòng trọ quan tâm.

Bảng 3.16.Collection Favorites

Tên trường	Kiểu dữ liệu	Mô tả
_id	ObjectId	Khóa chính
user	ObjectId	Mã người dùng
room	ObjectId	Mã phòng trọ

Tên trường	Kiểu dữ liệu	Mô tả
createdAt	Date	Ngày tạo
updatedAt	Date	Ngày cập nhật

3.4.6. Collection Comments

Collection Comments lưu trữ các bình luận và phản hồi của người dùng đối với các phòng trọ trên hệ thống. Thông tin trong Collection này giúp người dùng tham khảo đánh giá từ cộng đồng, đồng thời hỗ trợ chủ trọ tương tác với khách hàng tiềm năng.

Bảng 3.17. Collection Comments

Tên trường	Kiểu dữ liệu	Mô tả
_id	ObjectId	Khóa chính
room	ObjectId	Mã phòng trọ
user	ObjectId	Mã người dùng bình luận
content	String	Nội dung bình luận
status	String	Trạng thái bình luận
replyContent	String	Nội dung phản hồi
repliedAt	Date	Thời gian phản hồi
createdAt	Date	Ngày tạo
updatedAt	Date	Ngày cập nhật

3.4.7. Collection Interactions

Collection Interactions lưu trữ lịch sử tương tác của người dùng với các phòng trọ trên hệ thống. Dữ liệu này được sử dụng làm cơ sở cho hệ thống gợi ý phòng trọ cá nhân hóa.

Bảng 3.18. Collection Interactions

Tên trường	Kiểu dữ liệu	Mô tả
_id	ObjectId	Khóa chính
user	ObjectId	Mã người dùng
room	ObjectId	Mã phòng trọ
type	String	Loại tương tác
Count	Number	Số lần tương tác (View)
createdAt	Date	Ngày tạo
updatedAt	Date	Ngày cập nhật

3.4.8. Collection Reports

Collection Reports lưu trữ thông tin các báo cáo vi phạm liên quan đến phòng trọ được gửi bởi người dùng. Dữ liệu này giúp quản trị viên kiểm duyệt nội dung và đảm bảo chất lượng hệ thống.

Bảng 3.19. Collection Reports

Tên trường	Kiểu dữ liệu	Mô tả
_id	ObjectId	Khóa chính
room	ObjectId	Mã phòng trọ bị báo cáo
reportedBy	ObjectId	Mã người dùng báo cáo
reason	String	Lý do báo cáo
description	String	Mô tả chi tiết
status	String	Trạng thái xử lý
resolvedAction	String	Hành động xử lý
resolvedAt	Date	Thời gian xử lý
createdAt	Date	Ngày tạo
updatedAt	Date	Ngày cập nhật

3.4.9. Collection Conversations

Collection Conversations lưu trữ thông tin các cuộc trò chuyện giữa sinh viên và chủ trọ. Mỗi cuộc trò chuyện có thể bao gồm nhiều tin nhắn khác nhau.

Bảng 3.20. Collection Conversations

Tên trường	Kiểu dữ liệu	Mô tả
_id	ObjectId	Khóa chính
room	ObjectId	Mã phòng trọ
participants	Array<ObjectId>	Danh sách người tham gia
lastMessage	ObjectId	Tin nhắn cuối cùng
lastMessageAt	Date	Thời gian tin nhắn cuối
createdAt	Date	Ngày tạo
updatedAt	Date	Ngày cập nhật

3.4.10. Collection Messages

Collection Messages lưu trữ nội dung trao đổi giữa các người dùng trong cuộc trò chuyện. Đây là Collection phục vụ chức năng nhắn tin trực tuyến của hệ thống.

Bảng 3.21. Collection Messages

Tên trường	Kiểu dữ liệu	Mô tả
_id	ObjectId	Khóa chính
conversation	ObjectId	Mã cuộc trò chuyện
sender	ObjectId	Mã người gửi
appointmentRef	ObjectId	Mã lịch hẹn liên quan
content	String	Nội dung tin nhắn
isRead	Boolean	Trạng thái đã đọc
attachments	Array	Tệp đính kèm
messageType	String	Loại tin nhắn

Tên trường	Kiểu dữ liệu	Mô tả
createdAt	Date	Ngày tạo
updatedAt	Date	Ngày cập nhật

3.4.11. Collection Notifications

Collection Notifications lưu trữ các thông báo được gửi đến người dùng như thông báo lịch hẹn, tin nhắn mới, phản hồi bình luận hoặc các thông báo từ hệ thống.

Bảng 3.22. Collection Notifications

Tên trường	Kiểu dữ liệu	Mô tả
_id	ObjectId	Khóa chính
user	ObjectId	Mã người nhận
type	String	Loại thông báo
title	String	Tiêu đề thông báo
body	String	Nội dung thông báo
link	String	Đường dẫn liên kết
isRead	Boolean	Trạng thái đã đọc
createdAt	Date	Ngày tạo
updatedAt	Date	Ngày cập nhật

CHƯƠNG 4. KẾT QUẢ NGHIÊN CỨU

4.1. Môi trường phát triển

Để xây dựng hệ thống hỗ trợ tìm kiếm và gợi ý phòng trọ sinh viên, đề tài sử dụng các công cụ hiện đại nhằm đảm bảo khả năng phát triển, triển khai và vận hành hệ thống một cách hiệu quả. Môi trường phát triển được xây dựng theo mô hình Fullstack JavaScript với ReactJS ở phía giao diện người dùng, NodeJS và ExpressJS ở phía máy chủ, cùng với MongoDB làm hệ quản trị cơ sở dữ liệu.

4.1.1. Công cụ phát triển

Trong quá trình thực hiện đề tài, nhiều công cụ phát triển đã được sử dụng nhằm hỗ trợ xây dựng, kiểm thử và quản lý hệ thống. Visual Studio Code (VS Code) được lựa chọn làm môi trường lập trình chính để phát triển cả giao diện người dùng (Frontend) và phía máy chủ (Backend). MongoDB Compass được sử dụng để quản lý, theo dõi và kiểm tra dữ liệu trong cơ sở dữ liệu MongoDB. Bên cạnh đó, Postman hỗ trợ kiểm thử các RESTful API trong quá trình phát triển, giúp đảm bảo tính chính xác của các chức năng. Để quản lý phiên bản mã nguồn và lưu trữ dự án, hệ thống sử dụng Git kết hợp với GitHub. Ngoài ra, trình duyệt Google Chrome được sử dụng để kiểm tra giao diện, đánh giá khả năng tương thích và hỗ trợ gỡ lỗi trong quá trình phát triển hệ thống.

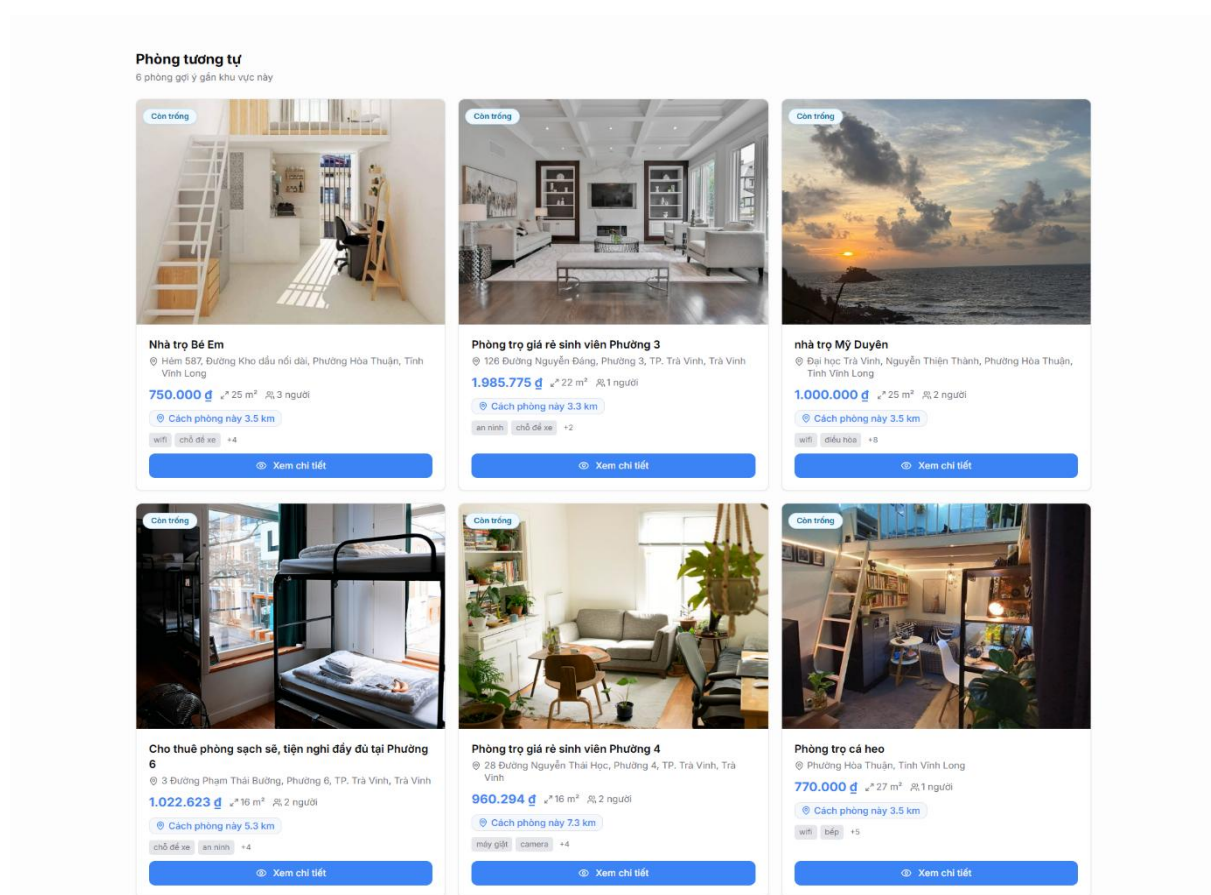
4.1.2. Môi trường triển khai

Hệ thống được triển khai trên máy chủ sử dụng hệ điều hành Ubuntu Server, trong đó cả Frontend và Backend được cài đặt và vận hành trên cùng một máy chủ, còn MongoDB được sử dụng làm cơ sở dữ liệu trung tâm để lưu trữ toàn bộ dữ liệu của hệ thống. Máy chủ ứng dụng được xây dựng trên nền tảng NodeJS, kết hợp với Nginx làm máy chủ web và bộ cân bằng tải để tiếp nhận, chuyển tiếp các yêu cầu từ người dùng đến ứng dụng. Để đảm bảo tính bảo mật trong quá trình truyền tải dữ liệu, hệ thống sử dụng giao thức HTTPS. Đồng thời, PM2 được sử dụng để quản lý tiến trình, duy trì hoạt động liên tục của ứng dụng và hỗ trợ tự động khởi động lại khi xảy ra sự cố. Môi trường triển khai này đáp ứng tốt các yêu cầu về hiệu năng, khả năng mở rộng và tính ổn định của hệ thống trong quá trình vận hành thực tế.

4.2. Kết quả xây dựng hệ thống gợi ý phòng trọ

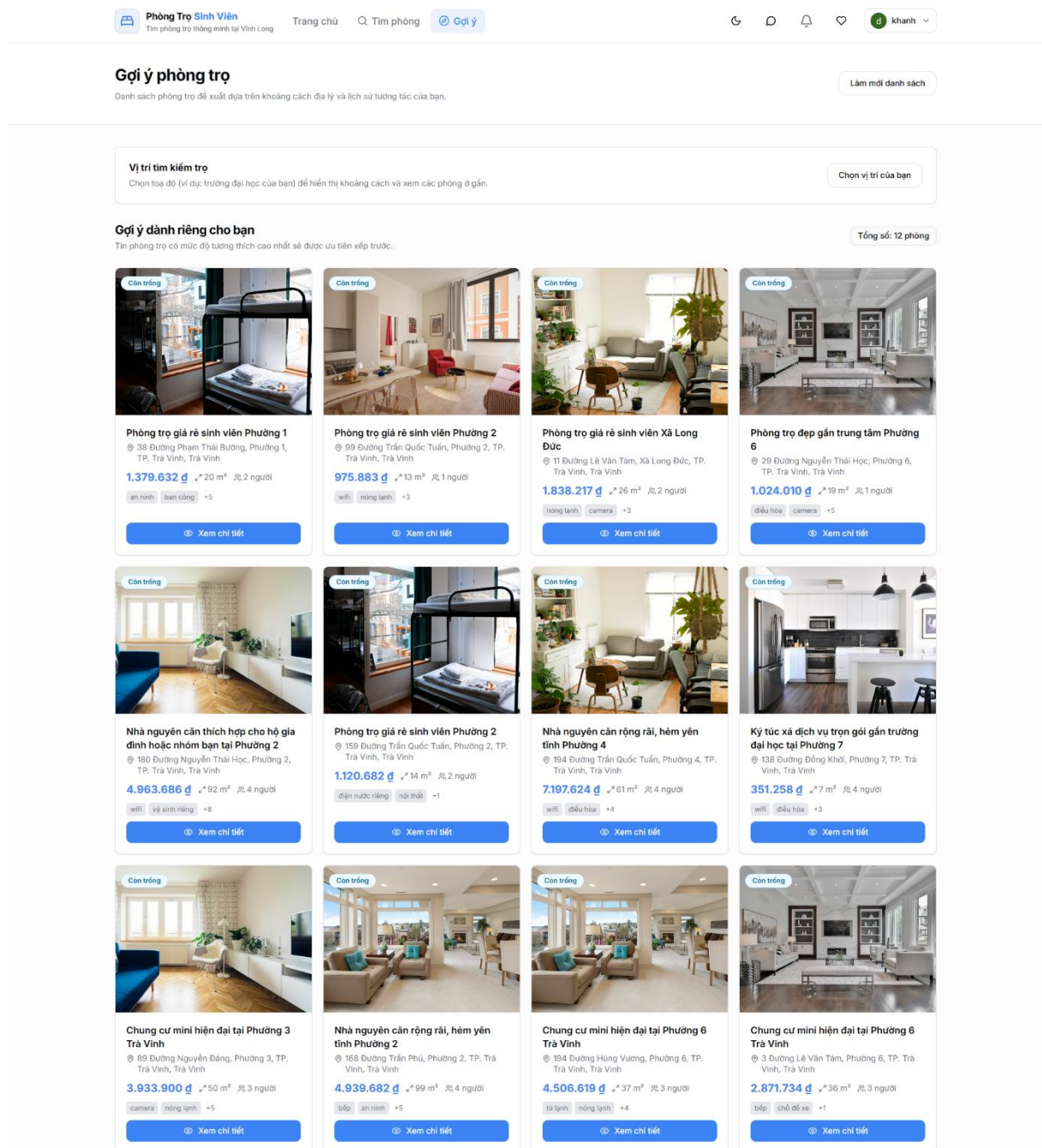
Dựa trên kiến trúc Hybrid Recommendation đã được thiết kế, đề tài đã xây dựng thành công hệ thống gợi ý phòng trọ và tích hợp trực tiếp vào hệ thống tìm kiếm phòng trọ. Hệ thống có khả năng phân tích dữ liệu phòng trọ, lịch sử tương tác của người dùng và thông tin vị trí địa lý để tạo ra các đề xuất phù hợp với nhu cầu thực tế.

Hệ thống gợi ý được triển khai dưới dạng các API độc lập, cho phép hỗ trợ nhiều kịch bản sử dụng khác nhau. Kết quả triển khai cho phép hệ thống thực hiện các chức năng chính như gợi ý phòng trọ tương tự, gợi ý theo tiêu chí tìm kiếm, gợi ý cá nhân hóa và gợi ý cho người dùng mới.



Hình 4.1. Chức năng gợi ý phòng trọ tương tự trên hệ thống

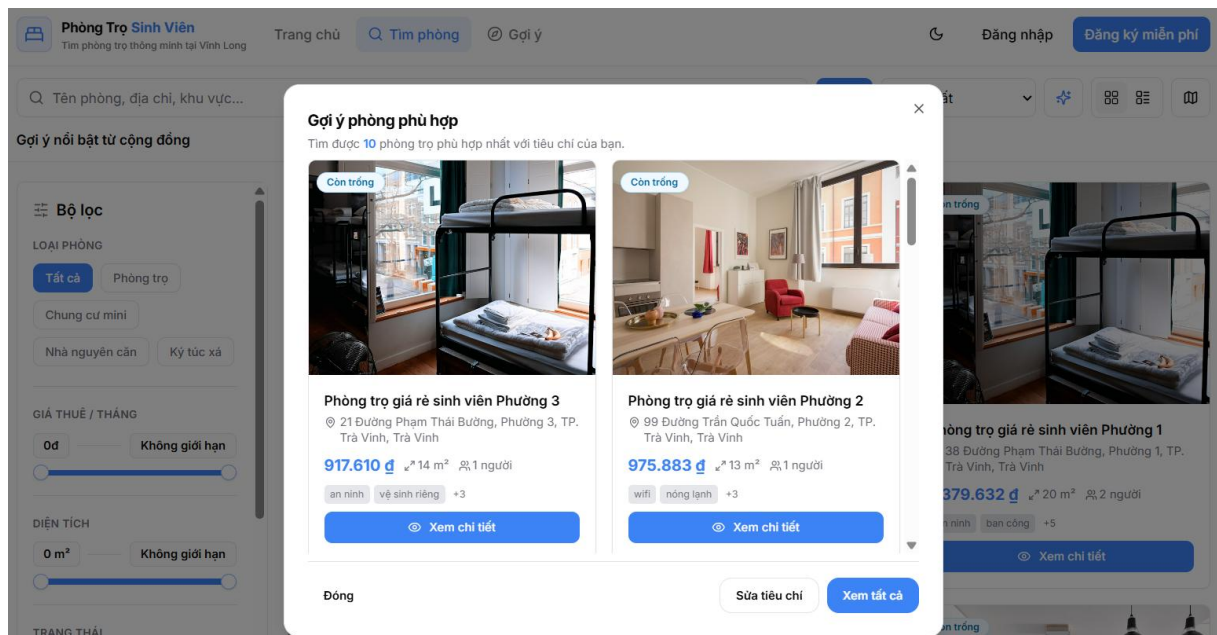
Đối với chức năng gợi ý phòng trọ tương tự, hệ thống đề xuất các phòng trọ có đặc điểm gần giống với phòng mà người dùng đang xem. Việc gợi ý được thực hiện dựa trên các thuộc tính của phòng trọ như giá thuê, diện tích, loại phòng, tiện ích và vị trí.



Hình 4.2. Chức năng gợi ý phòng trọ cá nhân hóa

Đối với người dùng đã đăng nhập, hệ thống sử dụng lịch sử xem phòng, danh sách yêu thích, lịch sử nhắn tin và lịch hẹn xem phòng để xây dựng hồ sơ sở thích cá nhân. Trên cơ sở đó, hệ thống đưa ra các đề xuất phù hợp với hành vi và nhu cầu của từng người dùng.

Ngoài ra, hệ thống còn hỗ trợ gợi ý cho người dùng mới thông qua dữ liệu vị trí địa lý và mức độ quan tâm của cộng đồng. Giải pháp này giúp người dùng vẫn nhận được các đề xuất phù hợp ngay cả khi chưa phát sinh dữ liệu tương tác trên hệ thống.



Hình 4.3. Chức năng gợi ý phòng trọ theo tiêu chí tìm kiếm

Đối với chức năng gợi ý theo tiêu chí tìm kiếm, hệ thống sử dụng các thông tin do người dùng cung cấp thông qua bộ lọc tìm kiếm để lựa chọn và sắp xếp các phòng trọ phù hợp nhất. Các tiêu chí có thể bao gồm giá thuê, diện tích, loại phòng, tiện ích và khu vực mong muốn.

Kết quả thực nghiệm cho thấy hệ thống hoạt động ổn định với tập dữ liệu thực tế và có khả năng cập nhật kết quả gợi ý theo thời gian thực. Các phòng trọ được đề xuất có mức độ phù hợp cao hơn so với phương pháp tìm kiếm thông thường, góp phần hỗ trợ người dùng rút ngắn thời gian tìm kiếm và nâng cao hiệu quả lựa chọn phòng trọ.

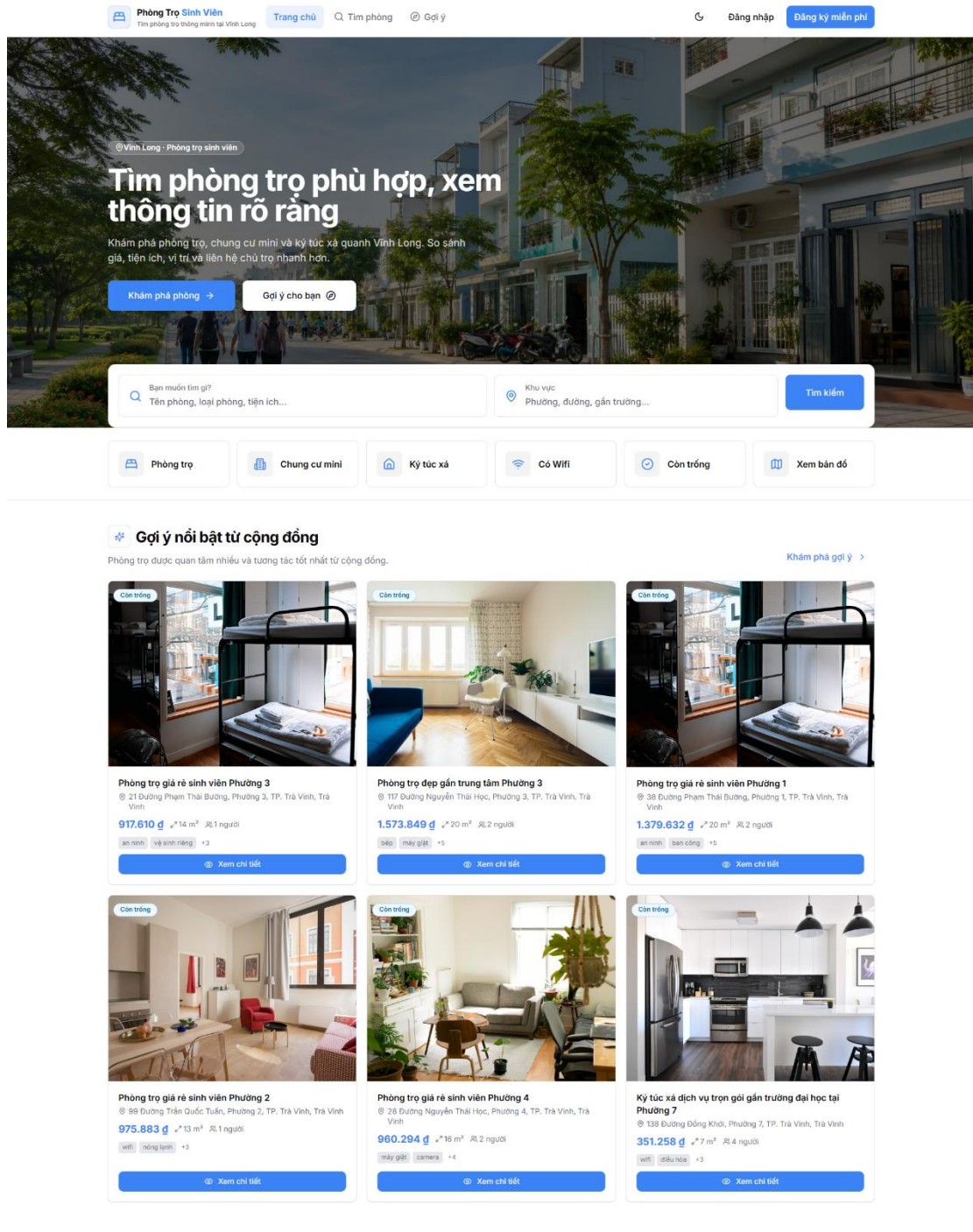
4.3. Giao diện hệ thống

4.3.1. Trang chủ

Trang chủ là giao diện đầu tiên khi người dùng truy cập vào hệ thống, đóng vai trò giới thiệu và điều hướng đến các chức năng chính. Giao diện được thiết kế trực quan, giúp người dùng dễ dàng tìm kiếm và tiếp cận các thông tin về phòng trọ.

Tại trang chủ, hệ thống hiển thị danh sách các phòng trọ nổi bật, phòng trọ mới đăng và hỗ trợ tìm kiếm theo các tiêu chí như địa điểm, mức giá, diện tích hoặc từ khóa. Đồng thời, hệ thống tích hợp chức năng gợi ý phòng trọ, đề xuất các phòng phù hợp dựa trên thông tin và lịch sử tương tác của người dùng, giúp nâng cao hiệu quả tìm kiếm.

Ngoài ra, trang chủ còn cung cấp thanh điều hướng đến các chức năng như đăng nhập, đăng ký, danh sách phòng trọ, tài khoản, nhấn tin và thông báo. Giao diện được tối ưu để hiển thị tốt trên nhiều loại thiết bị, mang lại trải nghiệm thuận tiện và dễ sử dụng cho người dùng.

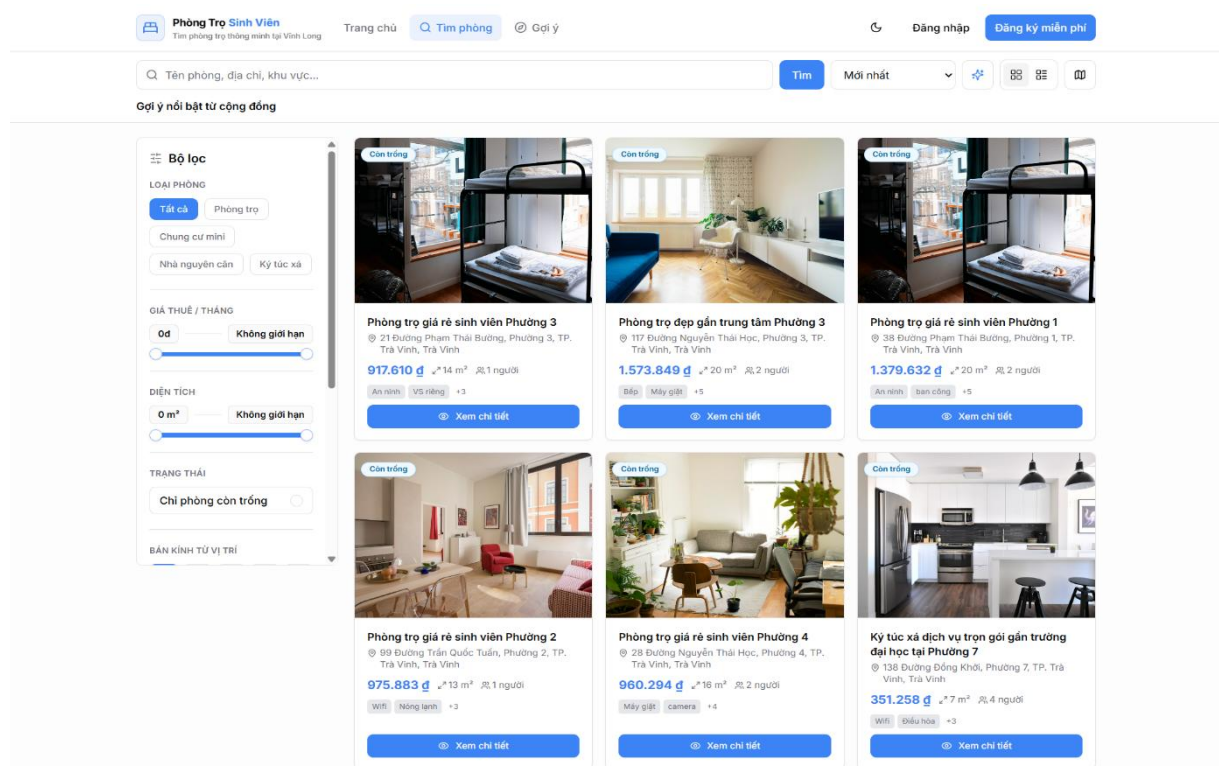


Hình 4.4. Giao diện trang chủ

4.3.2. Trang tìm kiếm phòng trọ

Trang tìm kiếm cho phép người dùng tìm kiếm phòng trọ dựa trên nhiều tiêu chí như giá thuê, diện tích, loại phòng, khu vực, địa chỉ và các tiện ích đi kèm. Người dùng có thể kết hợp nhiều bộ lọc để thu hẹp phạm vi tìm kiếm, từ đó nhanh chóng tìm được phòng trọ phù hợp với nhu cầu.

Kết quả tìm kiếm được hiển thị dưới dạng danh sách, trong đó mỗi phòng trọ cung cấp các thông tin cơ bản như hình ảnh, giá thuê, diện tích, địa chỉ và một số tiện ích nổi bật. Người dùng có thể dễ dàng so sánh các phòng trọ và lựa chọn xem thông tin chi tiết hoặc thực hiện các thao tác tiếp theo như lưu phòng hoặc liên hệ với chủ trọ.



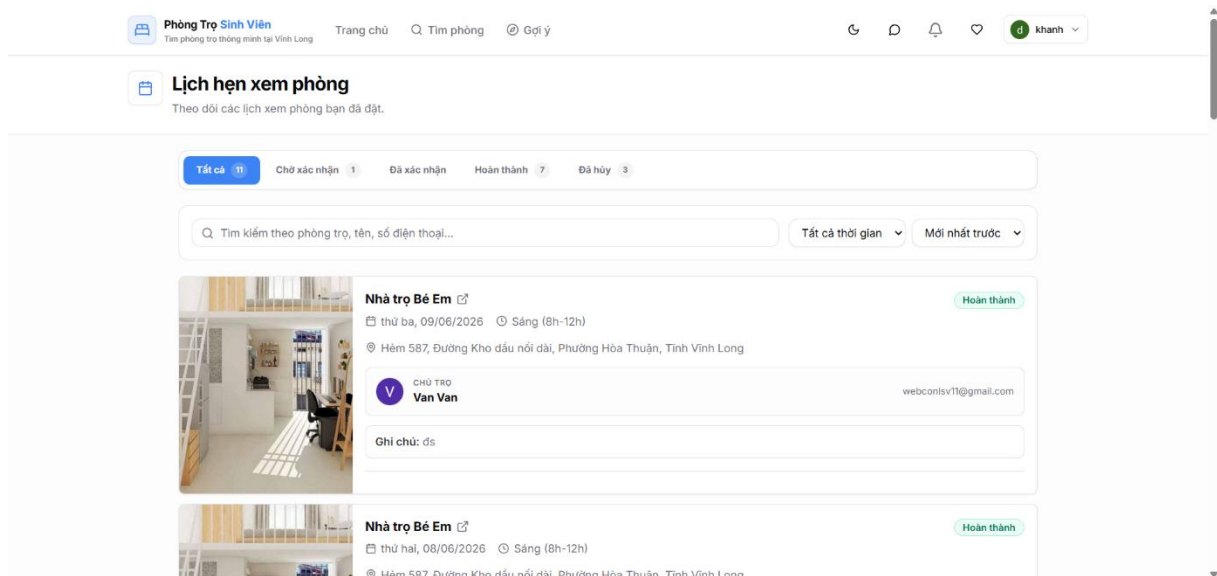
Hình 4.5. Giao diện trang tìm kiếm phòng trọ

4.3.3. Trang chi tiết phòng trọ

Trang chi tiết phòng trọ cung cấp đầy đủ thông tin liên quan đến một phòng trọ cụ thể, bao gồm hình ảnh minh họa, mô tả chi tiết, giá thuê, diện tích, các tiện ích đi kèm, vị trí hiển thị trên bản đồ và thông tin liên hệ của chủ trọ. Ngoài việc xem thông tin, người dùng còn có thể thực hiện các thao tác như lưu phòng trọ vào danh sách yêu thích, gửi tin nhắn trao đổi với chủ trọ hoặc đặt lịch hẹn để đến xem phòng trực tiếp.

4.3.4. Trang lịch hẹn xem phòng

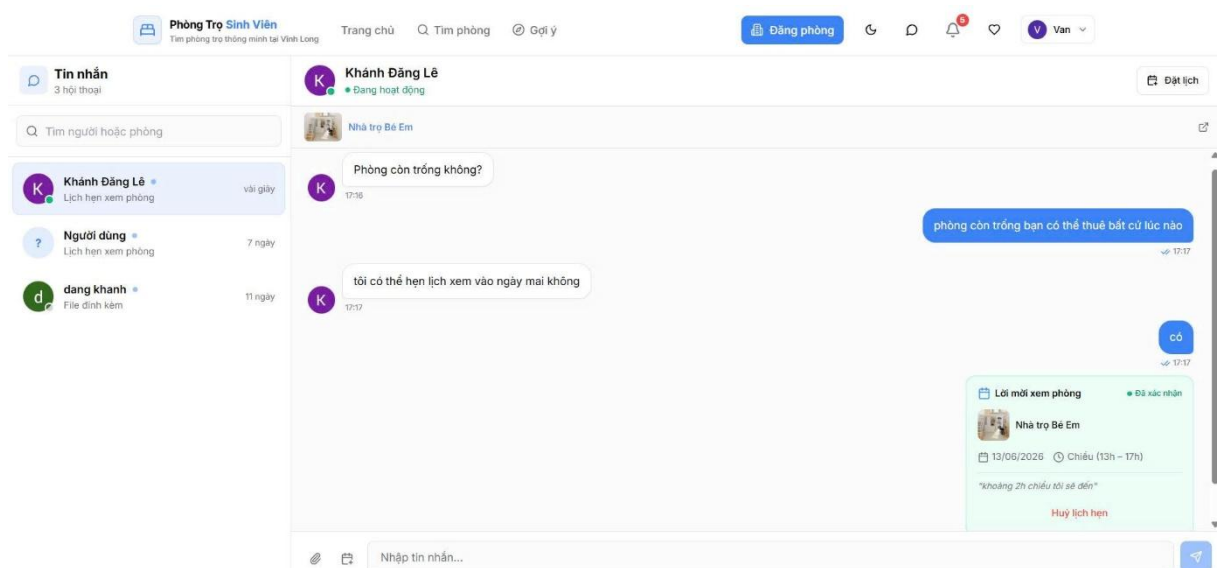
Trang lịch hẹn cho phép người dùng tạo và quản lý các lịch hẹn xem phòng với chủ trọ. Người dùng có thể theo dõi trạng thái lịch hẹn, thời gian hẹn và các thông tin liên quan.



Hình 4.7. Giao diện trang lịch hẹn xem phòng

4.3.5. Trang nhắn tin

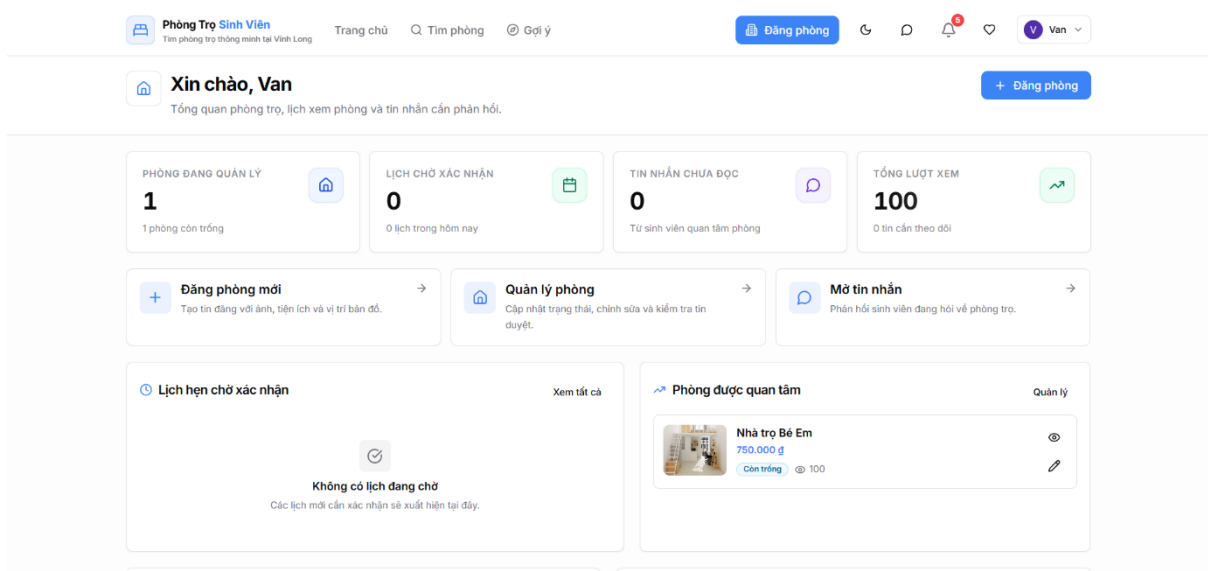
Trang nhắn tin hỗ trợ trao đổi trực tiếp giữa sinh viên và chủ trọ. Hệ thống cho phép gửi và nhận tin nhắn theo thời gian thực, giúp hai bên thuận tiện trong quá trình trao đổi thông tin về phòng trọ.



Hình 4.8. Giao diện trang nhắn tin

4.3.6. Trang quản lý phòng trọ

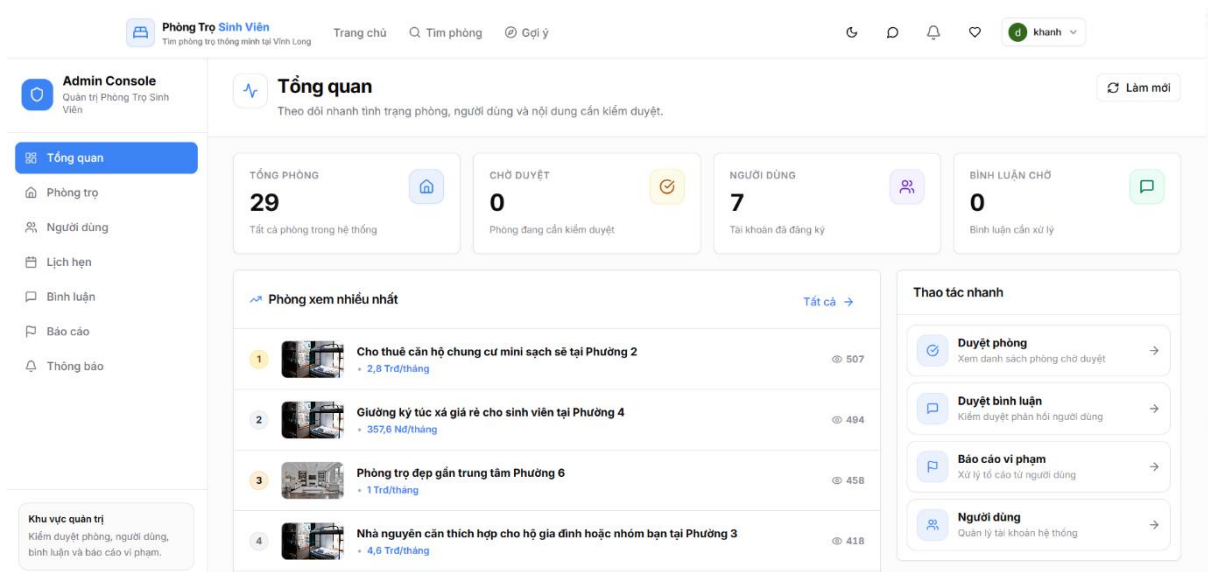
Trang quản lý phòng trọ dành cho chủ trọ. Chức năng này cho phép thêm mới, chỉnh sửa, cập nhật hoặc xóa thông tin phòng trọ đang đăng trên hệ thống. Chủ trọ cũng có thể theo dõi tình trạng hoạt động của các bài đăng.



Hình 4.9. Giao diện trang quản lý phòng trọ

4.3.7. Trang quản trị hệ thống

Trang quản trị hệ thống dành cho quản trị viên thực hiện các chức năng quản lý người dùng, quản lý phòng trọ, xử lý báo cáo vi phạm và theo dõi hoạt động chung của hệ thống. Đây là công cụ giúp đảm bảo hệ thống vận hành ổn định và hiệu quả.



Hình 4.10. Giao diện trang quản trị hệ thống

4.4. Kiểm thử hệ thống

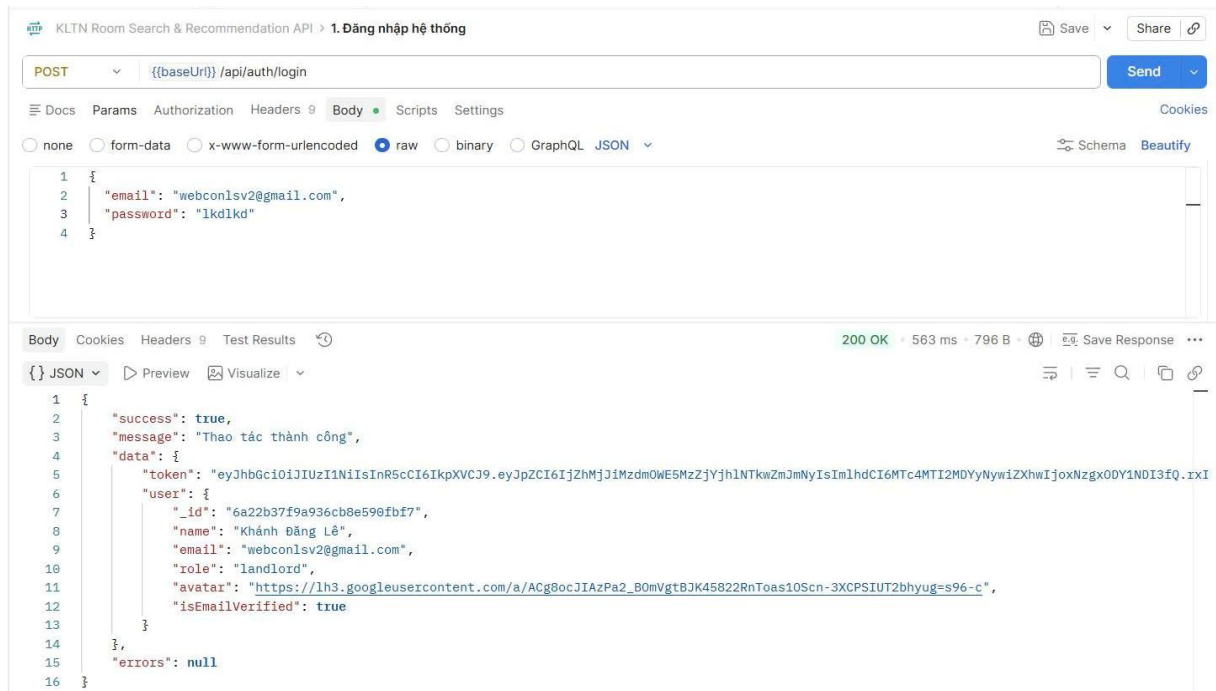
Sau khi hoàn thành quá trình xây dựng, hệ thống được tiến hành kiểm thử nhằm đánh giá tính chính xác của các chức năng Backend. Quá trình kiểm thử được thực hiện bằng công cụ Postman thông qua việc gửi các yêu cầu đến các API của hệ thống và kiểm tra dữ liệu phản hồi.

4.4.1. Kiểm thử chức năng

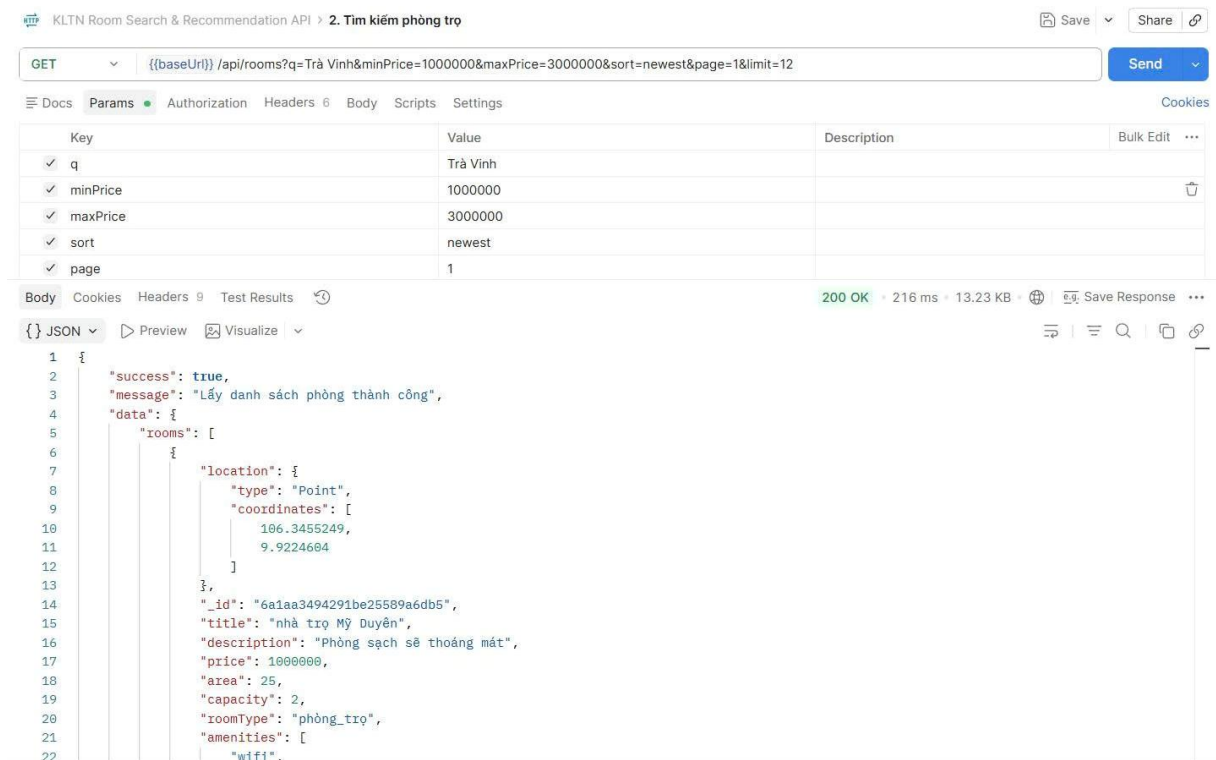
Các API chính của hệ thống được kiểm thử bao gồm đăng nhập, tìm kiếm phòng trọ, quản lý phòng yêu thích, đặt lịch hẹn, nhắn tin và hệ thống gợi ý phòng trọ. Kết quả kiểm thử cho thấy các API hoạt động ổn định, dữ liệu được xử lý chính xác và phản hồi đúng theo yêu cầu thiết kế.

Bảng 4.1. Kết quả kiểm thử chức năng

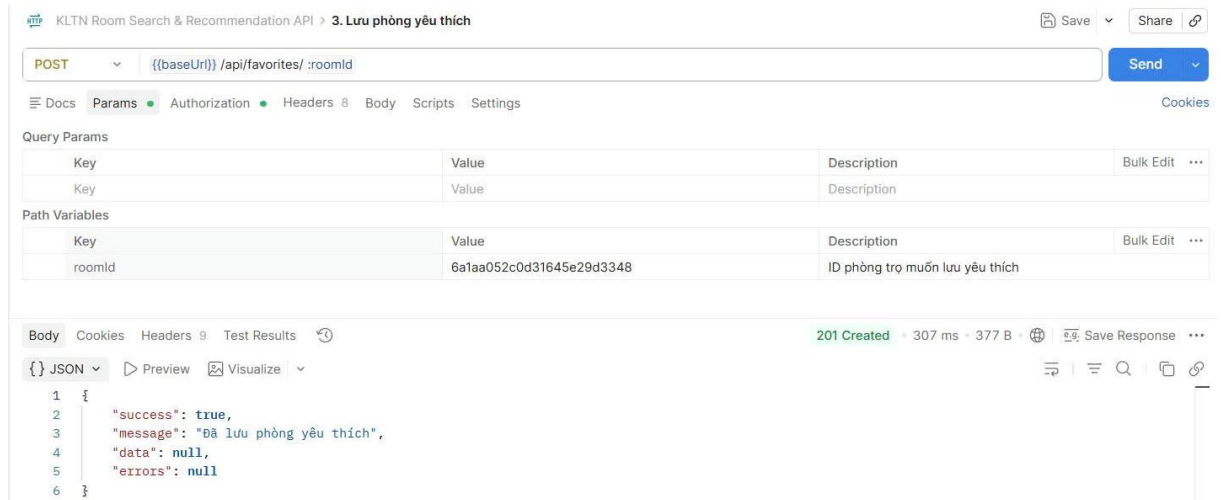
STT	Chức năng	API kiểm thử	Kết quả
1	Đăng nhập	POST /api/auth/login	Đạt
2	Tìm kiếm phòng trọ	GET /api/rooms?	Đạt
3	Lưu phòng yêu thích	POST /api/favorites/roomId	Đạt
4	Đặt lịch hẹn xem phòng	POST /api/appointments	Đạt
5	Gợi ý cá nhân hóa	GET /api/recommend/for-you	Đạt
6	Gợi ý phòng tương tự	GET /api/recommend/similar/roomId	Đạt
7	Gợi ý dựa trên hoạt động cộng đồng	GET /api/recommend/community	Đạt



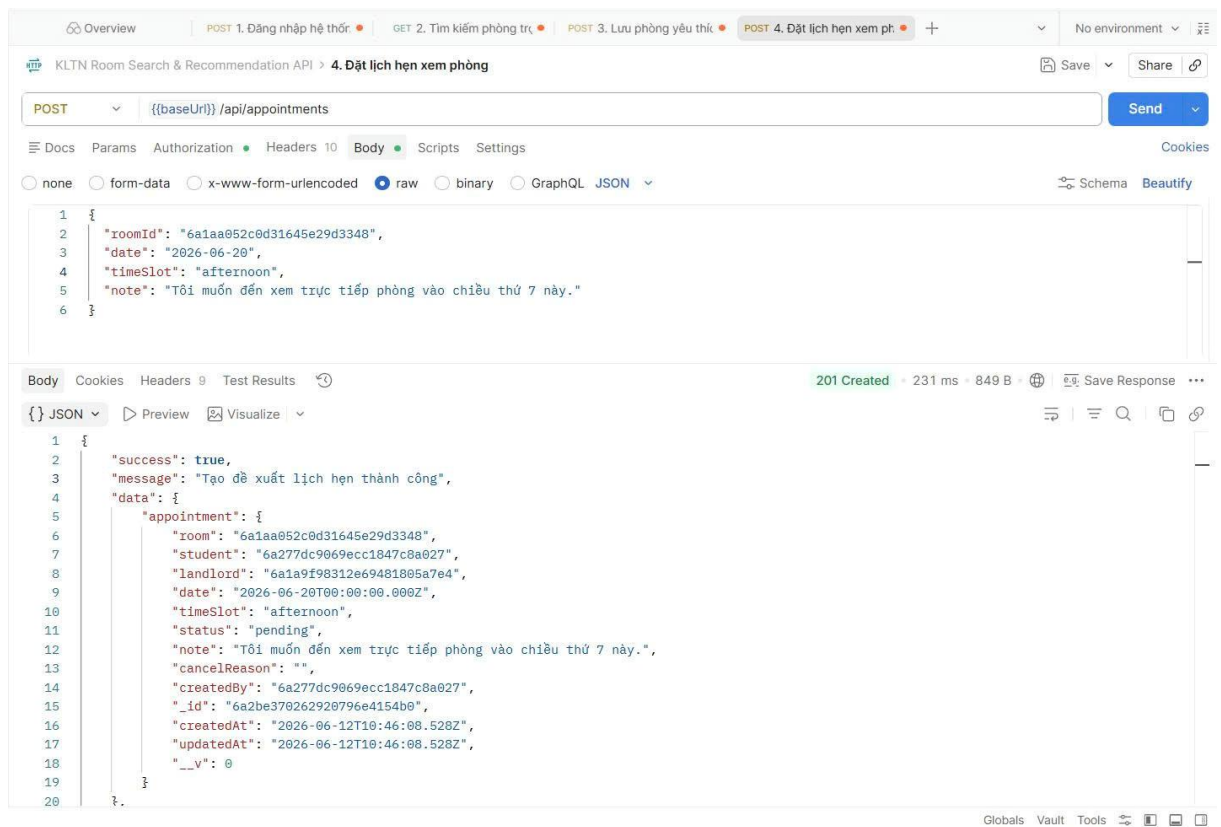
Hình 4.11. Kiểm thử API đăng nhập bằng Postman



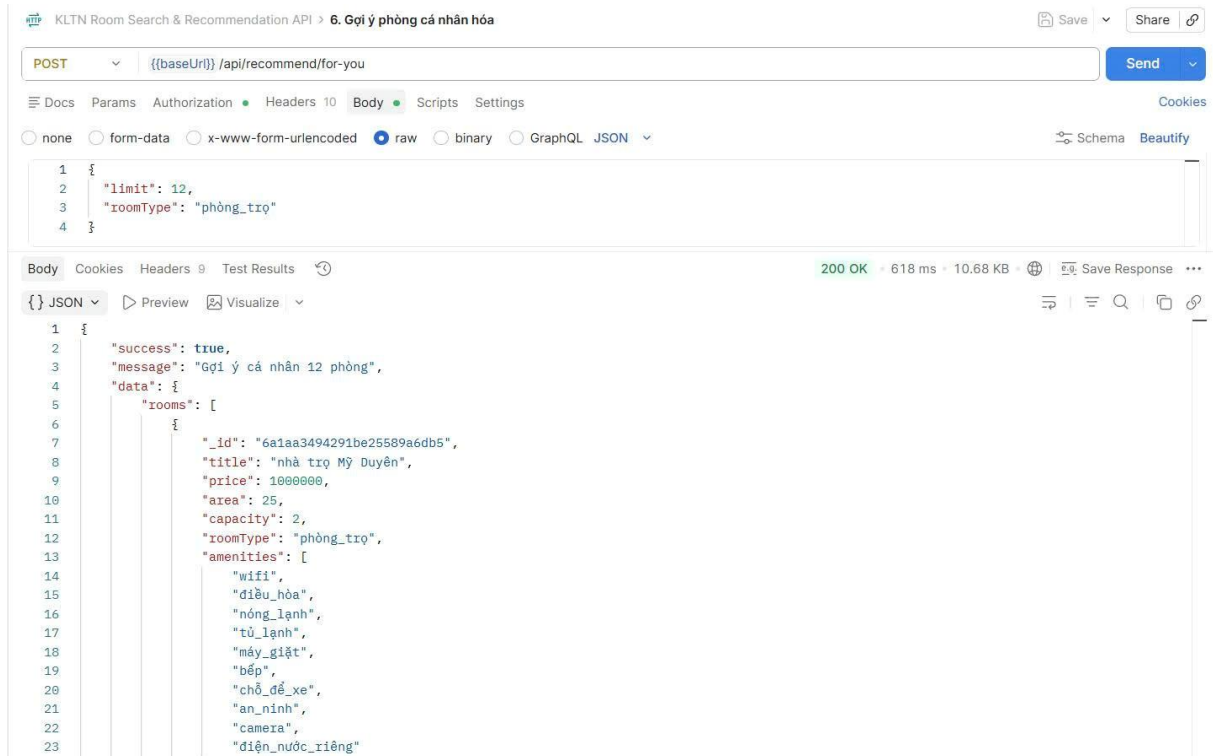
Hình 4.12. Kiểm thử API tìm kiếm phòng trọ bằng Postman



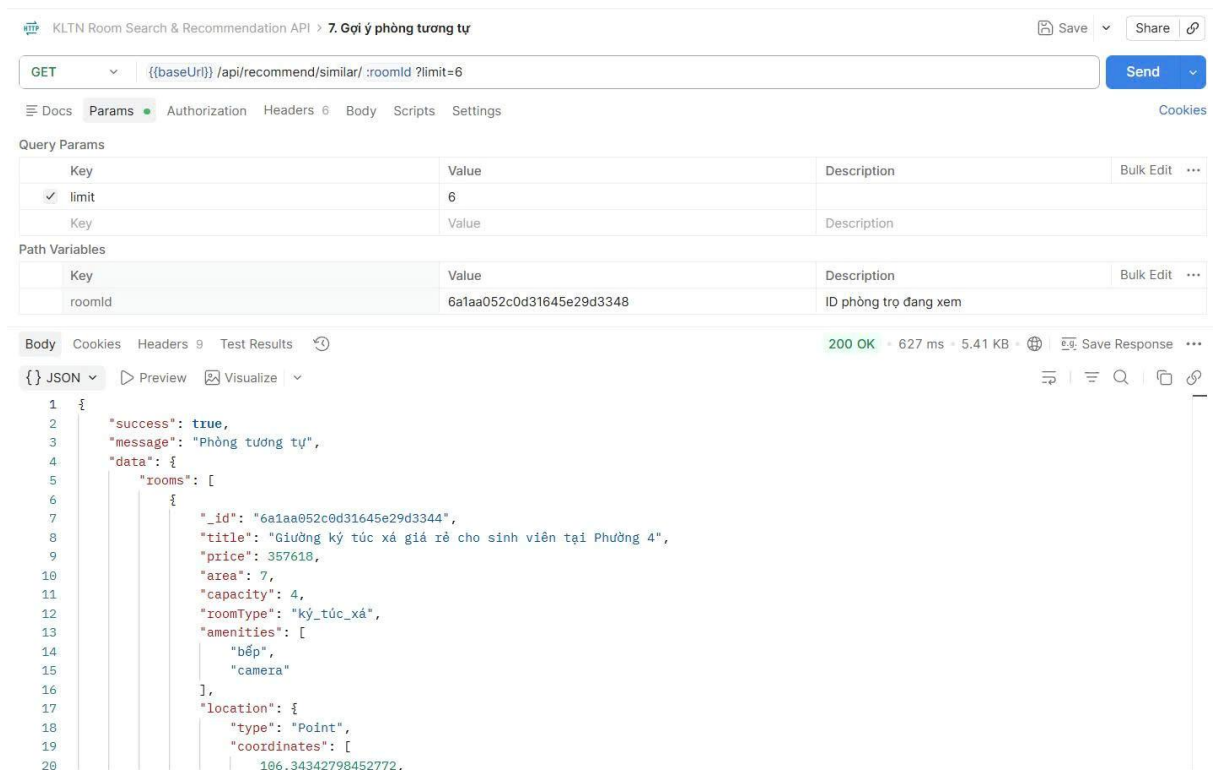
Hình 4.13. Kiểm thử API Lưu phòng yêu thích bằng Postman



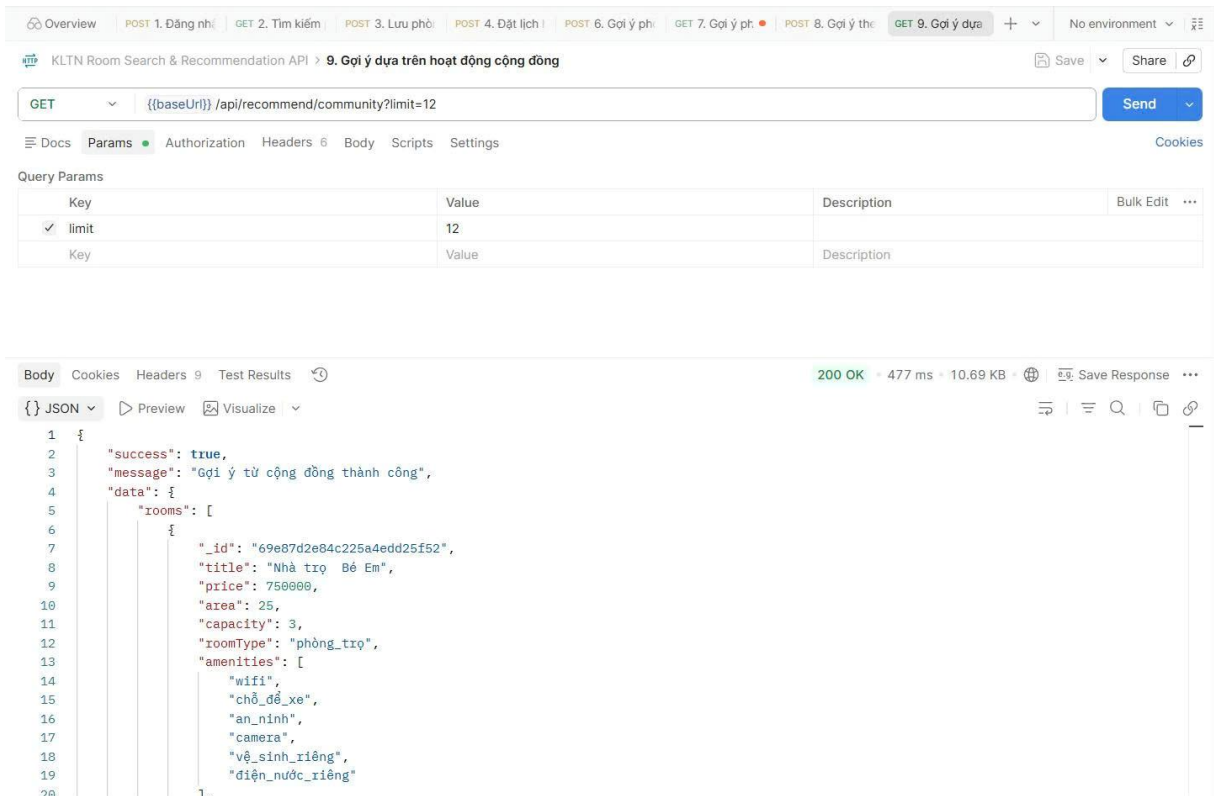
Hình 4.14. Kiểm thử API Đặt lịch hẹn xem phòng bằng Postman



Hình 4.15. Kiểm thử API Gợi ý cá nhân hóa bằng Postman



Hình 4.16. Kiểm thử API Gợi ý phòng tương tự bằng Postman



Hình 4.17. Kiểm thử API Gợi ý dựa trên hoạt động cộng đồng bằng Postman

Kết quả kiểm thử cho thấy các API đều trả về dữ liệu đúng định dạng, xử lý chính xác các yêu cầu và đảm bảo khả năng hoạt động ổn định của hệ thống.

4.4.2. Đánh giá kết quả kiểm thử

Qua quá trình kiểm thử, các chức năng chính của hệ thống đều hoạt động đúng theo yêu cầu đề ra. Các API phản hồi nhanh, dữ liệu được lưu trữ và truy xuất chính xác từ cơ sở dữ liệu MongoDB. Hệ thống gợi ý phòng trọ hoạt động ổn định và cung cấp các kết quả phù hợp với từng ngữ cảnh sử dụng của người dùng.

CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1. Kết quả đạt được

Sau quá trình nghiên cứu và triển khai, đề tài đã xây dựng thành công hệ thống hỗ trợ tìm kiếm và gợi ý phòng trọ cho sinh viên trên nền tảng web sử dụng ReactJS, Node.js, ExpressJS và MongoDB.

Hệ thống cung cấp đầy đủ các chức năng cơ bản như quản lý tài khoản, quản lý phòng trọ, tìm kiếm và lọc phòng trọ, lưu phòng yêu thích, đặt lịch hẹn xem phòng, nhắn tin giữa sinh viên và chủ trọ, thông báo và quản trị hệ thống.

Bên cạnh đó, đề tài đã xây dựng thành công hệ thống gợi ý phòng trọ dựa trên mô hình Hybrid Recommendation. Hệ thống kết hợp dữ liệu phòng trọ, vị trí địa lý và lịch sử tương tác của người dùng để đưa ra các đề xuất phù hợp với nhu cầu thực tế. Các chức năng gợi ý bao gồm gợi ý phòng tương tự, gợi ý theo tiêu chí tìm kiếm, gợi ý cá nhân hóa và gợi ý cho người dùng mới.

Ngoài ra, hệ thống còn tích hợp bản đồ số giúp hiển thị vị trí phòng trọ, hỗ trợ tìm kiếm theo khu vực và tính toán khoảng cách giữa người dùng với phòng trọ. Kết quả thực nghiệm cho thấy hệ thống hoạt động ổn định, đáp ứng các yêu cầu chức năng và hỗ trợ hiệu quả quá trình tìm kiếm phòng trọ của sinh viên.

5.2. Hạn chế của hệ thống

Mặc dù đã đạt được các mục tiêu đề ra, hệ thống vẫn còn một số hạn chế nhất định. Dữ liệu sử dụng trong quá trình thực nghiệm còn hạn chế về quy mô nên chưa đánh giá đầy đủ hiệu quả hoạt động của hệ thống trong môi trường thực tế với số lượng lớn người dùng và phòng trọ.

Bên cạnh đó, chất lượng gợi ý vẫn phụ thuộc vào dữ liệu tương tác của người dùng. Đối với người dùng mới hoặc có ít lịch sử hoạt động, mức độ cá nhân hóa của kết quả gợi ý chưa thực sự tối ưu. Ngoài ra, hệ thống hiện chưa áp dụng các mô hình học máy hiện đại để nâng cao khả năng dự đoán và tối ưu kết quả đề xuất.

Hệ thống hiện mới được triển khai trên nền tảng web và chưa phát triển ứng dụng dành cho thiết bị di động. Một số chức năng nâng cao như đánh giá phòng trọ, xếp hạng chủ trọ và phân tích dữ liệu thống kê vẫn chưa được triển khai trong phạm vi đề tài.

5.3. Hướng phát triển

Trong tương lai, hệ thống có thể được mở rộng bằng cách tăng quy mô dữ liệu và triển khai trên môi trường thực tế nhằm đánh giá toàn diện hơn hiệu quả hoạt động.

Bên cạnh đó, hệ thống gợi ý có thể được nâng cấp thông qua việc áp dụng các kỹ thuật Machine Learning và Deep Learning nhằm nâng cao độ chính xác và khả năng cá nhân hóa của các đề xuất. Đồng thời, việc bổ sung dữ liệu đánh giá và phản hồi từ người dùng sẽ góp phần cải thiện chất lượng gợi ý.

Ngoài ra, có thể phát triển ứng dụng trên nền tảng Android và iOS, bổ sung chức năng đánh giá và xếp hạng phòng trọ, đồng thời tiếp tục tối ưu hiệu năng, bảo mật và trải nghiệm người dùng nhằm nâng cao khả năng ứng dụng của hệ thống trong thực tế.

DANH MỤC TÀI LIỆU THAM KHẢO

- [1] React Team (2026), *React Documentation*. Available: <https://react.dev/>
- [2] Meta Open Source (2026), *React*. Available: <https://opensource.fb.com/projects/react/>
- [3] Node.js (2026), *Node.js Documentation*. Available: <https://nodejs.org/docs/latest/api/>
- [4] Express.js (2026), *Express.js Documentation*. Available: <https://expressjs.com/>
- [5] MongoDB Inc. (2026), *MongoDB Documentation*. Available: <https://www.mongodb.com/docs/>
- [6] Mongoose (2026), *Mongoose Documentation*. Available: <https://mongoosejs.com/docs/>
- [7] Socket.IO (2026), *Socket.IO Documentation*. Available: <https://socket.io/docs/v4/>
- [8] Leaflet (2026), *Leaflet Documentation*. Available: <https://leafletjs.com/>
- [9] OpenStreetMap Foundation (2026), *OpenStreetMap Wiki*. Available: <https://wiki.openstreetmap.org/>
- [10] Scikit-learn Developers (2026), *cosine_similarity*. Available: https://scikit-learn.org/stable/modules/generated/sklearn.metrics.pairwise.cosine_similarity.html
- [11] Veness, C. (2026), *Calculate distance, bearing and more between Latitude/Longitude points*. Available: <https://www.movable-type.co.uk/scripts/latlong.html>
- [12] Ricci, F., Rokach, L., Shapira, B., and Kantor, P. B. (2015), *Recommender Systems Handbook*, 2nd ed., Springer, New York